

第 4 章

處理器

4.1 介紹

4.2 邏輯設計慣例

4.3 建構數據通道

4.4 簡單實作方案

4.5 管線化處理概觀

4.6 管線化數據通道及控制

4.7 數據危障：前饋或停滯

4.8 控制危障

~~4.9 例外~~

4.10 平行性與進階的指令階層平行性

4.11 實例：ARM Cortex-A8 與 Intel Core i7 的管線

~~4.12 走得更快：指令階層平行性與矩陣乘法~~

4.14 謬誤與陷阱

4.15 總結評論

4.1 介紹 <Review: $Time = IC \times CPI \times Clock\ Cycle$ >

- ▣ 第一章說明了計算機的效能是由下列三個主要的因素所決定：指令個數、時脈週期時間以及平均每個指令的時脈週期數(CPI)
- ▣ 第二章說明了編譯器和指令集架構決定一個程式需要的指令個數
- ▣ 然而，處理器的實作決定時脈週期時間和平均每個指令的時脈週期數兩者
 - 本章中，我們對MIPS 指令集的兩種不同實作方式建構其數據通道(datapath)和控制單元(control unit)

基本的MIPS 實作

▣ 探討MIPS 核心指令集中部分指令的實作：

- 載入字組(lw)和儲存字組(sw)
- add、sub、AND、OR 和slt
- 若相等則分支(beq)以及跳躍指令(j)

mem reference: lw, sw
Arith. CMD: add, sub, and, or
flow Control: beq, j, slt

▣ 將可看到

- 指令集架構如何決定實作上的許多方面
- 不同實作策略如何影響計算機的時脈速率以及CPI

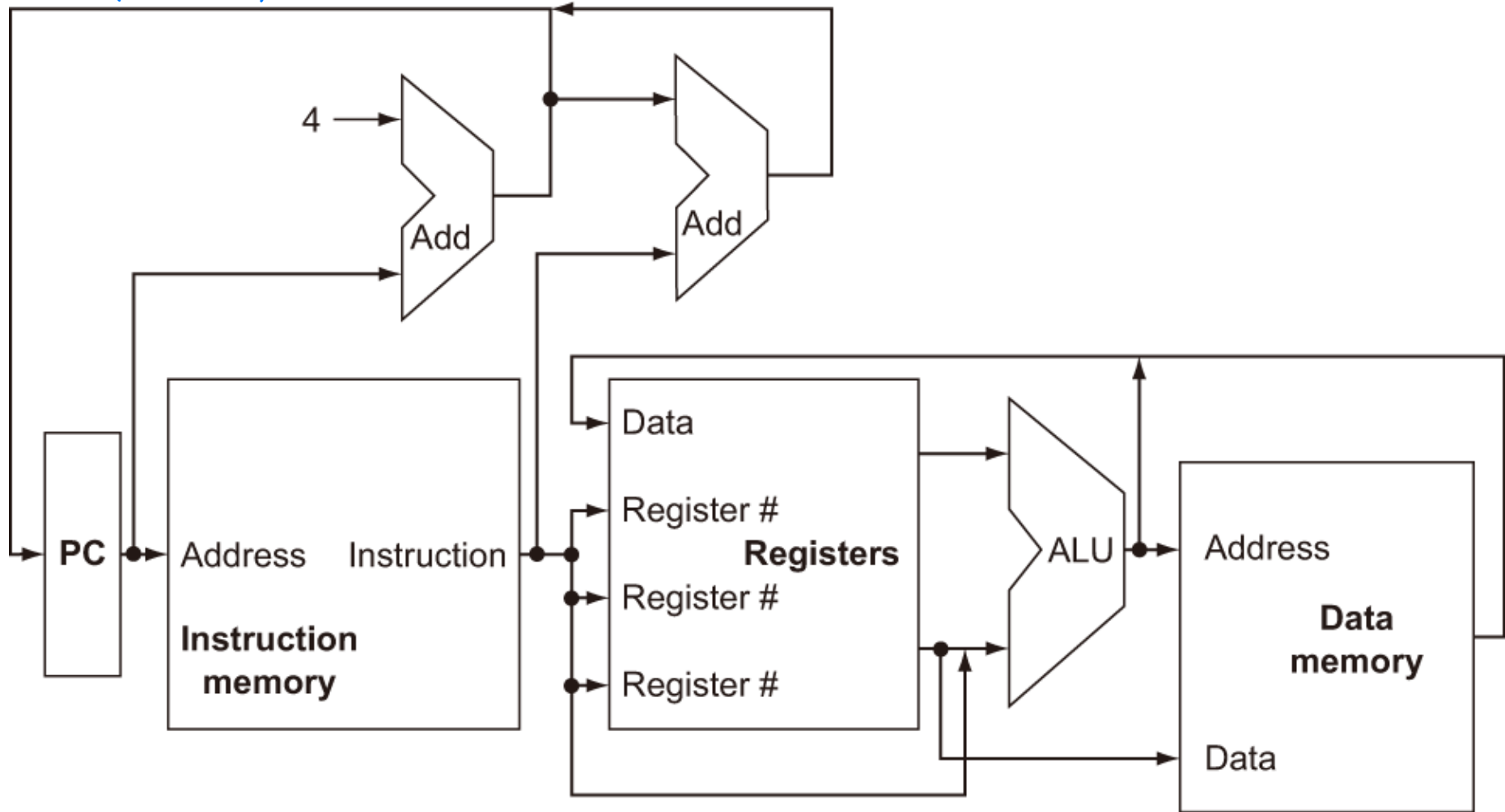
實作的概觀

- 在實作這些不同類別的指令時，大部分內容都是相同的。每個指令剛開始的兩個步驟都一樣：
 1. 將程式計數器(PC)的內容送到含有程式碼的記憶體，並且從記憶體中擷取指令
 2. 使用指令中的欄位來選擇並讀取一個或兩個暫存器的內容。我們只需對載入字組指令讀取一個暫存器，然而大部分其他指令則需讀取兩個暫存器

實作的概觀

- 在這兩個步驟之後，用來完成指令的動作與指令類別有關
 - 對三種指令類別(記憶體存取、算術邏輯及分支)的每一種而言，所需動作大致上還是相同的
- 圖4.1 以高階的觀點顯示MIPS 中不同的功能單元和它們之間的聯結的實作圖

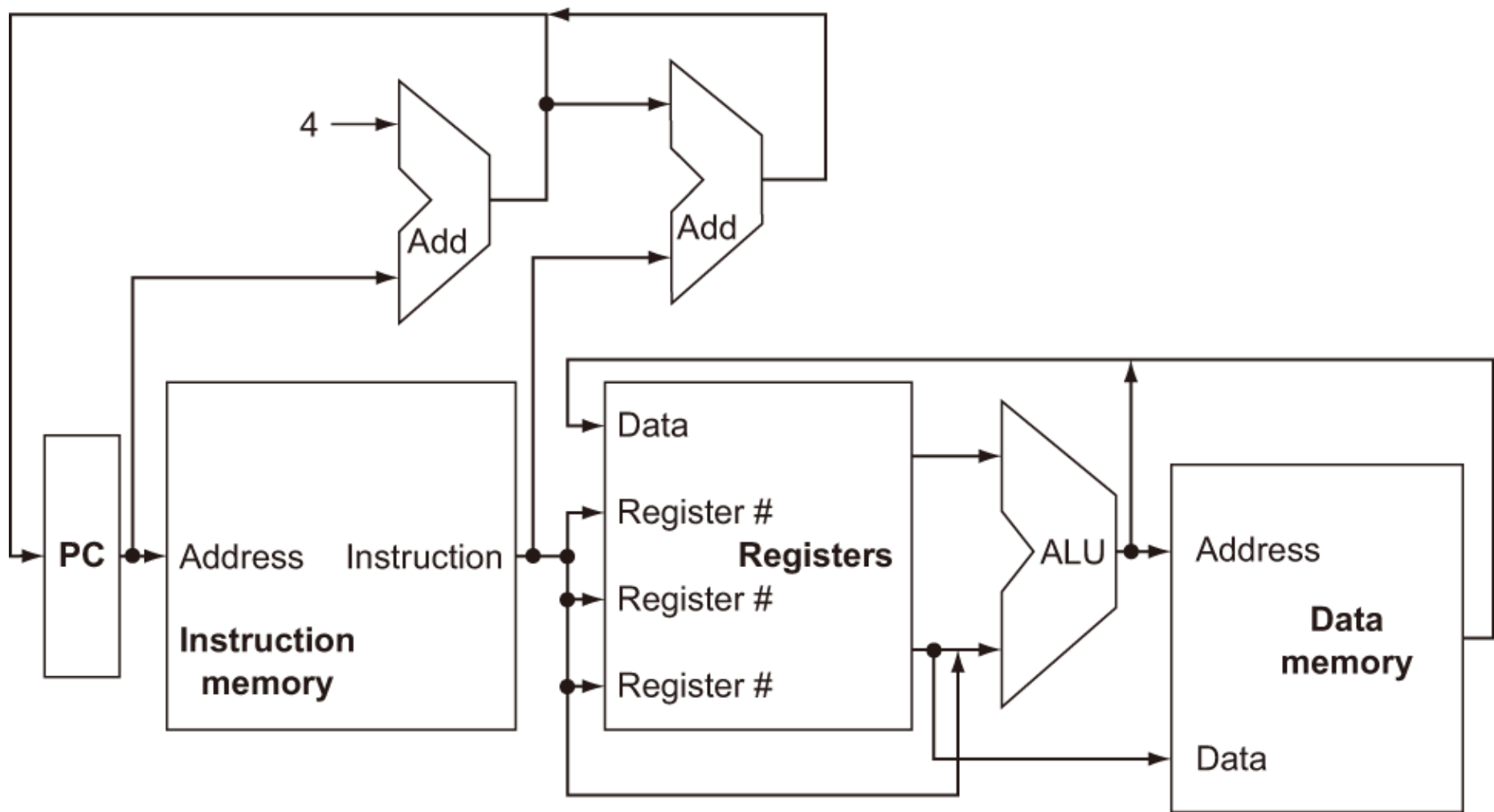
圖4.1 MIPS 指令集合實作的概觀圖，圖中表示了各主要功能單元和它們之間的主要連線關係
(Data Path)



op	rs	rt	rd	shamt	funct
----	----	----	----	-------	-------

6 個位元 5 個位元 5 個位元 5 個位元 5 個位元 6 個位元

R-type 指令

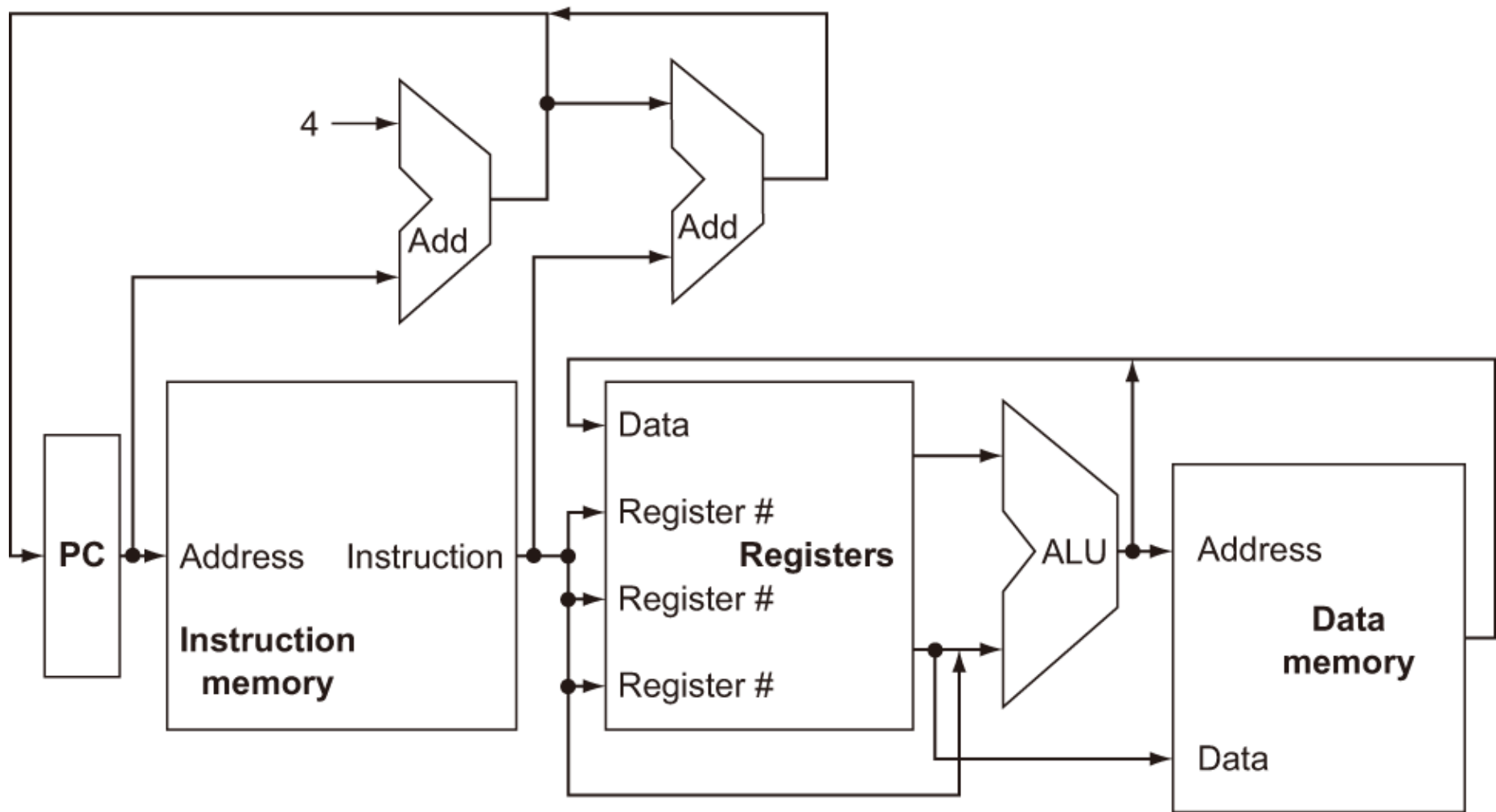


op	rs	rt	常數或位址
----	----	----	-------

6 個位元 5 個位元 5 個位元

16 個位元

I-type指令



2	10000
---	-------

6 個位元

26 個位元

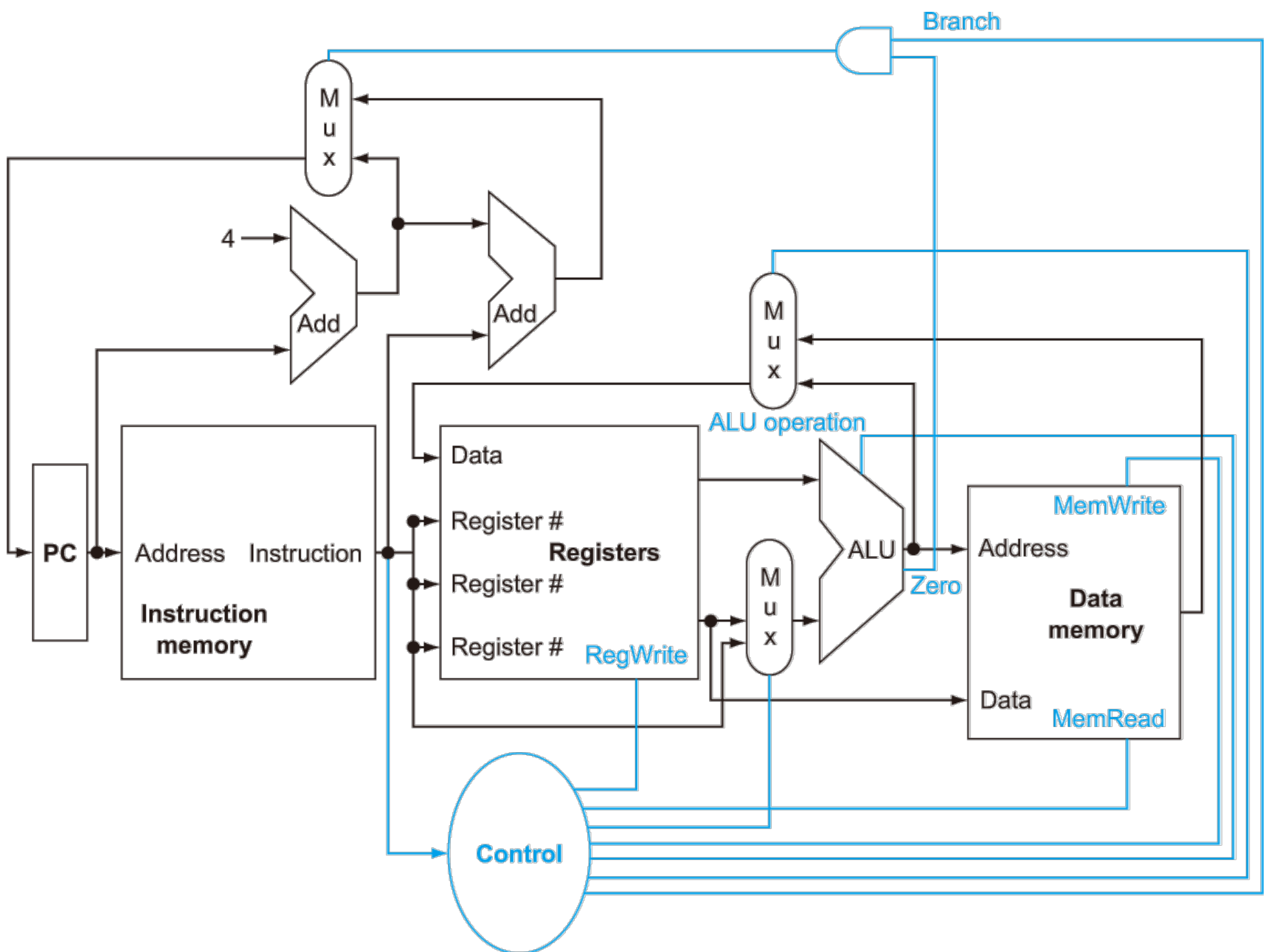
J-type指令

圖4.1 MIPS 指令集合實作的概觀圖，圖中表示了各主要功能單元和它們之間的主要連線關係

所有指令都是由使用程式計數器(PC)來提供指令位址給指令記憶體開始。在指令擷取進來之後，指令中的欄位指出指令所使用到的暫存器運算元。一旦暫存器運算元被擷取完成，則這些運算元可以被用來計算記憶體的位址(用於載入或儲存指令)、計算算術結果(用於整數的算術邏輯指令)或是比較(用於條件分支指令)。如果指令是算術邏輯指令，則必須將ALU 的運算結果寫到暫存器中。如果是載入或儲存指令，則ALU 的結果被當成記憶體位址來儲存一個暫存器的值到記憶體中或是從記憶體載入一個值到暫存器中。從ALU 或記憶體得到的結果會被寫回暫存器檔案。條件分支指令使用ALU 的輸出來決定下一道指令的位址，這個位址要不是來自一個加法器(將目前的PC 值+4 和條件分支的偏移量相加)就是來自另一個加法器的結果(將目前的PC 值累加4)。將各功能單元相連的粗體線條代表匯流排(bus)，其包含多條訊號線。使用箭頭讓讀者瞭解資料的流向。因為訊號線有可能會相互交錯，所以我在訊號線交錯的地方使用黑點來表示交錯的訊號線是相連的。

實作的概觀

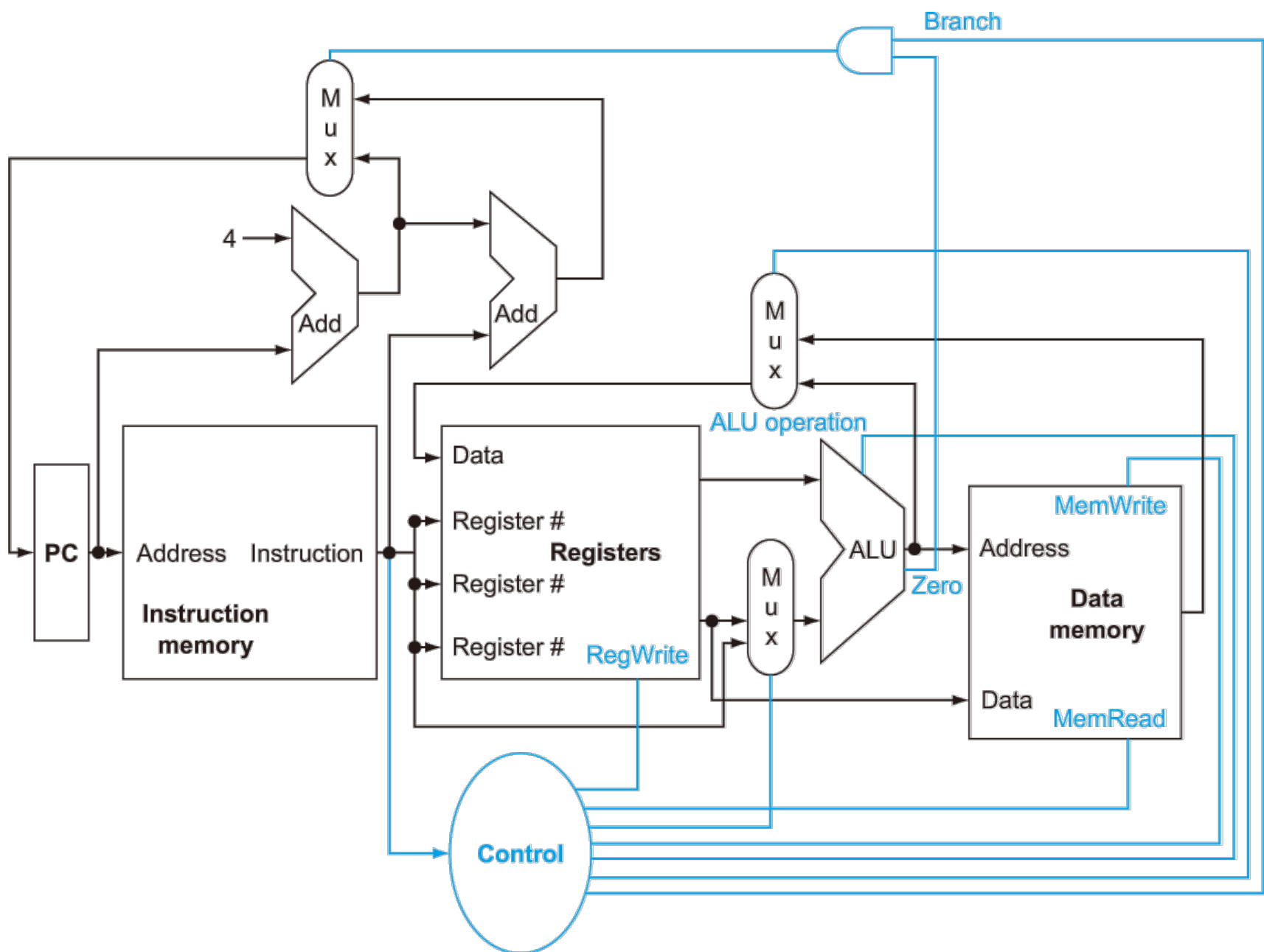
- 圖4.2 顯示圖4.1 加上三個所需的多工器以及主要功能單元的控制訊號後的數據通道
 - 每一道指令從一個時脈的邊緣開始執行並且於下一個時脈的邊緣完成
 - 這樣的設計方式並不實際
 - 因為時脈必須被拉長以容納最慢的指令



op	rs	rt	rd	Shamt	funct
----	----	----	----	-------	-------

6 個位元 5 個位元 5 個位元 5 個位元 5 個位元 6 個位元

R-type指令



op	rs	rt	常數或位址
----	----	----	-------

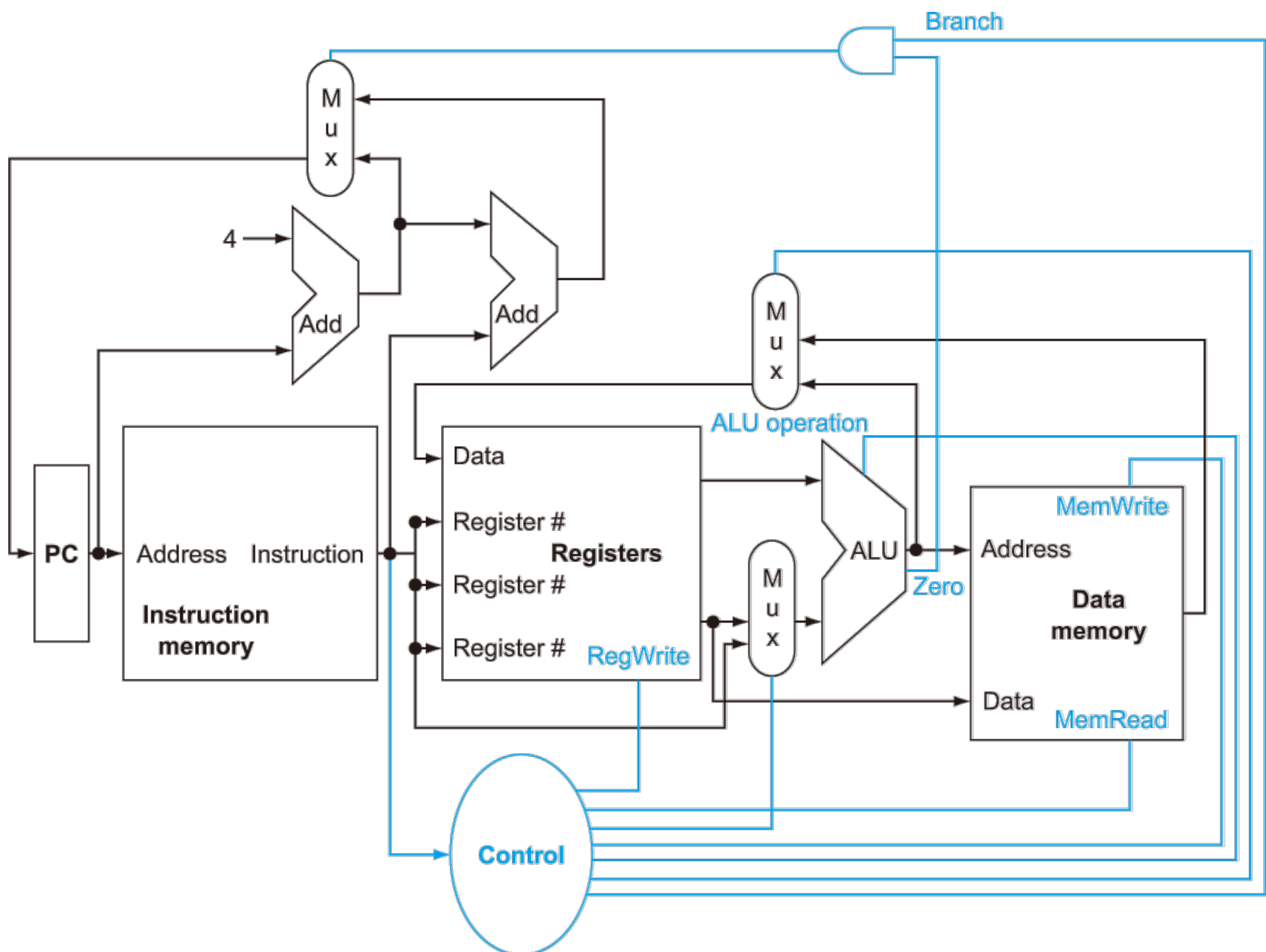
6 個位元

5 個位元

5 個位元

16 個位元

I-type指令



2	10000
---	-------

6 個位元

26 個位元

J-type指令

圖4.2 MIPS 子集合的基本實作，包括所需的多工器和控制訊號線

最上面的多工器控制PC的內容由哪一個值取代(PC+4 或是條件分支目的位址)；這一個多工器是由兩條訊號線經過AND閘後的結果來控制的，而這兩條訊號線是ALU的Zero輸出訊號線以及指出現在執行的指令是分支指令的控制訊號線。圖中間的多工器用來選擇是由ALU的輸出(在算術邏輯指令的情況下)或是數據記憶體輸出(在載入指令的情況下)寫回暫存器檔案中。最後，最下面的多工器用來決定ALU的第二個輸入是來自暫存器(在算術邏輯指令或分支的情況下)或是指令的偏移量欄位(在載入或儲存指令的情況下)。新增的控制訊號線淺而易懂，用來決定ALU要執行的動作、數據記憶體應該被讀取或寫入、或是暫存器要不要執行寫入的動作。我們將控制訊號線的部分著灰色，讓它們更容易被辨識

4.2 邏輯設計慣例

- ▣ 數據通道元件有兩種不同形式的邏輯元件：數值運算元件和狀態儲存元件
 - 數值運算元件全然屬於組合(combinationa)邏輯
 - ALU 就是一個組合邏輯元件的例子
 - 狀態元件(state elements)可以儲存狀態
 - 這些狀態元件完全地說明了計算機的現況
 - 指令記憶體、數據記憶體以及暫存器都是屬於狀態元件
 - 必要的輸入包括要寫入元件的數據內容以及決定何時將數據寫入的時脈

4.2 邏輯設計慣例

- 輸出則是在其之前被寫入的值
- 因為它們的輸出與它們的輸入以及內部狀態均有關而稱之為序向的(sequential)

時脈致動方法

- 時脈致動方法(clocking methodology)定義了訊號何時可以被讀取以及何時可以被寫入
 - 設計來保證事情的可預期性
 - 為了簡單起見，採用一種邊緣觸發(edge-triggered)的時脈致動方法
- 圖4.3 顯示了兩個狀態元件包圍著一個組合邏輯的區塊
 - 該組合邏輯區塊應在一個時脈週期內運作完成

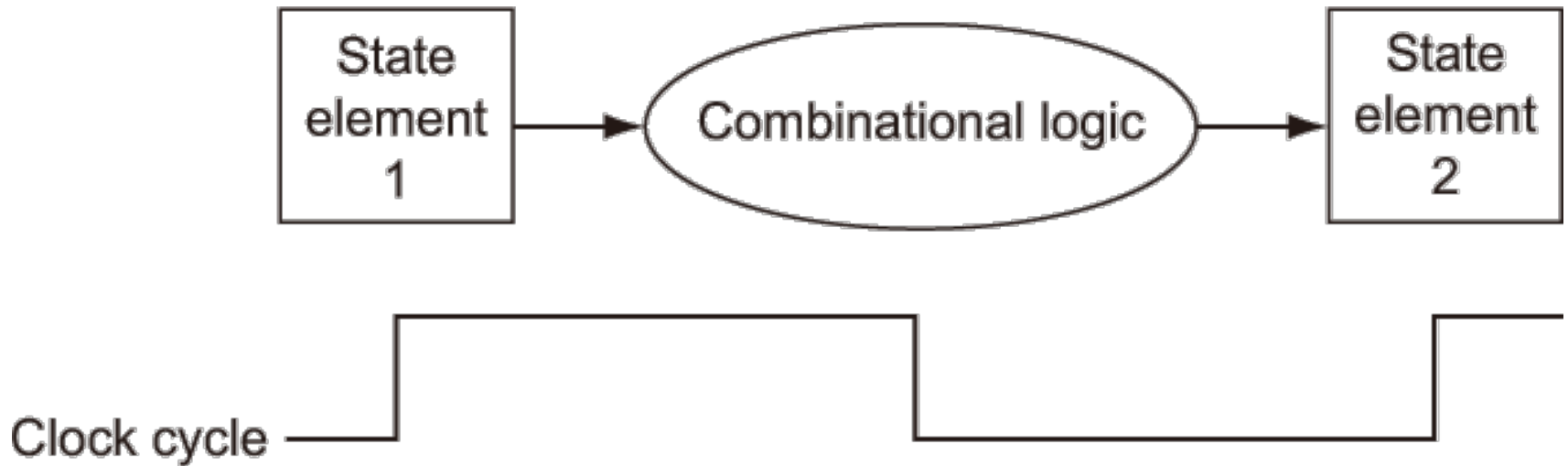


圖4.3 組合邏輯、狀態元件和時脈之間關係密切

在一個同步的數位系統中，時脈決定了狀態元件何時將值寫入內部的儲存體。任何狀態元件的輸入值在致動的時脈邊緣造成狀態被更新之前，都必須是穩定的(也就是說，到達的值必須一直維持穩定不變到時脈邊緣之後)。本章中所有的狀態元件(包括記憶體)都假設是邊緣觸發的

時脈致動方法

- 邊緣觸發的方法使得我們可以在同一個時脈週期中讀取暫存器的內容，之後對這個值做組合邏輯運算，再寫回原本的暫存器中
 - 圖4.4為一個通則性的例子
 - 採用邊緣觸發的時序方法時，在單一時脈週期內不會發生反饋(feedback)的情況

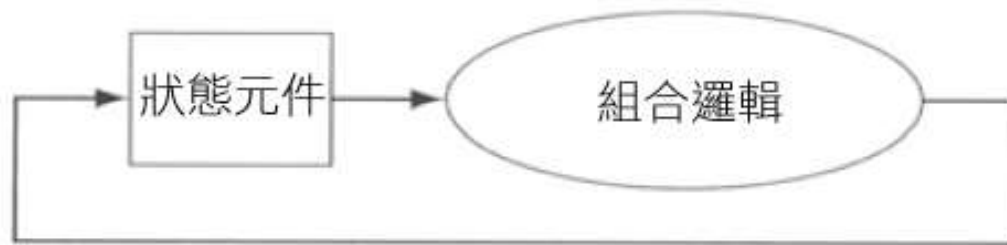


圖4.4 邊緣觸發的方法允許狀態元件同時在一個時脈週期內被讀取和寫入，而不會造成可能會產生不確定值的競賽(**race**)情況

當然，時脈週期仍然必須夠長到當致動時脈邊緣發生時，輸入的值已經穩定。因為狀態元件的更新是邊緣觸發的，所以時脈週期內並不會發生反饋的情況。如果反饋的情況可能發生的話，這樣的設計就無法正確地運作。我們在本章和下一章中的設計都是依靠如本圖所示的邊緣觸發方法及相同的結構

4.3 建構數據通道

- 先來看看每個指令需要何種數據通道元件(datapath elements)
- 圖 4.6 表示如何將圖 4.5 中的三個元件組合成一個可以擷取指令並且遞增PC 值以得知下一個連續指令位址的數據通道
- 現在讓我們來思考 R 格式的指令(見圖2.18)

圖2.18 MIPS 指令格式

名稱	欄 位						註 解
欄位大小	6 位元	5 位元	5 位元	5 位元	5 位元	6 位元	所有 MIPS 指令的長度都是 32 位元
R 格式	op	rs	rt	rd	shamt	funct	算術指令格式
I 格式	op	rs	rt	位址／立即值			傳輸、分支、imm 格式
J 格式	op	目標位址					跳躍指令格式

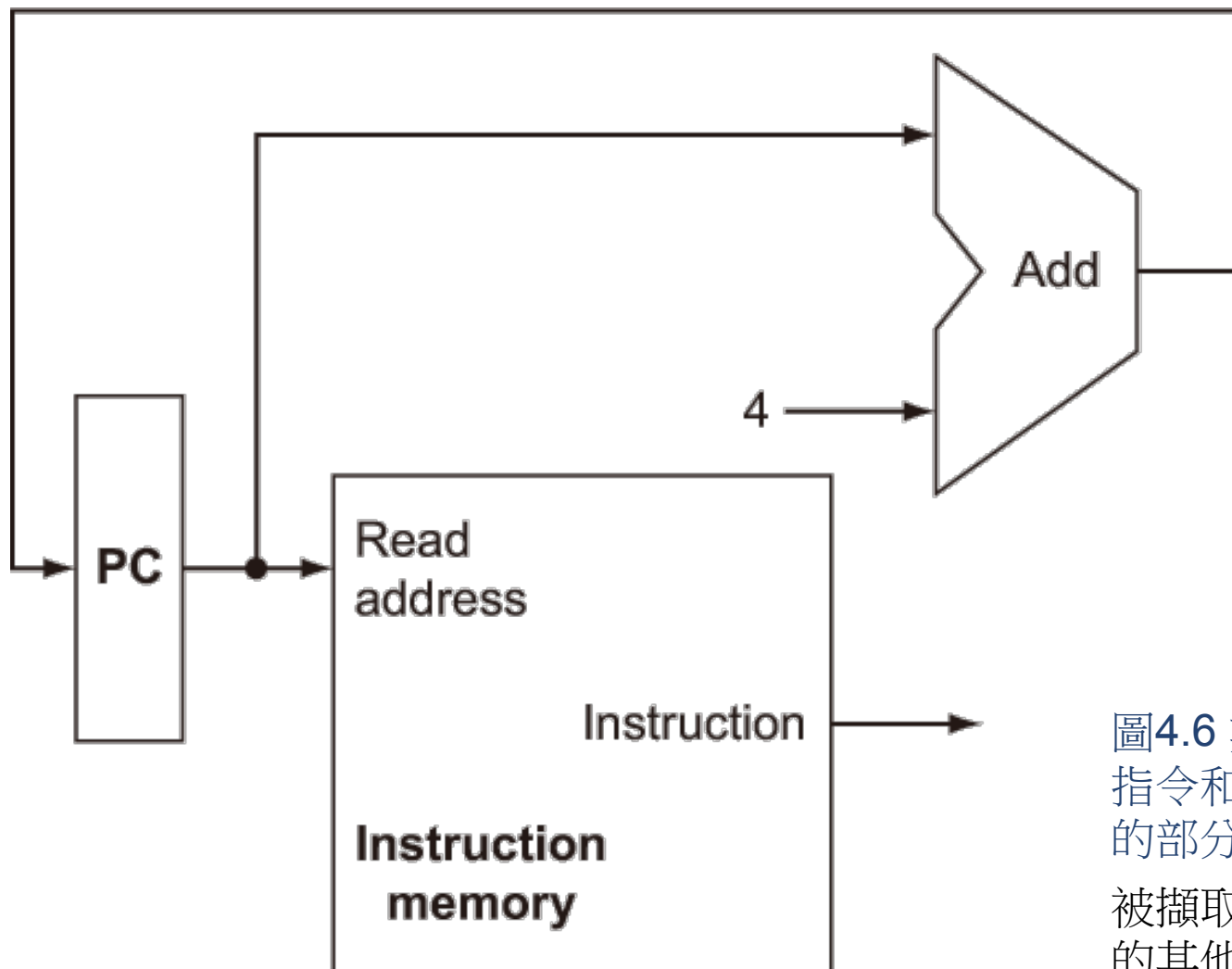
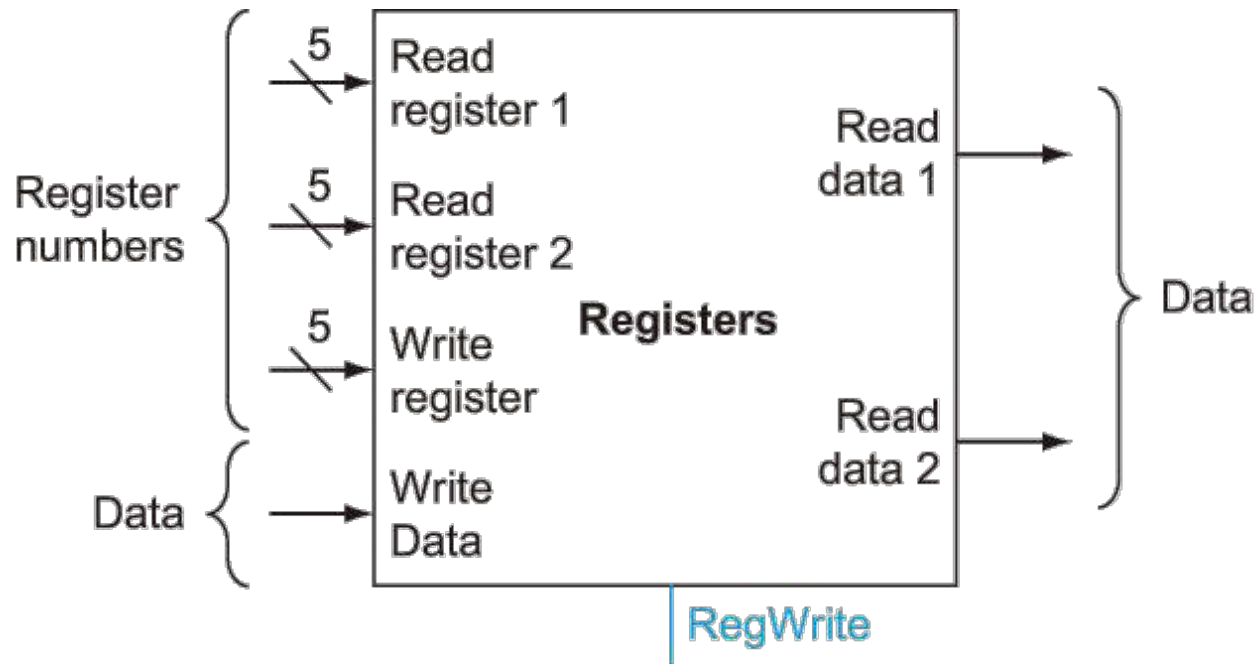
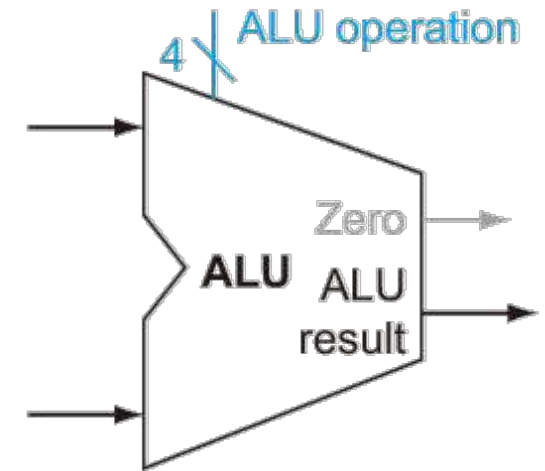


圖4.6 數據通道中用來擷取指令和遞增程式計數器(PC)的部分
被擷取的指令會被數據通道的其他部分使用

圖4.7 實作R 格式的ALU 運作所需的兩個元件是暫存器檔案和ALU



a. Registers



b. ALU

圖4.7 實作R 格式的ALU 運作所需的兩個元件是暫存器檔案和ALU

暫存器檔案包含了所有的暫存器並且有兩個讀取埠和一個寫入埠。暫存器檔案會一直將對應於**Read register**輸入埠的暫存器內容輸出，而不需要其他的控制訊號。相對地，暫存器的寫入就必須要明確地由寫入控制訊號來控制。記得寫入的動作屬於邊緣觸發，所以所有和寫入動作相關的輸入訊號(即被寫入的值、暫存器編號和寫入控制訊號)都必須在時脈邊緣時為有效。因為暫存器檔案的寫入動作屬於邊緣觸發，所以這種設計可以在一個時脈週期內對同一個暫存器讀取和寫入：讀取到的值是稍早的時脈週期所寫入的值，而寫入的值則可以在之後的時脈週期被讀取。暫存器檔案的暫存器編號輸入訊號都是**5** 個位元寬，而運送資料的訊號線都是**32** 個位元寬。要執行的ALU 運作是由**4** 位元ALU 運作控制訊號線來控制。我們稍後會使用ALU 的**Zero** 偵測輸出來實作條件分支(**branches**)。而ALU 的滿溢(**overflow**)輸出一直到**4.9** 節討論例外(**exceptions**)之前都還不需要，所以目前我們先忽略它

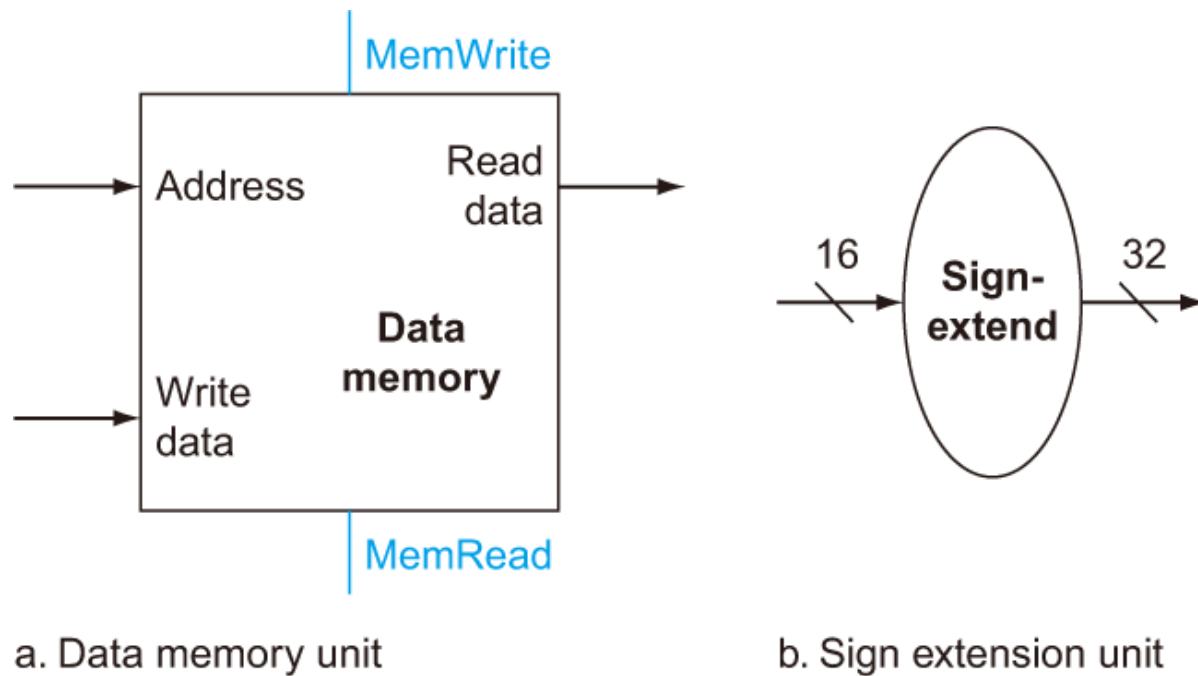


圖4.8 除了圖4.7 的暫存器檔案和ALU 外，實作載入指令和儲存指令還需要的兩個元件是數據記憶體單元和符號延伸單元

數據記憶體單元是一個具有位址輸入、寫入資料輸入和一個讀取結果輸出的狀態元件。它有個別的讀取和寫入控制訊號，即便在任何一個時脈中只有一條可能會被設定。記憶體單元需要讀取控制訊號，因為它並不像暫存器檔案，如果讀取一個無效位址的內容可能造成一些問題，如我們將會在第五章中所見。符號延伸單元具有一個 16 位元的輸入，經符號延伸成 32 位元的結果送到輸出上(見第二章)。我們假設數據記憶體的寫入屬於邊緣觸發。實際上標準的記憶體晶片會有一個寫入致能(write enable)訊號來控制寫入。儘管寫入致能並不屬於邊緣觸發，我們的邊緣觸發設計可以很容易地套用於真實記憶體晶片

4.3 建構數據通道

- ▣ 接下來討論MIPS 的載入字組和儲存字組指令
- ▣ 若相等則分支(beq)指令

圖4.9 分支指令的數據通道使用ALU 來評定分支條件以及加法器將遞增後的PC 值加上指令的最低16 個位元(分支位移量)經過符號延伸再左移2 個位元的值來計算分支目標位址

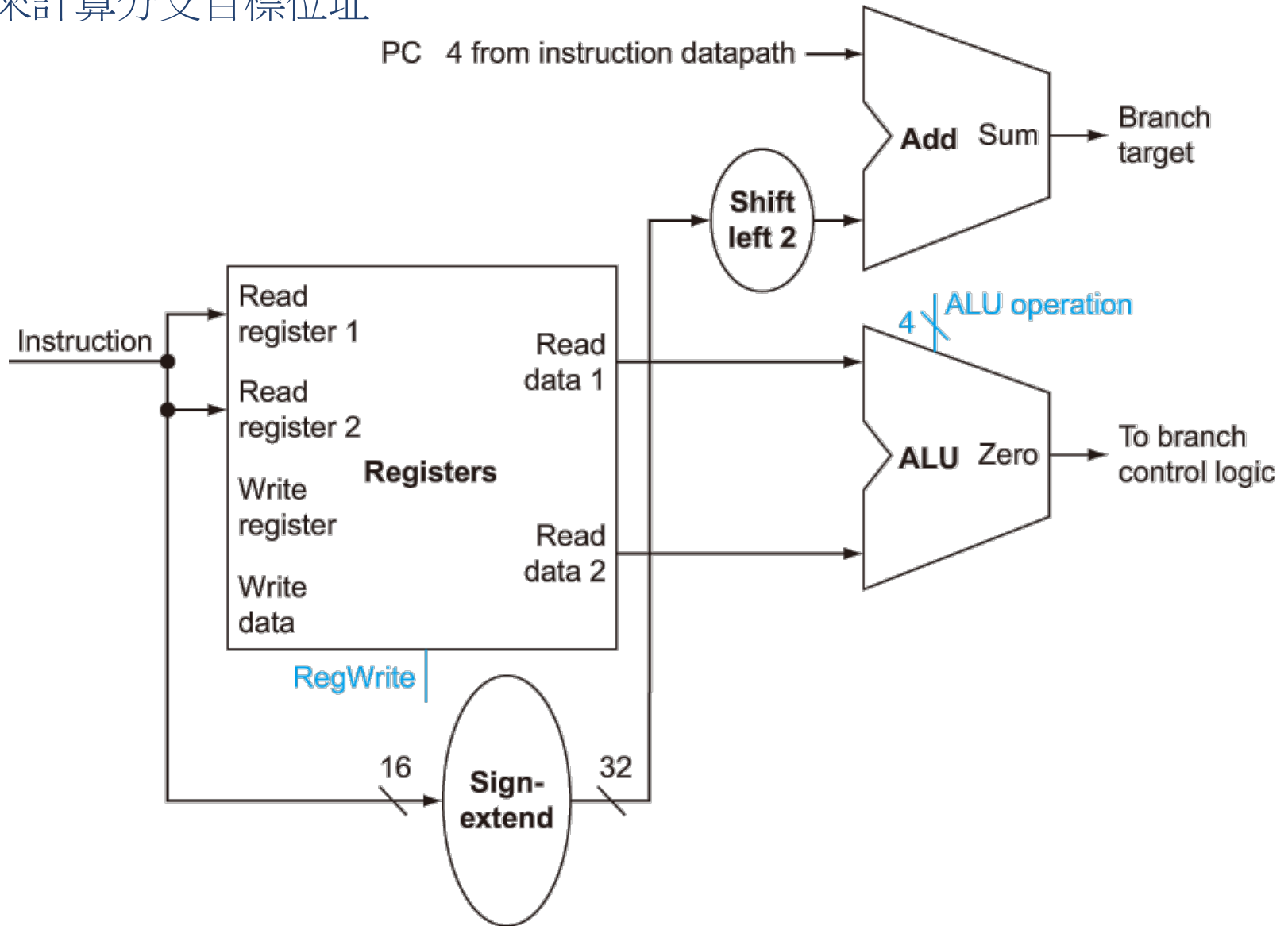


圖4.9 分支指令的數據通道使用**ALU** 來評定分支條件以及加法器將遞增後的**PC** 值加上指令的最低**16** 個位元(分支位移量)經過符號延伸再左移**2** 個位元的值來計算分支目標位址

標記為「左移**2** 位元」的單元其實只是將輸入和輸出之間的訊號線做簡單的繞線，在符號延伸後的偏移量欄位最右方加上**002**；因為位移的數目是常數，所以並不需要真的位移硬體。因為我們知道這個偏移量是由**16** 位元的資料符號延伸而來，所以位移的動作只會把代表正負號的位元扔掉。控制邏輯根據**ALU** 的**Zero**輸出訊號決定將**PC** 值代以遞增後的**PC** 值或是分支目標位址

- 在MIPS 指令集中，分支指令是被延遲的(delayed)
 - 意為不論分支條件是否成立，緊接在分支指令之後的指令一定會被執行
 - 使用被延遲的分支指令其動機來自管線化對分支指令的影響(見4.8 節)
 - 為了簡單起見，我們在本章中先忽略延遲的分支來實作一個非延遲的beq 指令

做出一個數據通道

- 這個最簡單的數據通道執行每個指令都只需一個時脈週期的時間
 - 代表在執行每道指令時所有數據通道的資源都至多只能被使用一次
 - 還是有許多元件可以被不同的指令共用
- 為了讓兩個不同種類的指令共用數據通道中的元件
 - 可能需要將多個來源連接到元件的輸入，並使用多工器和控制訊號來選擇不同的輸入

做出一個數據通道

■ MIPS架構的簡單數據通道：圖4.11

- 之後即可開始加入控制單元
- 控制單元必須可以接受輸入，並且產生每個狀態元件的寫入控制訊號、每個多工器的選擇控制訊號和ALU 的控制訊號

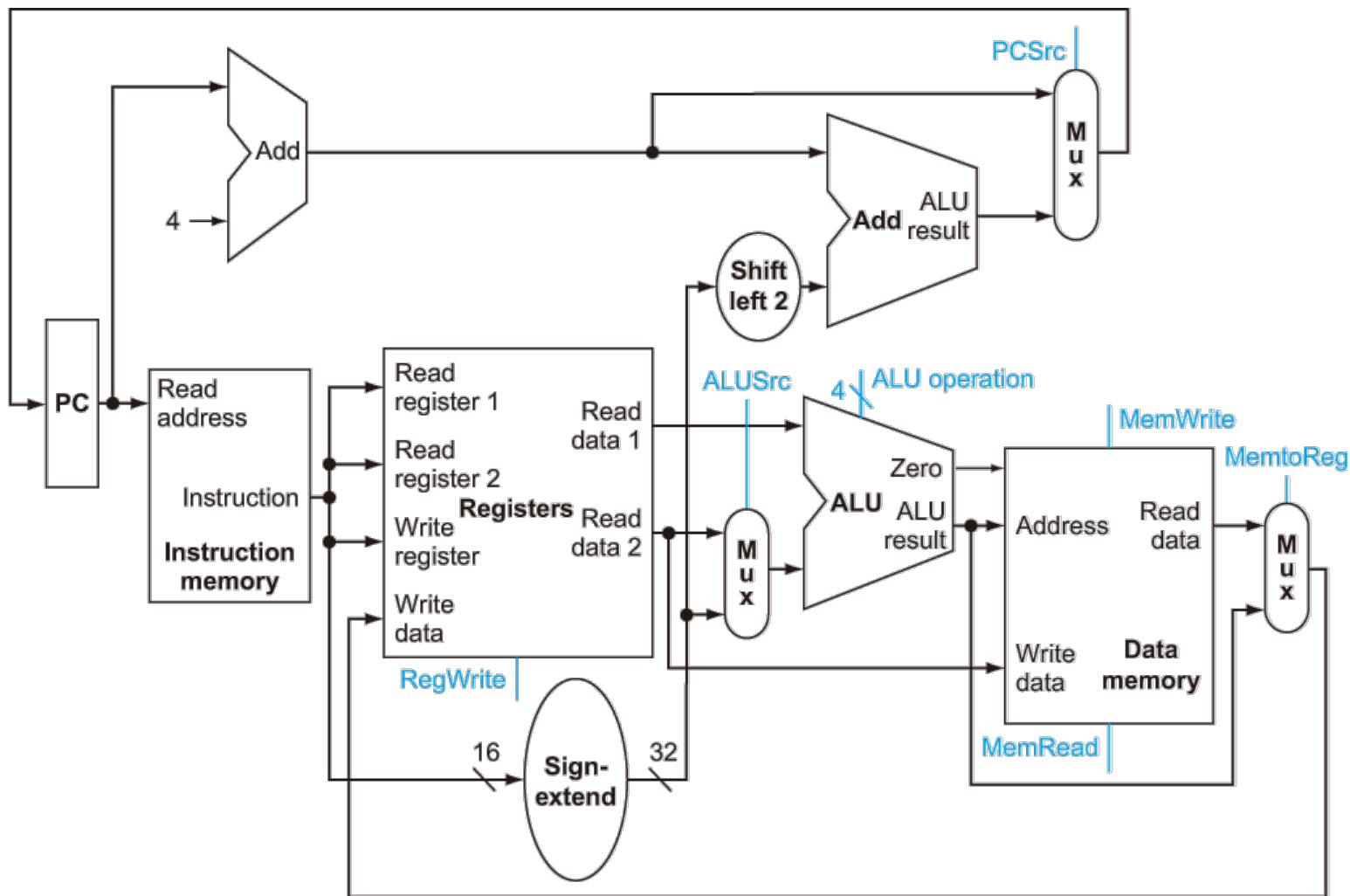


圖4.11 結合不同類別指令所需元件的MIPS 架構簡單數據通道

這些組件來自圖4.6、4.9 與4.10。這個數據通道可以在單一時脈週期中執行基本的指令(載入/儲存字組、算術邏輯運作及分支)。我們需要額外的一個多工器來整合入分支。至於跳躍指令的部分將會在稍後加入



MIPS處理器的簡單數據通道(dat

① Start presenting to display the poll results on this slide.

4.4 簡單實作方案

- ▣ 可處理載入字組(lw) 、儲存字組(sw) 、相等時分支 (beq)及算術邏輯指令 add 、 sub 、 AND 、 OR 及 set on less than
- ▣ 稍後還會再處理跳躍指令(j)

ALU 的控制

ALU 控制線	功 能
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR



ALU的控制有那些功能？

① Start presenting to display the poll results on this slide.

ALU 的控制

- 對於載入字組及儲存字組指令，我們使用ALU 做加法來計算記憶體的字址
- 對於R 型指令，ALU 則需根據指令中最低6 個位元功能(func)欄位的值來決定執行五種動作 (AND 、 OR 、 subtract 、 add 或set on less than)之一(見第二章)
- 對於相等時分支ALU 則必須做減法

ALU 的控制

- ▣ 使用一個小控制單元並以指令的功能(func`unct`)欄位以及我們稱之為ALUOp 的2 位元的控制欄位作為輸入
 - ALUOp 表示要執行的運算是載入與儲存的加法(00)、beq 的減法(01)或是由指令的功能欄位所編碼的動作來決定(10)



ALUOp有2位元控制欄位，可以

① Start presenting to display the poll results on this slide.

ALU 的控制

lwi	10	載入字組	100110	加	0010
指令運作碼	ALUOp	指令運作	功能欄位	需要的 ALU 動作	ALU 控制輸入
LW	00	載入字組	XXXXXX	加	0010
SW	00	儲存字組	XXXXXX	加	0010
Branch equal	01	若等於則分支	XXXXXX	減	0110
R-型	10	加	100000	加	0010
R-型	10	減	100010	減	0110
R-型	10	及	100100	及	0000
R-型	10	或	100101	或	0001
R-型	10	若小於則設定	101010	若小於則設定	0111

圖4.12 ALU 控制訊號如何設定視 ALUOp 控制位元及R-型指令的不同功能碼而定

第一行所列出的運作碼決定了第二行的 ALUOp 位元。所有的編碼都以二進位來表示。注意當 ALUOp 的值為 00 或 01 時，所需的 ALU 動作和功能欄位的值無關；所以我們使用「XXXXXX」來表示我們對功能欄位的值並無所謂(don't care)。如果ALUOp 的值為 10 時，則 ALU 控制輸入的值會依功能欄位而定



MIPS這9個指令中，那些用到AL

① Start presenting to display the poll results on this slide.



MIPS這9個指令中，那個用到AL

① Start presenting to display the poll results on this slide.



MIPS這9個指令中，那些用到AL

① Start presenting to display the poll results on this slide.

ALU 的控制

ALUOp		功能欄位						運作(ALU 控制訊號)
ALUOp1	ALUOp0	F5	F4	F3	F2	F1	F0	
0	0	X	X	X	X	X	X	0010
X	1	X	X	X	X	X	X	0110
1	X	X	X	0	0	0	0	0010
1	X	X	X	0	0	1	0	0110
1	X	X	X	0	1	0	0	0000
1	X	X	X	0	1	0	1	0001
1	X	X	X	1	0	1	0	0111

圖4.13 4 位元 ALU 控制訊號(稱為運作(operation))的真值表

輸入部分包含 ALUOp 及功能碼欄位。表中只顯示 ALU 控制被設定的項目。一些「無所謂」的項目也已經表出。例如，ALUOp 並沒有使用 11 的編碼，所以真值表可以使用 1X 和 X1，而不必是 10 和 01。注意當功能欄位被使用時，這些指令前兩個位元(F5 和 F4)的值總是等於 10，所以它們屬於「無所謂」的項目並且在真值表中以 XX 來表示

設計主要的控制單元

▣ 繼續探討控制的其他部分

▣ 圖4.14 表示指令的格式

- Op 欄位，Op[5 : 0] 來表示這個欄位
- 在 R 型指令、相等則分支(beq)指令及儲存指令中，被讀取的暫存器總是以位於指令的25 : 21 位元及 20 : 16 位元的rs 欄位及 rt 欄位來指出
- 載入及儲存指令中的基底暫存器總是在指令的25 : 21 位元(rs)
- 相等則分支(beq)指令、載入指令及儲存指令的16 位元偏移量(offset)總是在指令的15 : 0 位元

圖4.14 三種指令類別(R-型指令、載入/儲存指令及分支指令)使用兩種不同的指令格式

Field	0	rs	rt	rd	shamt	funct
Bit positions	31:26	25:21	20:16	15:11	10:6	5:0

a. R-type instruction

Field	35 or 43	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

b. Load or store instruction

Field	4	rs	rt	address
Bit positions	31:26	25:21	20:16	15:0

c. Branch instruction

圖4.14 三種指令類別(R-型指令、載入/儲存指令及分支指令)使用兩種不同的指令格式

跳躍指令(jump)使用另外一種格式，我們稍後會再討論。(a)R-型指令的指令格式，其運作碼(opcode)均為零。這些指令有三個暫存器的運算元：**rs**、**rt** 及 **rd**。**rs** 和 **rt** 欄位為來源(source)暫存器，**rd** 欄位為目的(destination)暫存器。**ALU** 的功能是利用前一節所設計的**ALU** 控制單元來對功能(func)欄位解碼所決定。我們實作的R-型指令有**add**、**sub**、**and**、**or** 及 **slt**。位移量(**shamt**)欄位僅使用在移位指令中；我們在本章中先忽略它。(b)載入指令(opcode=3510)和儲存指令(opcode=4310)的指令格式。**rs** 暫存器為基底暫存器(base register)，其值與16位元位址欄位的值相加後得到記憶體的位置。在載入指令中，**rt** 暫存器是載入值的目的暫存器。在儲存指令中，**rt** 暫存器是來源暫存器，其值要儲存到記憶體。(c)相等時分支指令(opcode = 410)的指令格式。**rs** 和 **rt** 暫存器是用於比較是否相等的來源暫存器。而16位元的位址欄位會被符號延伸、移位後和**PC+4** 值相加來計算分支目標位址

設計主要的控制單元

- 目的暫存器有可能出現在兩個地方，20 : 16位元(rt)或15 : 11 位元(rd)，必須加上一個多工器來選擇rt 或是rd 欄位
 - 規律性易導致簡單的設計
- 圖4.15 顯示上述新增以及ALU 控制區塊、狀態元件寫入控制訊號、數據記憶體의讀取訊號及多工器的控制訊號

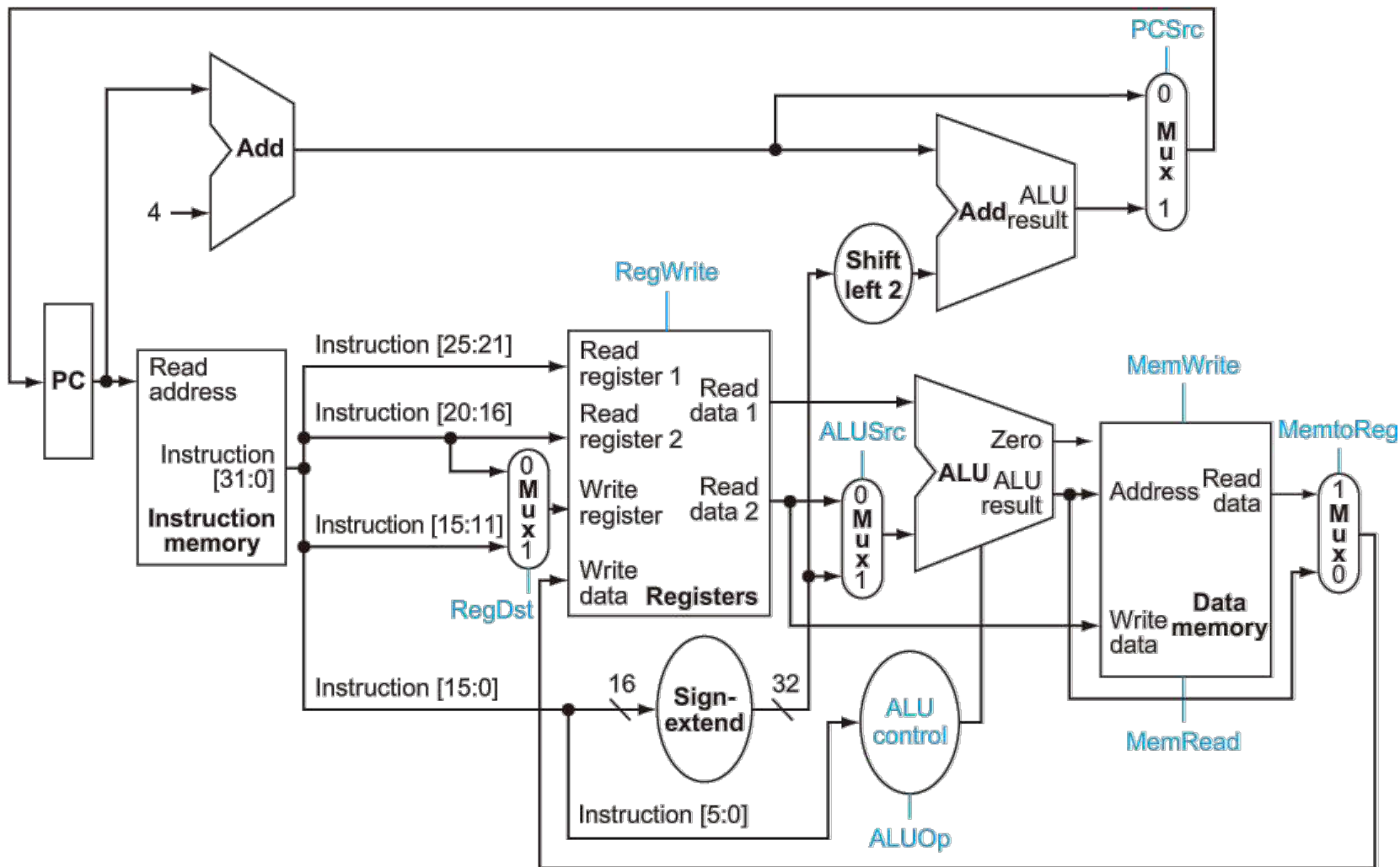


圖4.15 圖4.12 的數據通道以及所有需要的多工器及控制訊號

灰色的線表示控制訊號。ALU 控制區塊也已經加入圖中。因為PC 暫存器在每個時脈週期都會寫入一次，所以不需要寫入控制；分支控制邏輯決定寫入PC；暫存器的值是遞增後的PC 或是分支目標位址

設計主要的控制單元

■ 圖4.16 說明了七條控制訊號的功能

訊號名稱	未被設定時的效果	被設定時的效果
RegDst	Write register 目的暫存器編號來自 <code>rt</code> 欄位(指令的 20:16 位元)。	Write register 目的暫存器編號來自 <code>rd</code> 欄位(指令的 15:11 位元)。
RegWrite	無。	將 Write data 輸入的值寫入 Write register 輸入所指定的暫存器中。
ALUSrc	ALU 的第二個運算元來自暫存器檔案第二個輸出埠(Read data 2)。	ALU 的第二個運算元來自指令中最低的 16 個位元經過符號延伸後的值。
PCSrc	由計算 $PC + 4$ 的加法器的輸出來替換原本的 PC 值。	由計算分支目標位址的加法器的輸出來替換原本的 PC 值。
MemRead	無。	將數據記憶體中 Address 輸入所指定位址的內容送至 Read data 輸出。
MemWrite	無。	將數據記憶體中 Address 輸入所指定位址的內容置換為 Write data 輸入的值。
MemtoReg	暫存器 Write data 輸入的值來自 ALU。	暫存器 Write data 輸入的值來自數據記憶體。

設計主要的控制單元

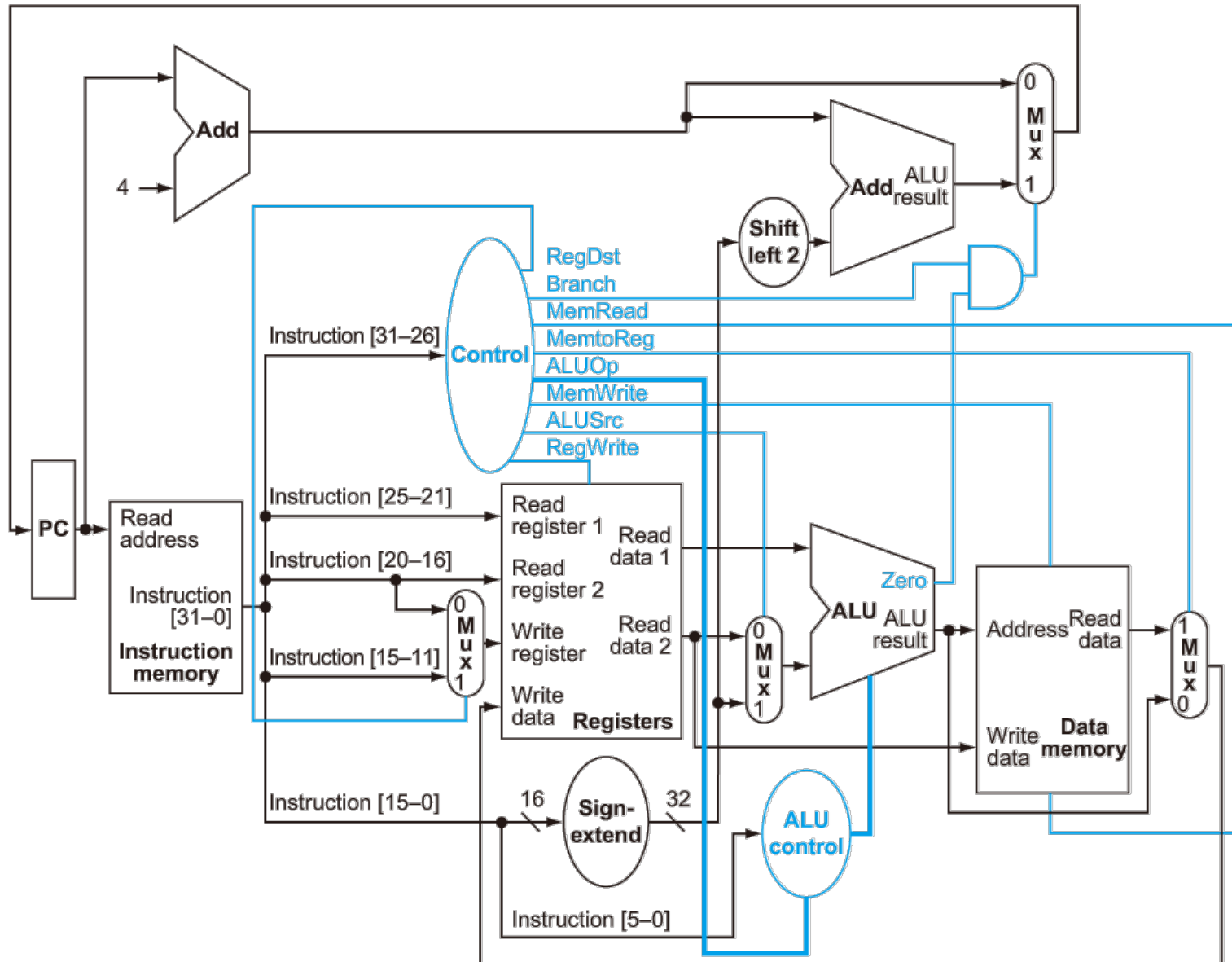
圖4.16 七條控制訊號每一條的效果

在二對一多工器的1 位元控制訊號被設定(**asserted**)時，多工器選擇 1 號輸入埠的值。否則當控制訊號未被設定(**deasserted**)時，多工器選擇 0 號輸入埠的值。記得所有的狀態元件都隱含了時脈輸入並使用時脈來控制寫入。在狀態元件之外先對時脈訊號進行邏輯運算可能會造成時序的問題

op	rs	rt	rd	Shamt	funct
----	----	----	----	-------	-------

6 個位元 5 個位元 5 個位元 5 個位元 5 個位元 6 個位元

R-type指令



■ 圖 4.17 表示包含控制單元以及控制訊號的數據通道

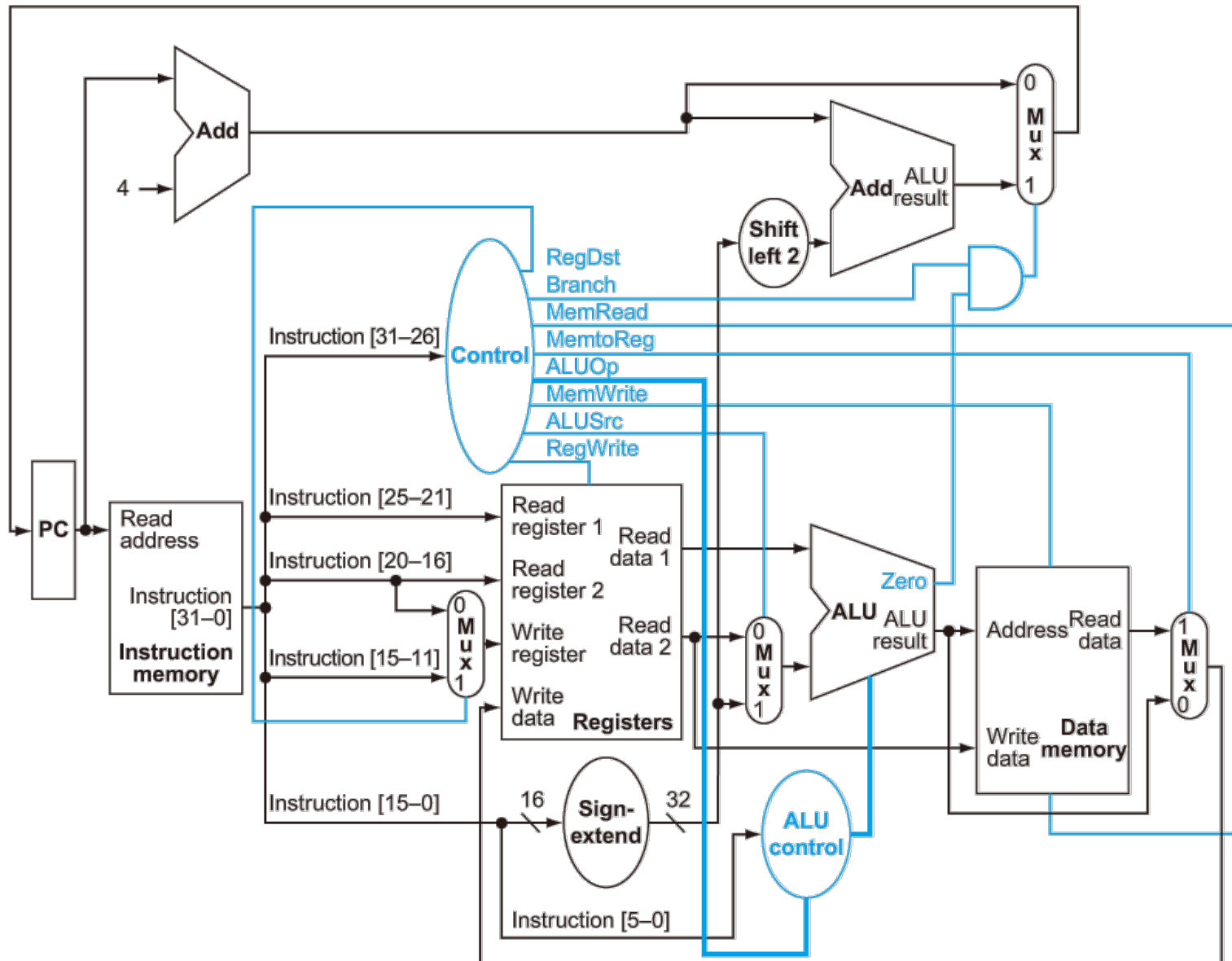


本章MIPS處理器的主要控制單元

① Start presenting to display the poll results on this slide.

op	rs	rt	常數或位址
6 個位元	5 個位元	5 個位元	16 個位元

I-type指令



■ 圖 4.17 表示包含控制單元以及控制訊號的數據通道

圖4.17 含有控制單元的簡單數據通道

控制單元的輸入為指令中的**6** 位元運作碼欄位。控制單元的輸出包含了三條**1** 位元的訊號線用來控制多工器(**RegDst**、**ALUSrc** 及**MemtoReg**)，三條訊號線用來控制暫存器檔案及數據記憶體的讀與寫(**RegWrite**、**MemRead** 及**MemWrite**)，一條**1** 位元的訊號線用來決定是否分支(**Branch**)，以及**2** 位元的訊號線用來控制**ALU**(**ALUOp**)。使用一個**AND** 閘來運算 **Branch** 控制訊號及**ALU** 的**Zero** 輸出訊號線；**AND** 閘的輸出控制下一個**PC** 值的選擇。注意**PCSrc** 是衍生出來的訊號，而不是直接來自於控制單元，因此在接下來的圖中，我們將不顯示它的名稱



控制單元輸出，那些訊號是用來

① Start presenting to display the poll results on this slide.



控制單元輸出，那些訊號是用來

① Start presenting to display the poll results on this slide.

設計主要的控制單元

- 圖4.18 定義所有控制訊號線應該如何對每一種運作碼來設定

指令	RegDst	ALUSrc	Mem-toReg	Reg-Write	Mem-Read	Mem-Write	Branch	ALUOp1	ALUOp0
R 格式	1	0	0	1	0	0	0	1	0
lw	0	1	1	1	1	0	0	0	0
sw	X	1	X	0	0	1	0	0	0
beq	X	0	X	0	0	0	1	0	1
lwi	1	0	1	1	1	0	0	1	0



R格式指令的控制輸出，那些會被

① Start presenting to display the poll results on this slide.



beq指令的控制輸出，那些會被計

① Start presenting to display the poll results on this slide.

slido



sw指令的控制輸出，那些會被設

① Start presenting to display the poll results on this slide.



lw指令的控制輸出，那些會被設

① Start presenting to display the poll results on this slide.

圖4.18 控制訊號線的設定完全取決於指令中的運作碼欄位

表中的第一列對應到R 格式的指令(add、sub、and、or 及slt)。這些指令的來源暫存器欄位都是rs 和rt，目的暫存器都是rd；這定義了ALUSrc 和RegDst 控制訊號線如何設定。此外，R-型指令將運算結果寫入暫存器(RegWrite=1)，但是既不會讀取也不會寫入數據記憶體。當Branch 控制訊號等於0 時，PC 的值無條件地由PC+4 取代；否則，如果ALU 的Zero 輸出值也為1 時，PC 的值由分支目標位址取代。R-型指令的ALUOp 欄位被設定為10，表示ALU 的控制是由功能欄位(func field)來產生。在表中的第二列及第三列說明lw 指令及sw 指令的控制訊號設定。這裡的ALUSrc 和ALUOp 欄位設定成執行位址的計算。而Mem-Read 及MemWrite 設定成執行記憶體的存取。最後，對於載入指令，RegDst 及RegWrite 設定成將結果儲存到rt 暫存器中。分支指令和R 格式指令相似，都是將rs 和rt 暫存器的值送到ALU 去運算。分支指令的ALUOp 欄位設定成執行減法(ALU 控制=01)，用來測試兩個值是否相等。注意如果RegWrite 控制訊號為0 時MemtoReg 欄位是無關緊要的：因為暫存器不會被寫入，所以暫存器的Write data 輸入值不會被用到。因此，在表格最後兩列的MemtoReg 項目用X 來表示，代表「無所謂」。當RegWrite 為0 時，RegDst 也可以使用X 來表示。設計者必須要使用這種「無所謂」項目，因為其關乎對數據通道運作的瞭解

數據通道的運作

- 在接下來的幾張圖中，我們表示三種不同種類的指令通過數據通道的流程
 - 每一張圖中，被設定的控制訊號線以及動作的數據通道元件都會以用灰色來強調
- 圖4.19 顯示數據通道對R 型指令
 - 一個時脈週期內可以把它想像成執行了四個步驟：

數據通道的運作

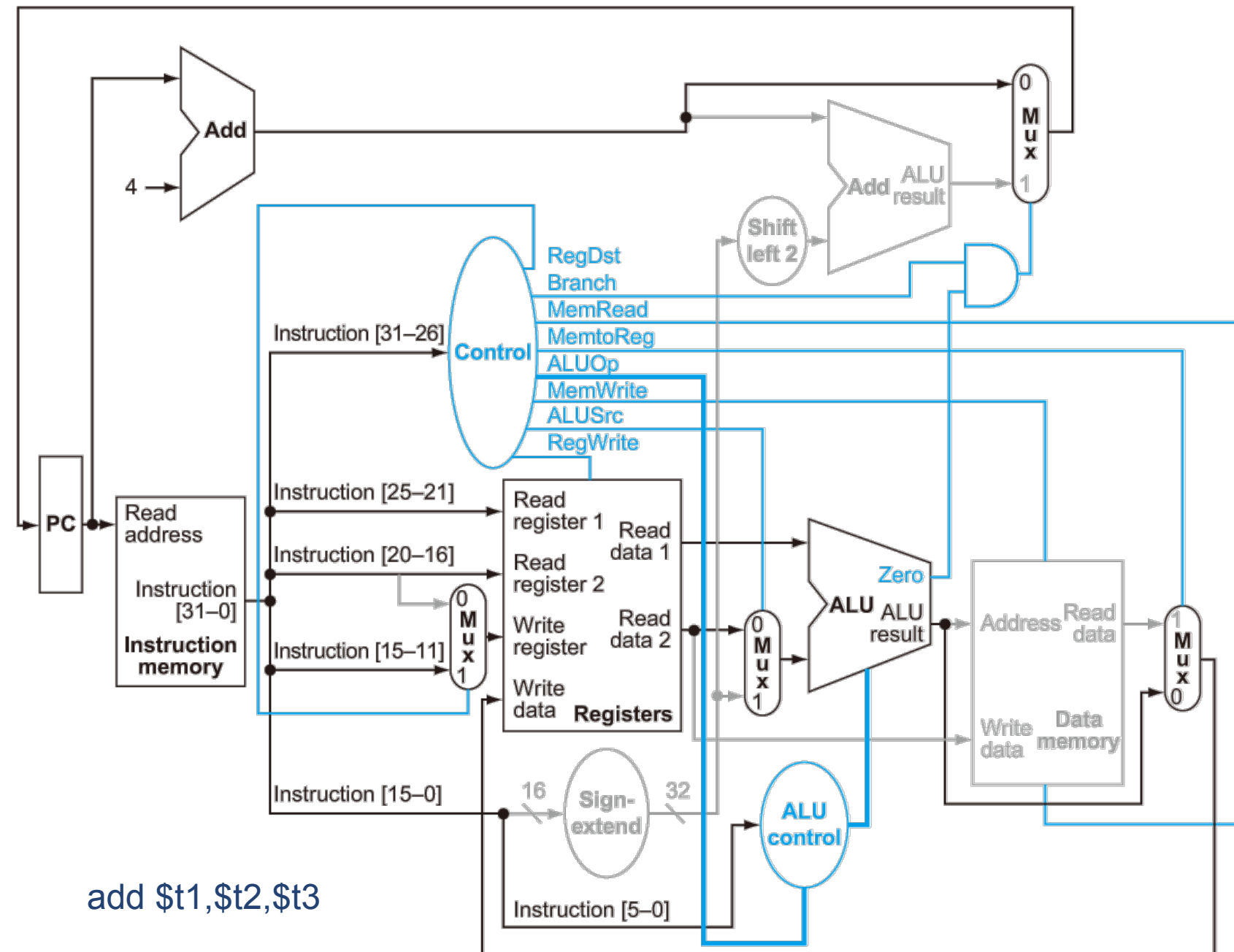
- ① 指令被擷取，並且遞增PC 的值
- ② 兩個暫存器從暫存器檔案中讀出；同時，主控制單元計算控制訊號線的值
- ③ ALU 根據功能碼(指令中的5 :0 位元功能欄位)來產生ALU 功能的控制，以對從暫存器檔案中讀出的值運算
- ④ ALU 的運算結果使用指令的15:11 位元來選擇目的暫存器(\$t1)，以寫入暫存器檔案

add \$t1,\$t2,\$t3

名 稱	暫存器號碼
\$zero	0
\$v0-\$v1	2-3
\$a0-\$a3	4-7
\$t0-\$t7	8-15
\$s0-\$s7	16-23
\$t8-\$t9	24-25
\$gp	28
\$sp	29
\$fp	30
\$ra	31

圖4.19
數據通道對於如add \$t1,\$t2,\$t3 的R型指令運作方式
運作中的控制訊號線、數據通道單元及其連線均以灰色強調

add \$t1,\$t2,\$t3



數據通道的運作

- 圖4.20 顯示**載入指令**中有動作的功能單元和被設定的控制訊號
 - 想像成執行了五個步驟：
 - ① 指令從指令記憶體中被擷取，並且遞增PC 的值
 - ② 暫存器的值從暫存器檔案中讀出
 - ③ ALU 計算由暫存器檔案中讀出的值和符號延伸過的指令中較低的16 位元偏移量的和
 - ④ ALU 所得的和作為數據記憶體的位址
 - ⑤ 記憶體傳回的數據寫入暫存器檔案；暫存器的目的地可由指令中的位元20:16 得知

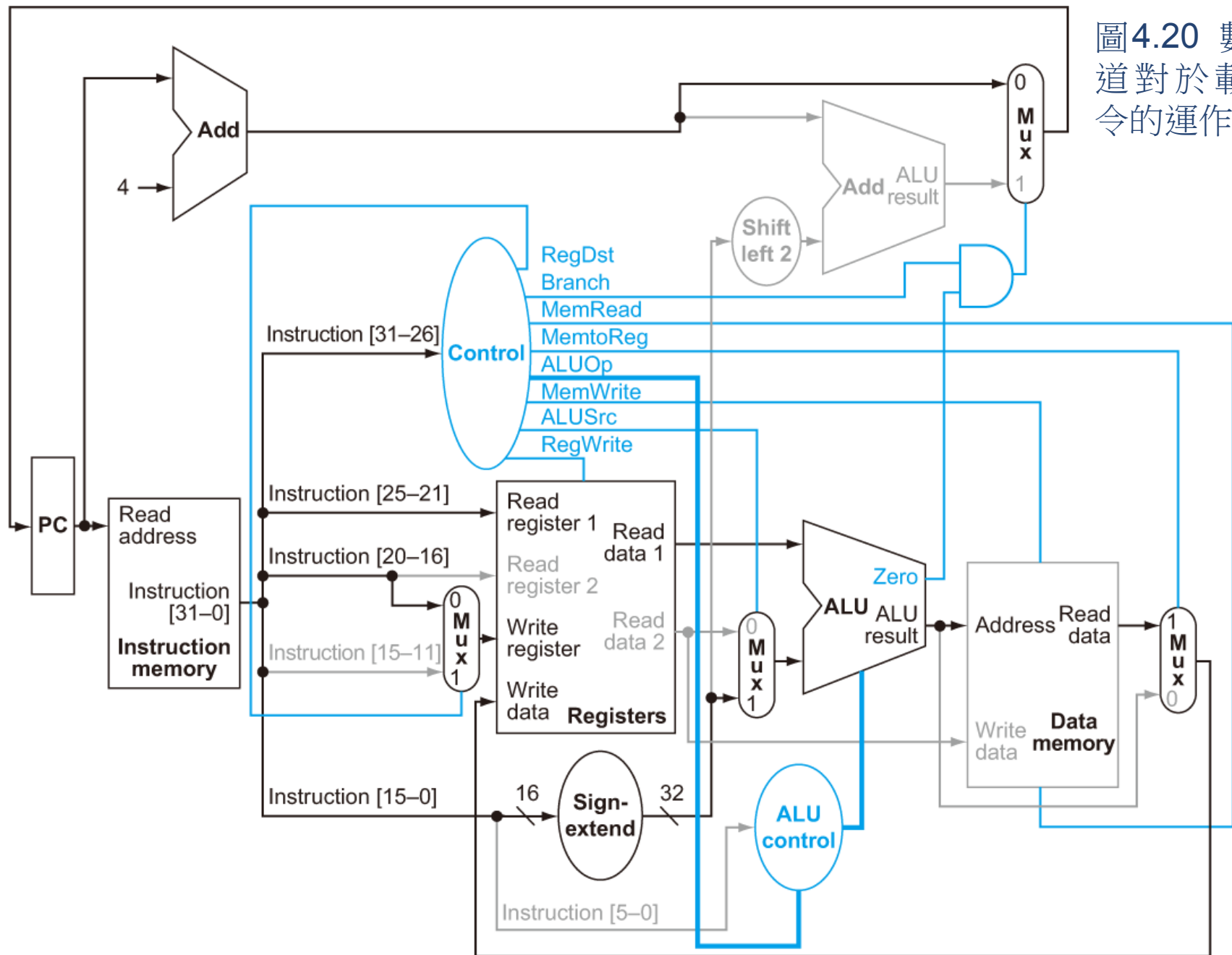


圖4.20 數據通道對於載入指令的運作方式

圖4.20 數據通道對於載入指令的運作方式

運作中的控制訊號線、數據通道單元及其連線均以灰色強調。儲存指令的運作方式非常類似。主要的不同在記憶體的控制部分是寫入而不是讀取，而寫入的值來自於讀取第二個暫存器所得的值且不會將數據記憶體的值寫回暫存器檔案

數據通道的運作

■ 圖4.21 顯示**beq**執行時的四個步驟：

- ① 指令從指令記憶體中被擷取，並且遞增PC 的值
- ② 兩個暫存器\$t1 和\$t2 從暫存器檔案中被讀出
- ③ ALU 對暫存器檔案中讀出的值執行減法。PC+4 的值與符號延伸並左移兩位的指令中較低的16 位元(偏移量)相加；其結果即為分支目的位址
- ④ ALU 的Zero 輸出被用來決定哪一個加法器的結果要寫回PC

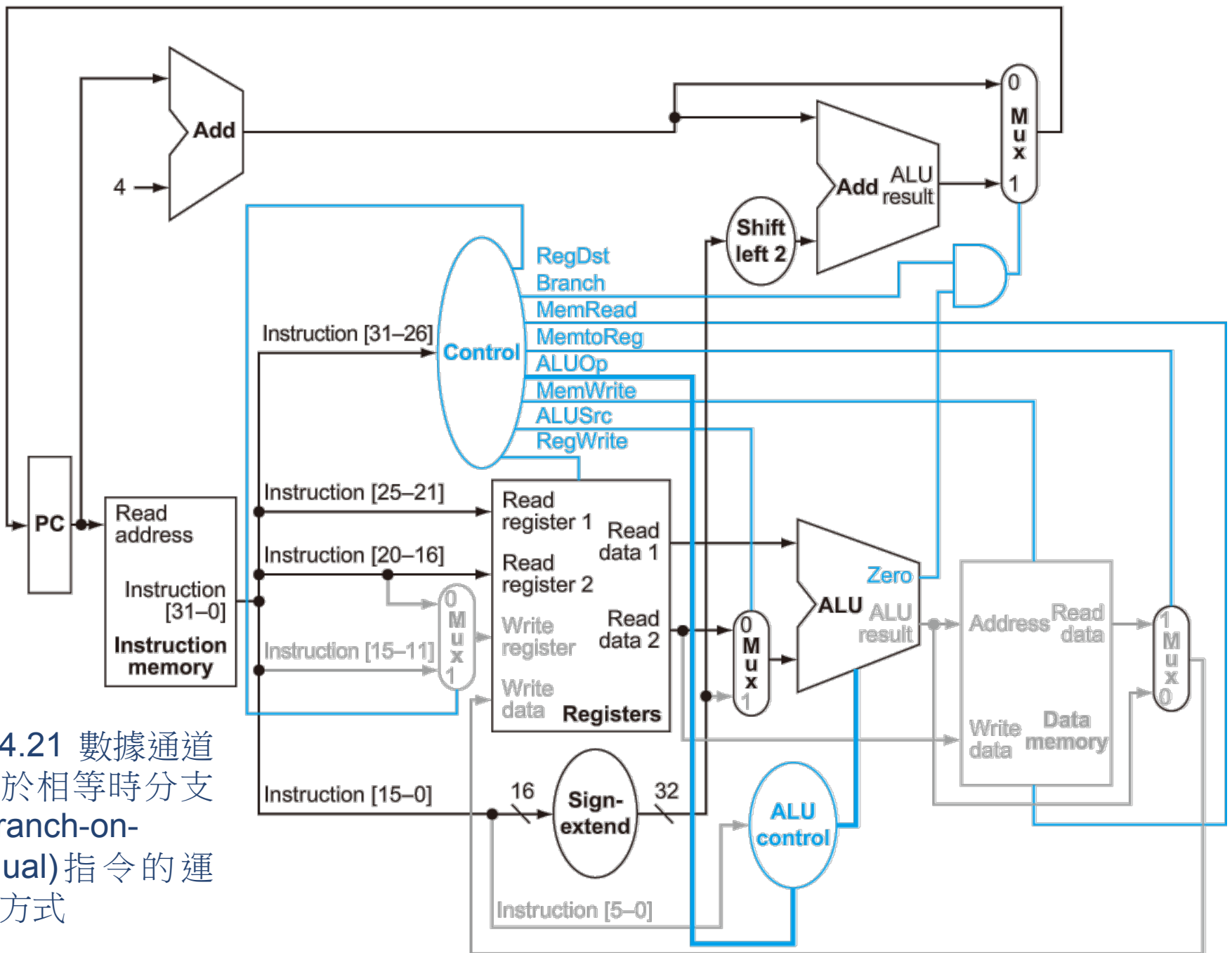


圖4.21 數據通道
對於相等時分支
(branch-on-
equal)指令的運
作方式

圖4.21 數據通道對於相等時分支(branch-on-equal)指令的運作方式

運作中的控制訊號線、數據通道單元及其連線均以灰色強調。使用暫存器讀出的值和**ALU** 來執行比較之後，**Zero** 輸出在兩個候選值之間選擇下一個程式計數器(**PC**)的值

圖4.22
簡單單一
週期製作
的控制功
能完整說
明於此真
值表中

輸入或輸出	訊號名稱	R 格式	lw	sw	beq	lwi
輸 入	Op5	0	1	1	0	0
	Op4	0	0	0	0	0
	Op3	0	0	1	0	0
	Op2	0	0	0	1	0
	Op1	0	1	1	0	0
	Op0	0	1	1	0	0
輸 出	RegDst	1	0	X	X	1
	ALUSrc	0	1	1	0	0
	MemtoReg	0	1	X	X	1
	RegWrite	1	1	0	0	1
	MemRead	0	1	0	0	1
	MemWrite	0	0	1	0	0
	Branch	0	0	0	1	0
	ALUOp1	1	0	0	0	1
	ALUOp0	0	0	0	1	0

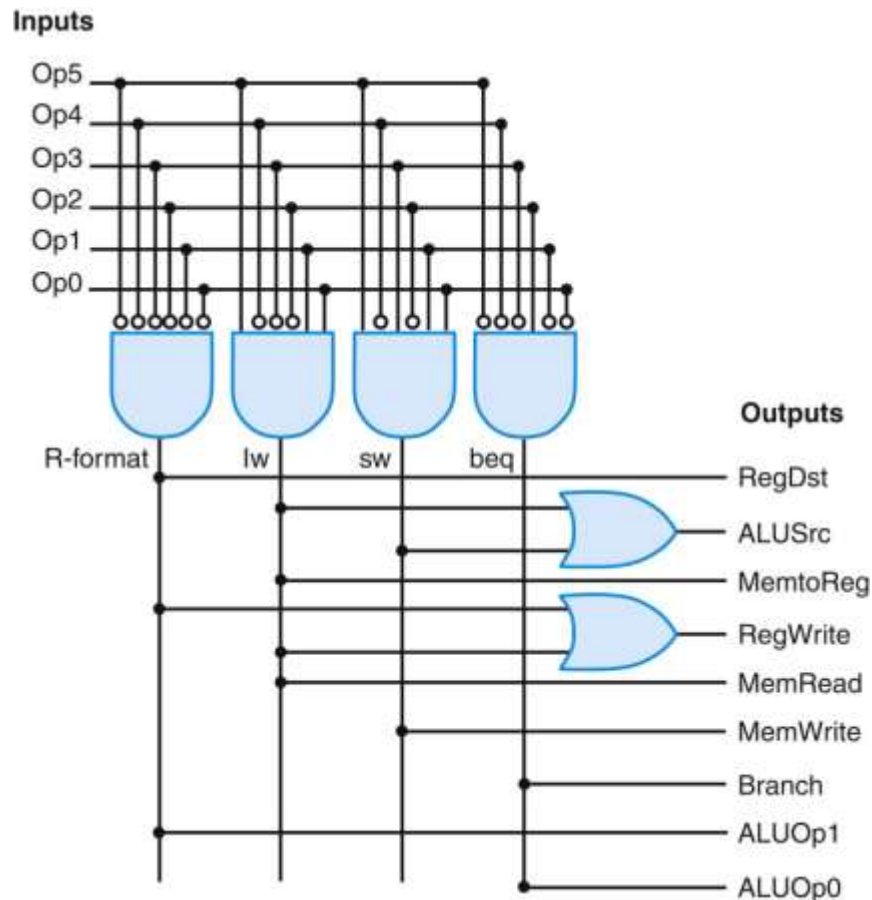


FIGURE D.2.5 The structured implementation of the control function as described by the truth table in Figure D.2.4. The structure, called a programmable logic array (PLA), uses an array of AND gates followed by an array of OR gates. The inputs to the AND gates are the function inputs and their inverses (bubbles indicate inversion of a signal). The inputs to the OR gates are the outputs of the AND gates (or, as a degenerate case, the function inputs and inverses). The output of the OR gates is the function outputs.



Single-Cycle MIPS CPU, 控制輸出ALUSrc由那些Opcode解碼器輸出決定？



Single-Cycle MIPS CPU, 控制輸出RegWrite由那些Opcode解碼器輸出決定？

① Start presenting to display the poll results on this slide.

Chapter 4 Exercise Assignment 1

- 4.2.1-4.2.3
- 4.2.4 List all output control signals for the LWI instruction and expand the tables of Fig. 4.12 , 4.18 & 4.22.

圖4.22 簡單單一週期製作的控制功能完整說明於此真值表中。在表的上半部四行中是四種運作碼用來決定輸出控制訊號設定對應的輸入訊號線組合並且(記得Op [5 :0] 是指令31 : 26位元的運作碼欄位。)表格的下半部則是四種運作碼的輸出。因此，輸出**RegWrite** 在兩種輸入的組合會被設定。如果我們只考慮表中的四種運算碼，則我們可以在輸入的部分使用「無所謂」來簡化真值表。例如，我們可以使用Op5 · Op2來偵測出R 格式的指令，因為這已經足夠來區分R 格式指令和lw、sw 及beq。但是因為在完整的實作中會加入其他MIPS 運作碼，所以我們並不採用這樣的簡化方式。

完成控制

- ▣ 圖4.24 顯示圖4.17 中加入跳躍指令所作的額外增加

完成控制

- 圖4.22 以真值表的形式表示了控制單元中的邏輯電路
- 以範例的方式加入跳躍(jump)指令，以便說明如何在基本的數據通道及控制中，再作延伸以處理其他的指令

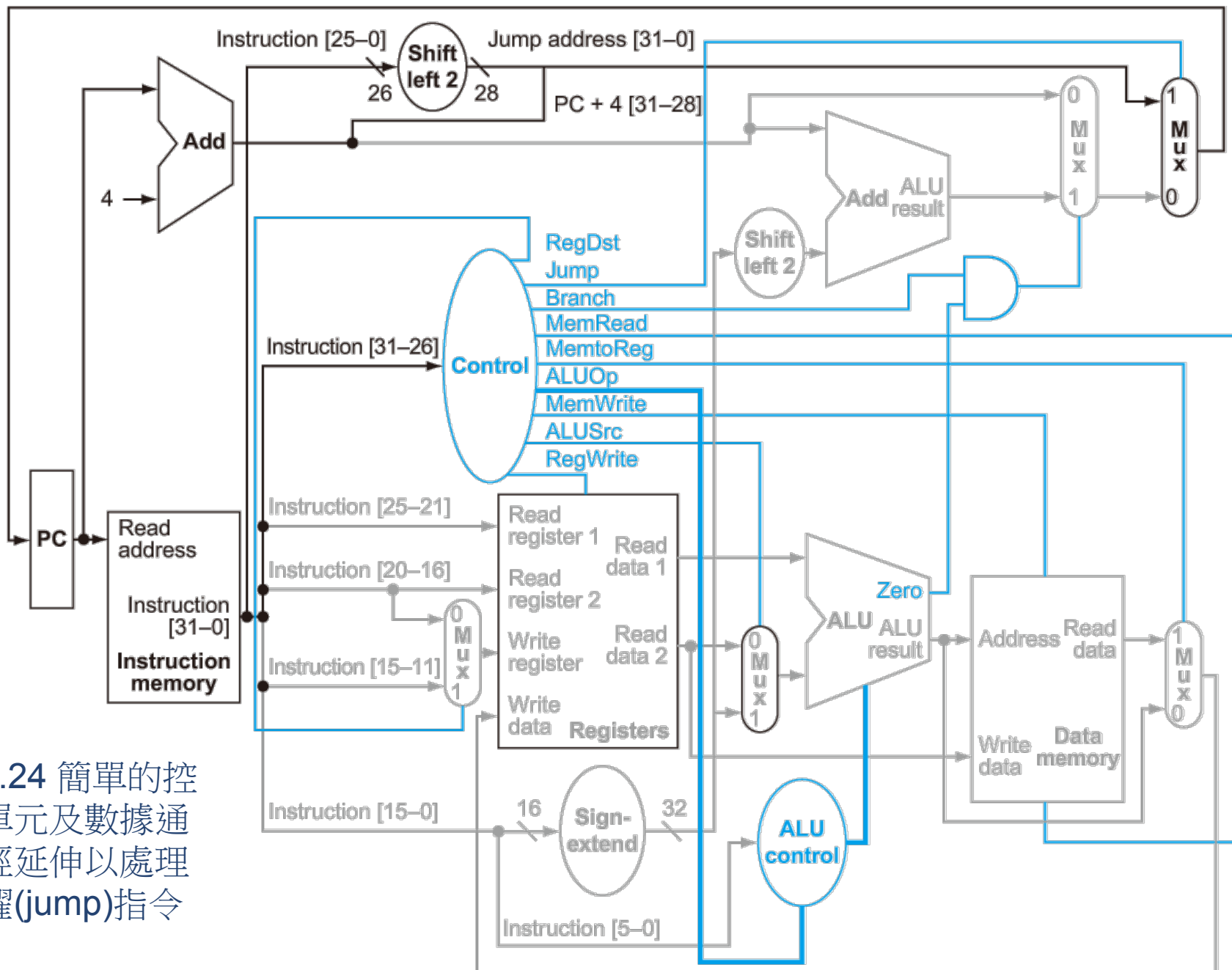


圖4.24 簡單的控制單元及數據通道經延伸以處理跳躍(jump)指令



MIPS處理器要完成jump指令(j)

① Start presenting to display the poll results on this slide.

100000000000000000000000000000000000100000

Loop:	sll	\$t1,	\$s3,	2	0x80000020	
	add	\$t1,	\$t1,	\$s6	0x80000024	
	lw	\$t0,	0(\$t1)		0x80000028	
	bne	\$t0,	\$s5,	Exit	0x8000002c	
	addi	\$s3,	\$s3,	1	0x80000030	
	j	Loop			0x80000034	<- PC

Exit: ...

PC+4 = 0x80000038

PC+4[31~28] = 1000₂

j Loop => 000010 000000000000000000000000000000001000

Jump address = PC+4[31~28]+

00000000000000000000000000000000100000

= 100000000000000000000000000000000000100000

= 0x800000020

為何今天已不採用單一週期設計

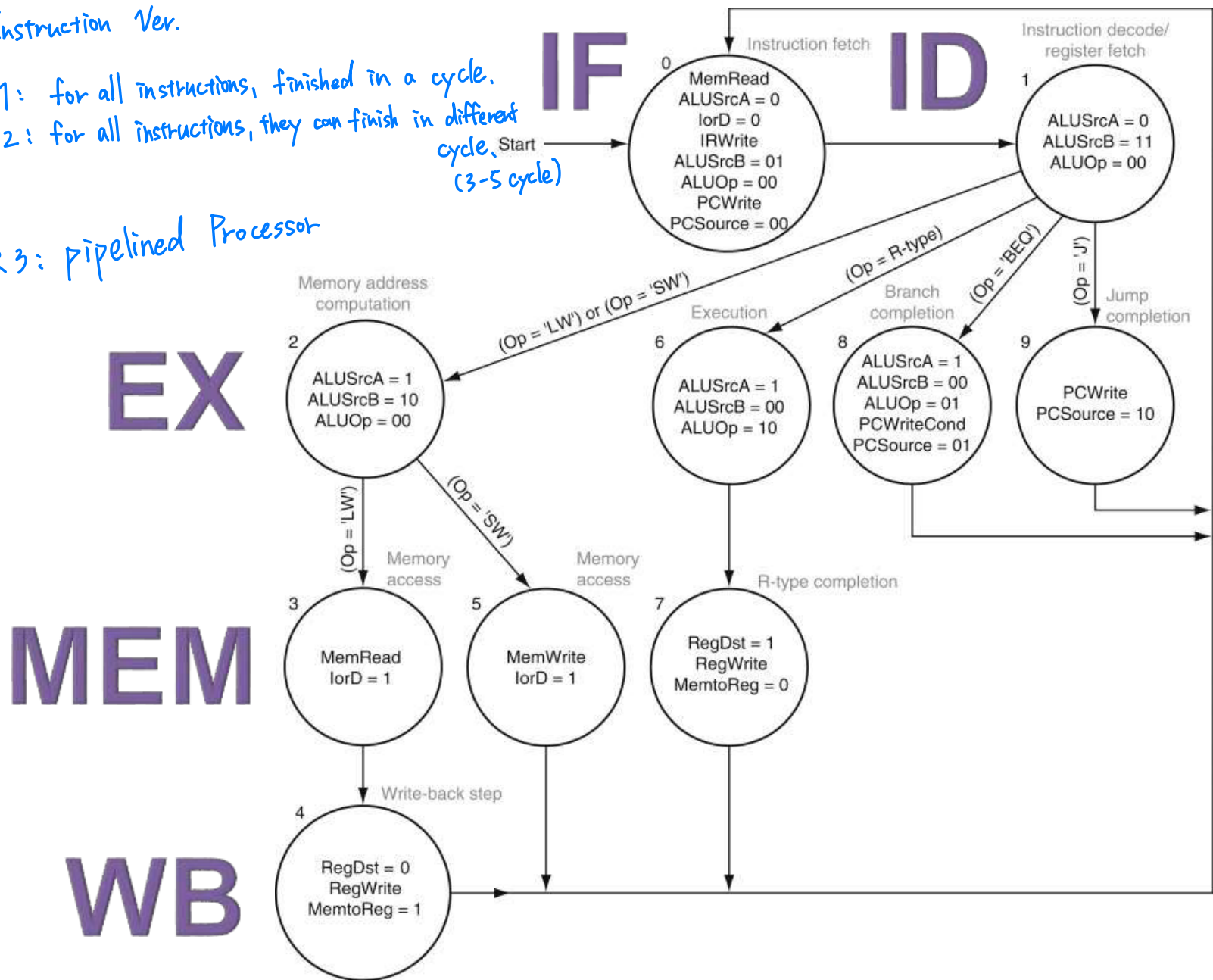
■ 單一週期的設計效率不好

- 時脈週期要由處理器中可能的最長路徑來決定
 - 這條路徑幾乎可以確定是載入指令
- 儘管CPI 為1，單一週期設計的整體效率因為時脈週期太長而可能很差
- 違反了我們第二章中所提讓經常的情形變快的主要設計原則

Instruction Ver.

R1: for all instructions, finished in a cycle.
 R2: for all instructions, they can finish in different cycle. (3-5 cycle)

R3: pipelined Processor



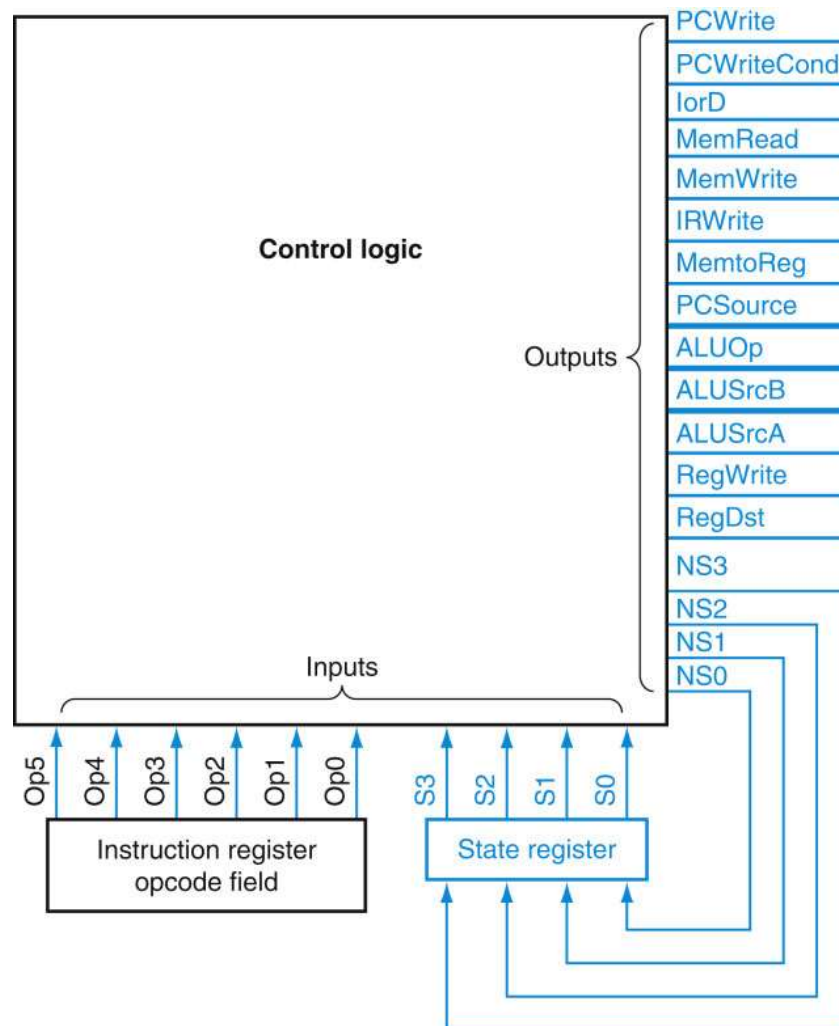


FIGURE D.3.2 The control unit for MIPS will consist of some control logic and a register to hold the state. The state register is written at the active clock edge and is stable during the clock cycle

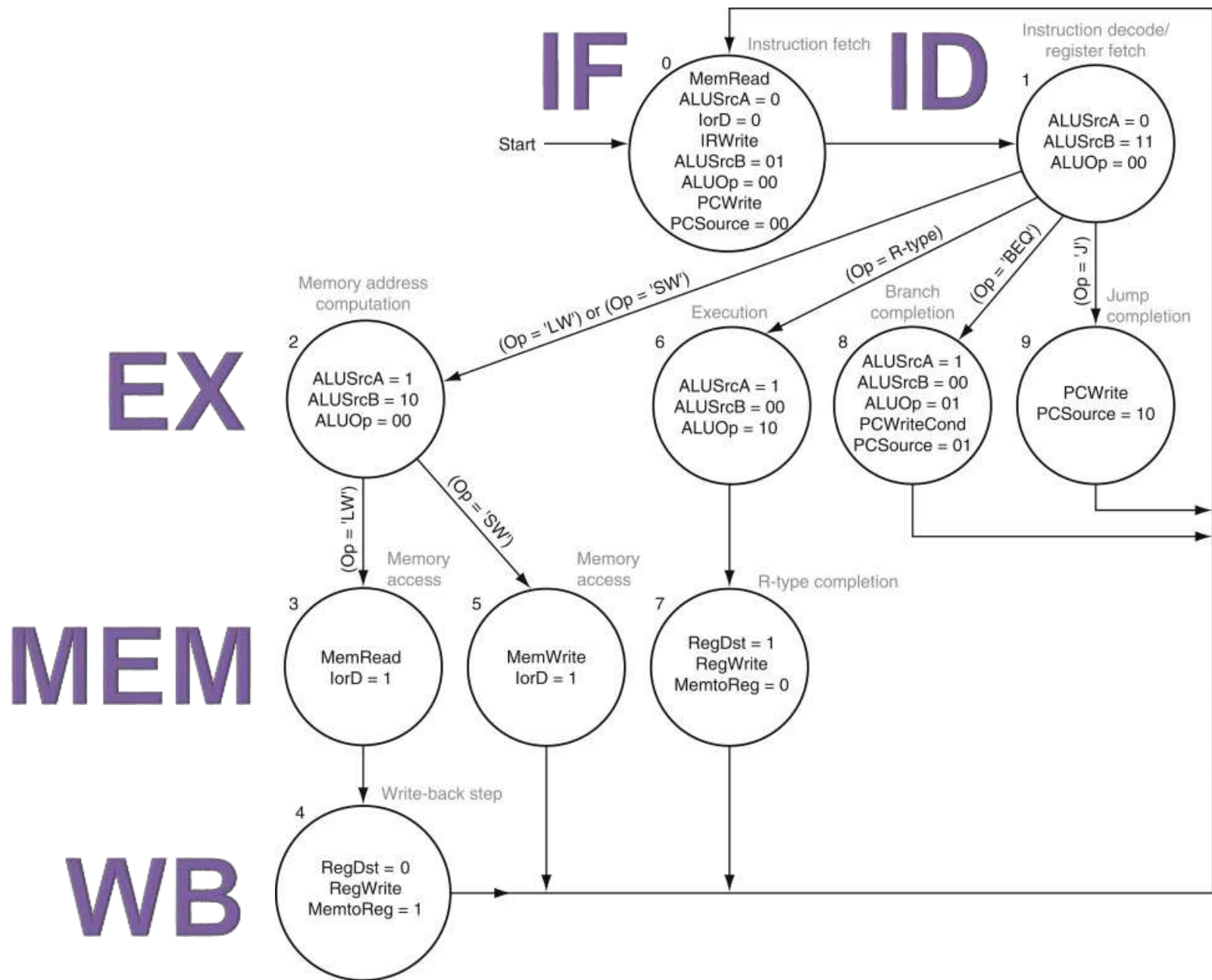


FIGURE D.3.1 The finite-state diagram for multicycle control.

	Op [5–0]					
Current state S[3–0]	000000 (R-format)	000010 (jmp)	000100 (beq)	100011 (lw)	101011 (sw)	Any other value
0000	0001	0001	0001	0001	0001	0001
0001	0110	1001	1000	0010	0010	Illegal
0010	XXXX	XXXX	XXXX	0011	0101	Illegal
0011	0100	0100	0100	0100	0100	Illegal
0100	0000	0000	0000	0000	0000	Illegal
0101	0000	0000	0000	0000	0000	Illegal
0110	0111	0111	0111	0111	0111	Illegal
0111	0000	0000	0000	0000	0000	Illegal
1000	0000	0000	0000	0000	0000	Illegal
1001	0000	0000	0000	0000	0000	Illegal

FIGURE D.3.8 This table contains the lower 4 bits of the control word (the NS outputs), which depend on both the state inputs, S[3–0], and the opcode, Op[5–0], which correspond to the instruction opcode. These values can be determined from Figure D.3.5. The opcode name is shown under the encoding in the heading. The four bits of the control word whose address is given by the current-state bits and Op bits are shown in each entry. For example, when the state input bits are 0000, the output is always 0001, independent of the other inputs; when the state is 2, the next state is don't care for three of the inputs, 3 for lw, and 5 for sw. Together with the entries in Figure D.3.7, this table specifies the contents of the control unit ROM. For example, the word at address 1000110001 is obtained by finding the upper 16 bits in the table in Figure D.3.7 using only the state input bits (0001) and concatenating the lower four bits found by using the entire address (0001 to find the row and 100011 to find the column). The entry from Figure D.3.7 yields 00000000000011000, while the appropriate entry in the table immediately above is 0010. Thus the control word at address 1000110001 is 00000000000011000010. The column labeled “Any other value” applies only when the Op bits do not match one of the specified opcodes.

Outputs	Input values (S[3-0])									
	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001
PCWrite	1	0	0	0	0	0	0	0	0	1
PCWriteCond	0	0	0	0	0	0	0	0	1	0
IorD	0	0	0	1	0	1	0	0	0	0
MemRead	1	0	0	1	0	0	0	0	0	0
MemWrite	0	0	0	0	0	1	0	0	0	0
IRWrite	1	0	0	0	0	0	0	0	0	0
MemtoReg	0	0	0	0	1	0	0	0	0	0
PCSource1	0	0	0	0	0	0	0	0	0	1
PCSource0	0	0	0	0	0	0	0	0	1	0
ALUOp1	0	0	0	0	0	0	1	0	0	0
ALUOp0	0	0	0	0	0	0	0	0	1	0
ALUSrcB1	0	1	1	0	0	0	0	0	0	0
ALUSrcB0	1	1	0	0	0	0	0	0	0	0
ALUSrcA	0	0	1	0	0	0	1	0	1	0
RegWrite	0	0	0	0	1	0	0	1	0	0
RegDst	0	0	0	0	0	0	0	1	0	0

FIGURE D.3.6 The truth table for the 16 datapath control outputs, which depend only on the state inputs. The values are determined from Figure D.3.4. Although there are 16 possible values for the 4-bit state field, only ten of these are used and are shown here. The ten possible values are shown at the top; each column shows the setting of the datapath control outputs for the state input value that appears at the top of the column. For example, when the state inputs are 0011 (state 3), the active datapath control outputs are IorD or MemRead.

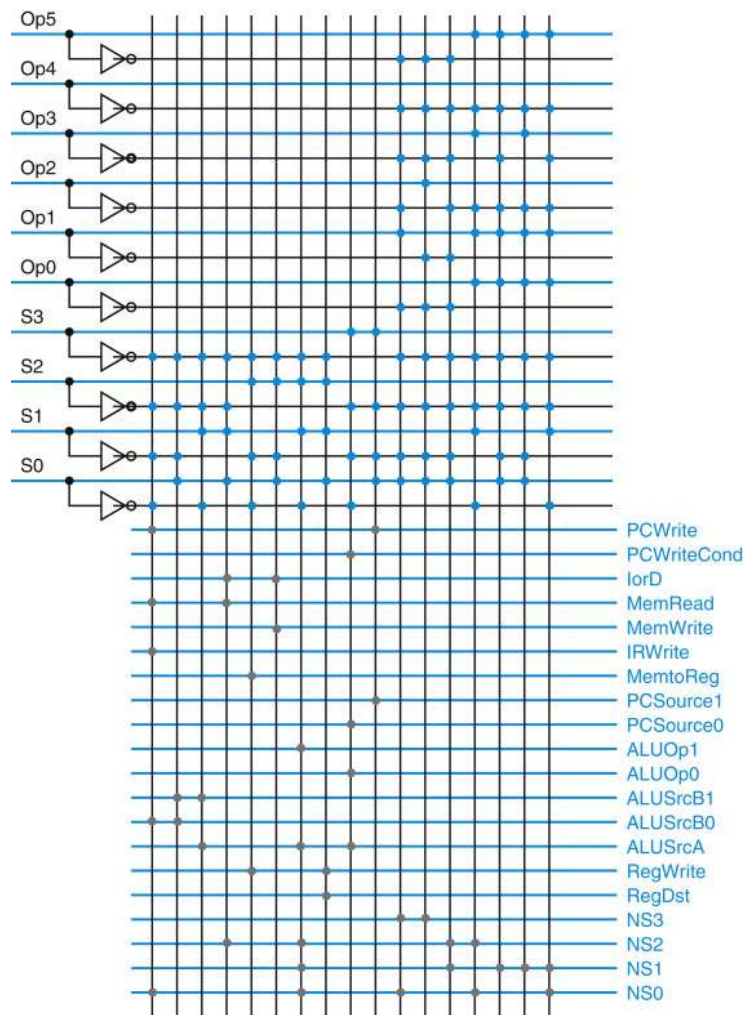


FIGURE D.3.9 This PLA implements the control function logic for the multicycle implementation. The inputs to the control appear on the left and the outputs on the right. The top half of the figure is the AND plane that computes all the minterms. The minterms are carried to the OR plane on the vertical lines. Each colored dot corresponds to a signal that makes up the minterm carried on that line. The sum terms are computed from these minterms, with each gray dot representing the presence of the intersecting minterm in that sum term. Each output consists of a single sum term.

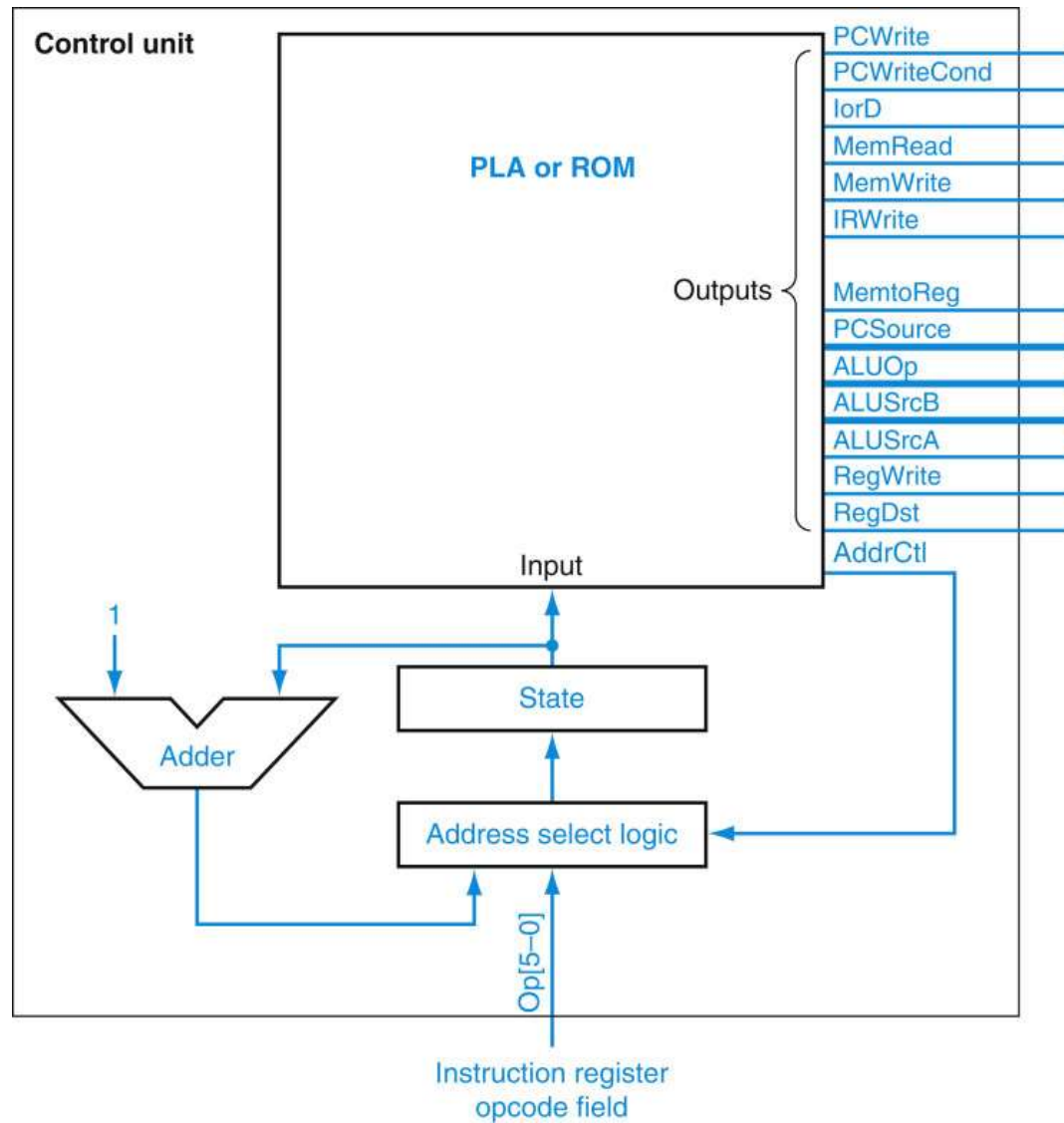


FIGURE D.4.1 The control unit using an explicit counter to compute the next state. In this control unit, the next state is computed using a counter (at least in some states). By comparison, Figure D.3.2 encodes the next state in the control logic for every state. In this control unit, the signals labeled *AddrCtl* control how the next state is determined.

為何今天已不採用單一週期設計

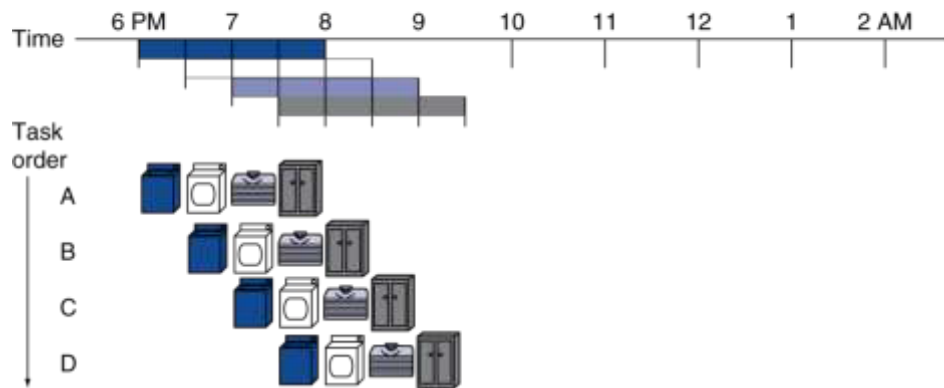
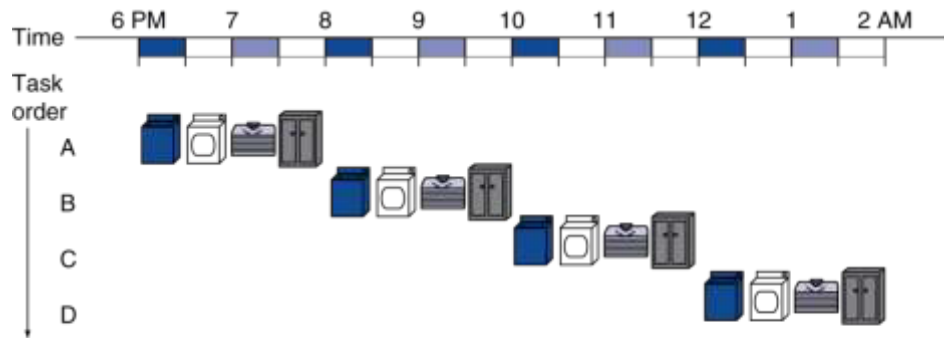
- 另一種稱為管線化(pipelining)的製作技法，其使用一個和單一週期數據通路非常類似的數據通路，然而具有很高的處理量因此非常有效率
 - 管線化藉由同時執行多道指令來增加效率

4.5 管線(管線、流水線)化處理概觀

- ▣ 管線化處理(pipelining)是一種能讓多個指令在執行時重疊(overlapped)的實作技巧
 - 目前，管線化幾乎已被普遍採用
- ▣ 管線化的方式需時遠較少，如圖4.25 所示
 - 只要我們有各別的資源提供給每一個階段，我們就可以將各項工作管線化
 - 管線化改善了處理量 (throughput)
 - 其並不會減少單獨一件工作所需時間

Pipelining Analogy

- ▣ Pipelined laundry: overlapping execution
 - Parallelism improves performance



- **Four loads:**

- **Speedup**
 $= 8/3.5 = 2.3$

- **Non-stop:**

- **Speedup**
 $= 2n/(0.5n + 1.5) \approx 4$
 $= \text{number of stages}$

MIPS Pipeline MIPS處理器管線

- ▣ Five stages, one step per stage
 1. IF: Instruction fetch from memory
 2. ID: Instruction decode & register read
 3. EX: Execute operation or calculate address
 4. MEM: Access memory operand
 5. WB: Write result back to register

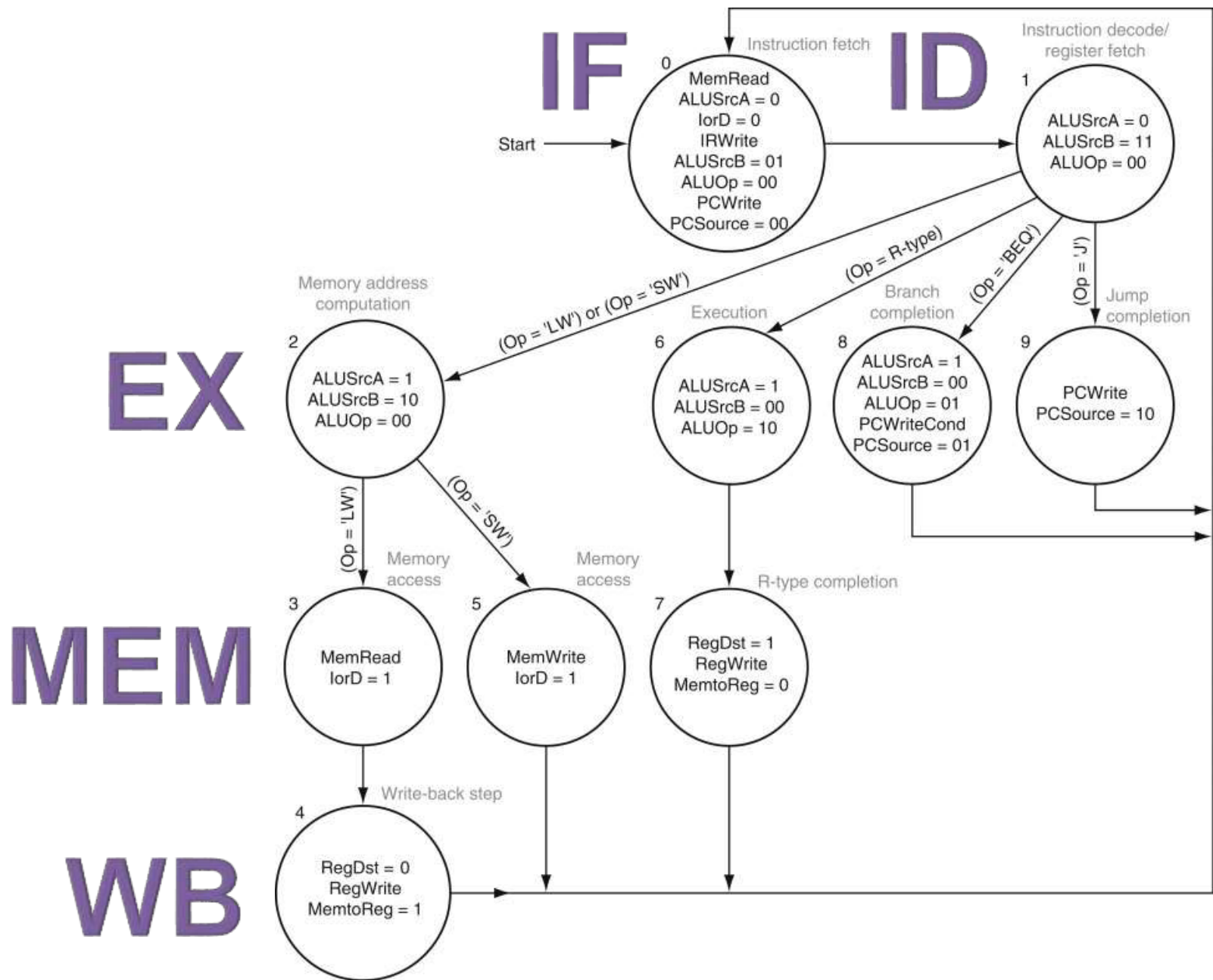


FIGURE D.3.1 The finite-state diagram for multicycle control.

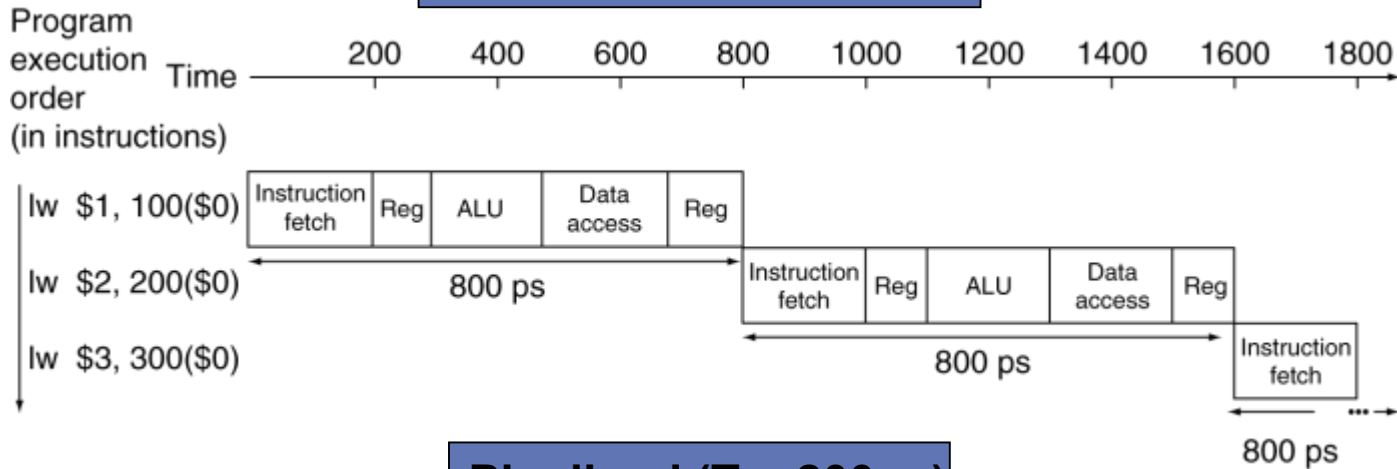
Pipeline Performance

- ▣ Assume time for stages is
 - 100ps for register read or write
 - 200ps for other stages
- ▣ Compare pipelined datapath with single-cycle datapath

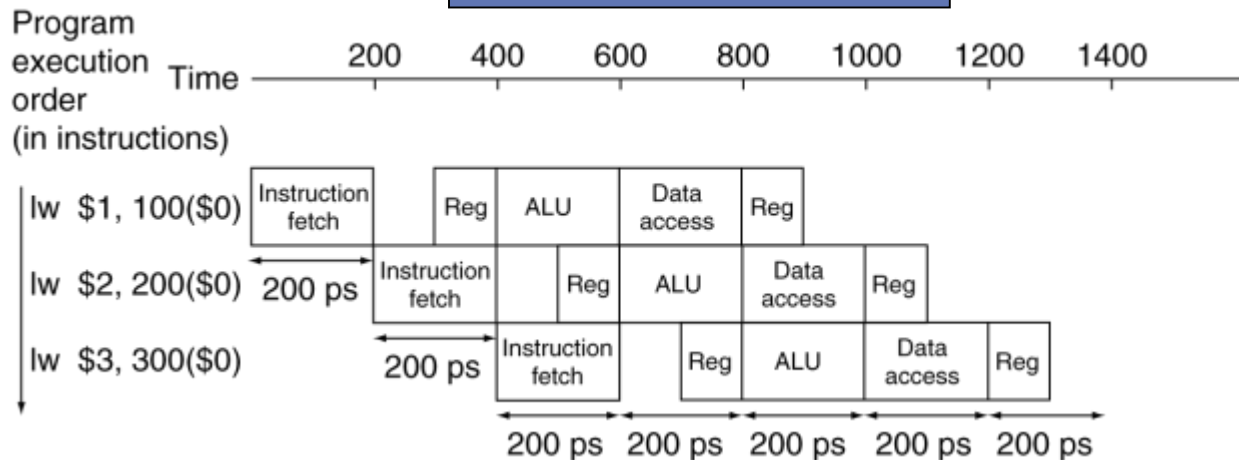
Instr	Instr fetch	Register read	ALU op	Memory access	Register write	Total time
lw	200ps	100 ps	200ps	200ps	100 ps	800ps
sw	200ps	100 ps	200ps	200ps		700ps
R-format	200ps	100 ps	200ps		100 ps	600ps
beq	200ps	100 ps	200ps			500ps

Pipeline Performance

Single-cycle ($T_c = 800\text{ps}$)



Pipelined ($T_c = 200\text{ps}$)



Pipeline Speedup

- ▣ If all stages are balanced

- i.e., all take the same time

- Time between instructions_{pipelined} =
$$\frac{\text{Time between instructions}_{\text{nonpipelined}}}{\text{Number of stages}}$$

- ▣ If not balanced, speedup is less

- ▣ Speedup due to increased throughput

- Latency (total time for each instruction) does not decrease

4.5 管線化處理概觀

▣ MIPS 的指令基本上需要五個步驟：

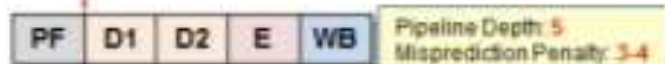
1. 從記憶體中擷取(fetch)指令
2. 解碼(decode)指令並同時讀取暫存器的值。MIPS 規律的指令格式允許解碼和讀取同時進行
3. 執行(execute)運作或計算位址
4. 存取數據記憶體(memory)中的運算元
5. 將結果寫回(write back)暫存器中

▣ 因此，我們在本章中探討的MIPS 管線具有五級

Intel x86 Pipeline History

<http://molesterwaterball.blogspot.com>
電腦水球部落格

Pentium
(P5 / P54C)



Pentium MMX
(P55C)



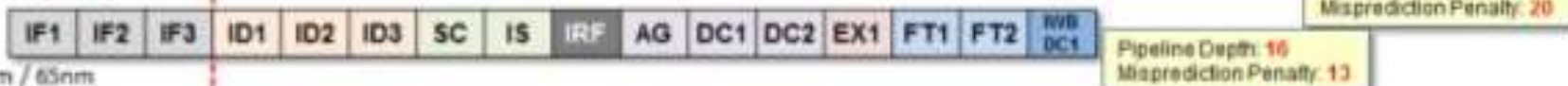
Intel P6 Family
Pentium M



180nm / 130nm
NetBurst



Bonnell (Atom)



90nm / 65nm
NetBurst



Core
(Yonah)



Core 2
(Merom / Penryn)



Nehalem
Westmere





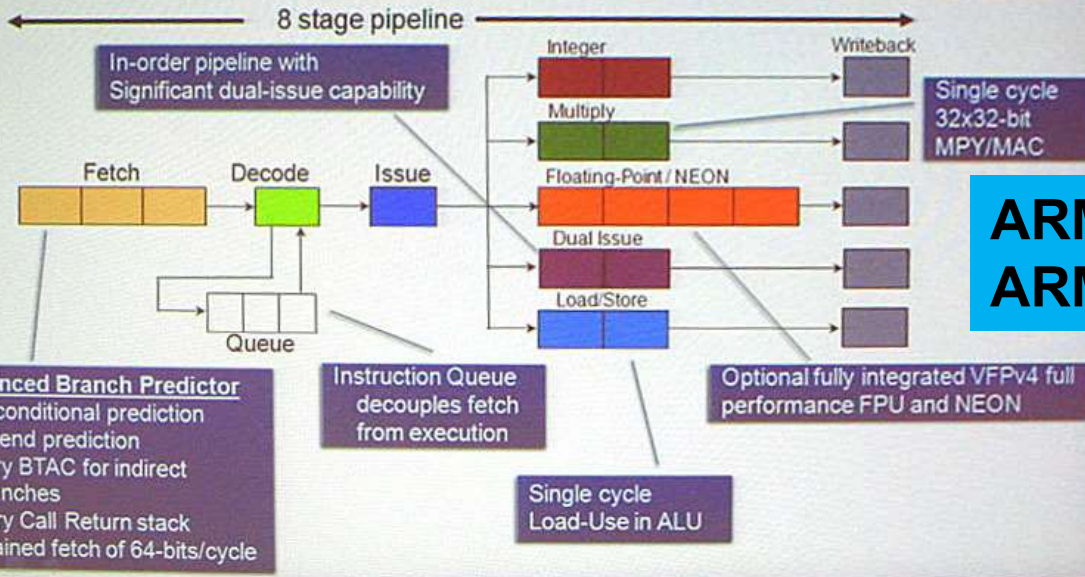
以下關於處理器的管線化設計的

① Start presenting to display the poll results on this slide.

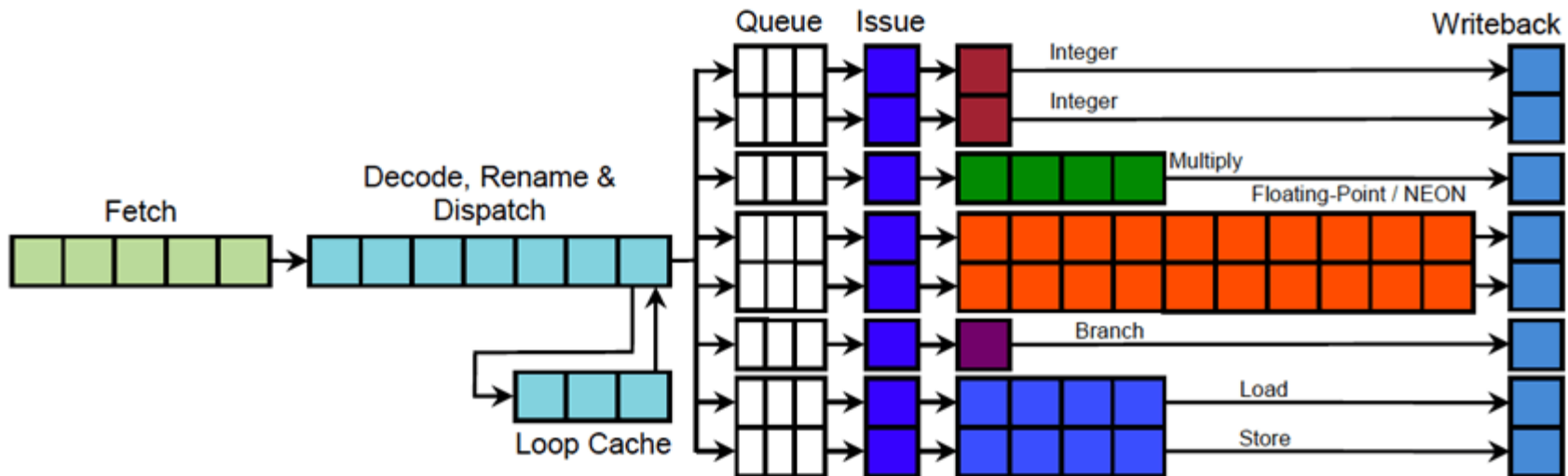
Cortex-A7 Pipeline

ARM®
2011 TechCon™
Join the community defining the future

- Design focused on power efficiency first
- Microarchitecture features improve performance over Cortex-A5 with little to no power impact



ARM Cortex A7: 8-10 stages
ARM Cortex A15: 15-28 stages



4.5 管線化處理概觀

- ▣ 在理想的情況下並且具有大量指令時，從管線化所得到的加速(speed-up)約略等於管線級數
 - 管線階段的時間可能不是那麼平均
 - 管線化會造成一些額外的負擔(overhead)
- ▣ 管線化藉著增加指令處理量來改善效能，而不是減少個別指令的執行時間
 - 由於真實程式會執行數以億計的指令，所以指令的處理量是重要的指標

為管線化來設計指令集

- MIPS 指令集設計的意涵，即是為了管線化的執行而設計
 - 第一，所有的MIPS 指令都擁有相同的長度
 - 第二，MIPS 只有少數幾種指令格式，並且每道指令中來源暫存器欄位都在相同的位置。解碼時可以同時開始讀取暫存器檔案
 - 第三，MIPS 中的記憶體運算元只出現於載入或儲存指令中。可以利用執行級來計算記憶體位址，然後於下一級存取記憶體



以下關於管線化處理器設計的敘述

① Start presenting to display the poll results on this slide.

管線的危障

- 在管線中有時下一道指令不能緊跟著在下個時脈週期中執行(停滯stall或等待wait情形)。這樣的情況稱為危障(hazards)，危障有三種不同的類型：

① 結構危障(structural hazard)

表示硬體無法支援我們要在同一個時脈週期中執行的指令的組合，如非哈佛式的von Neumann架構，單一(指令資料不分開)記憶體會造成不同指令IF級與MEM級的結構危障

② 數據危障(data hazards)：當管線因為某一個步驟必須等待其他步驟的完成而停滯(stall)時

- 因為一道指令的運算元輸入對稍早的一道指令的輸出結果有相依性(dependency)，且稍早的那道指令還在管線中未完成而發生

③ 控制危障(control hazard)



什麼是危障(hazard)：處理器管線

① Start presenting to display the poll results on this slide.



若處理器的記憶體，採用非哈佛式

① Start presenting to display the poll results on this slide.



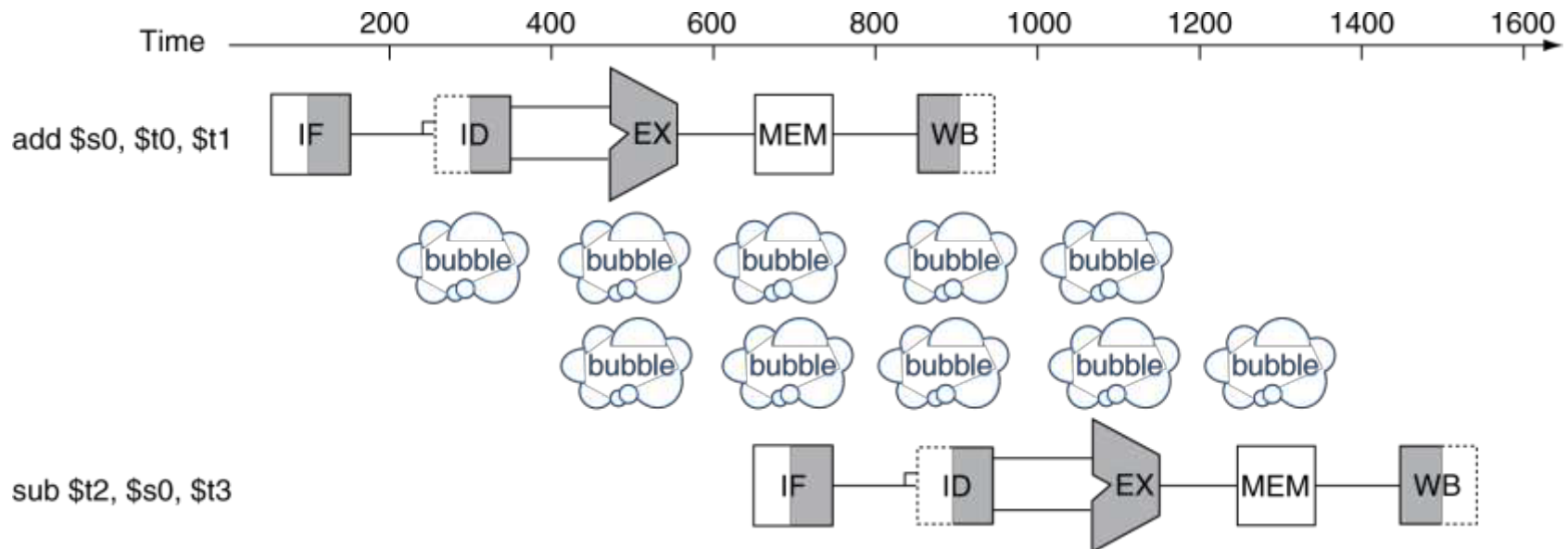
要管線化處理的指令中有資料相

① Start presenting to display the poll results on this slide.

(RAW) Data Hazards

- An instruction depends on completion of data access by a previous instruction, this is the **Read After Write** (RAW) Data Hazard

- add \$s0, \$t0, \$t1
sub \$t2, \$s0, \$t3



管線的數據危障

- 一旦ALU 產生加法的和，我們就可以把它送去當成減法的輸入
- 加入特殊的硬體來提早從內部資源獲取所需的資料稱為前饋(forwarding)或繞送(bypassing)
 - 圖4.29

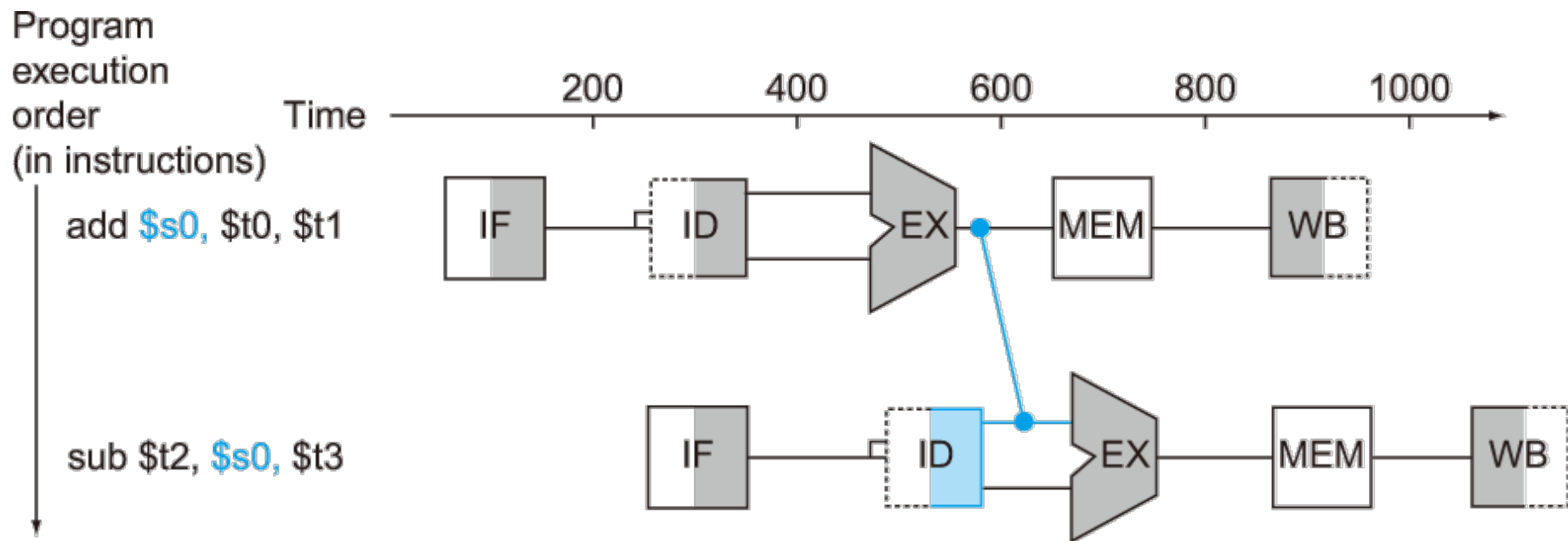


圖4.29 前饋的圖形表示法

連線表示從 **add** 的 *EX* 級輸出到 **sub** 的 *EX* 級輸入的前饋路徑，以取代 **sub** 在第二級從暫存器 **\$s0** 中讀取的值

管線的危障

② 數據危障(data hazards)

- 前饋並無法避免所有的管線停滯。
 - 仍需對載入－使用數據危障(load-use data hazard)停滯一級，如圖4.30 所示
 - 正式的名稱是管線停滯(pipeline stall)，但通常俗稱氣泡(bubble)或wait state

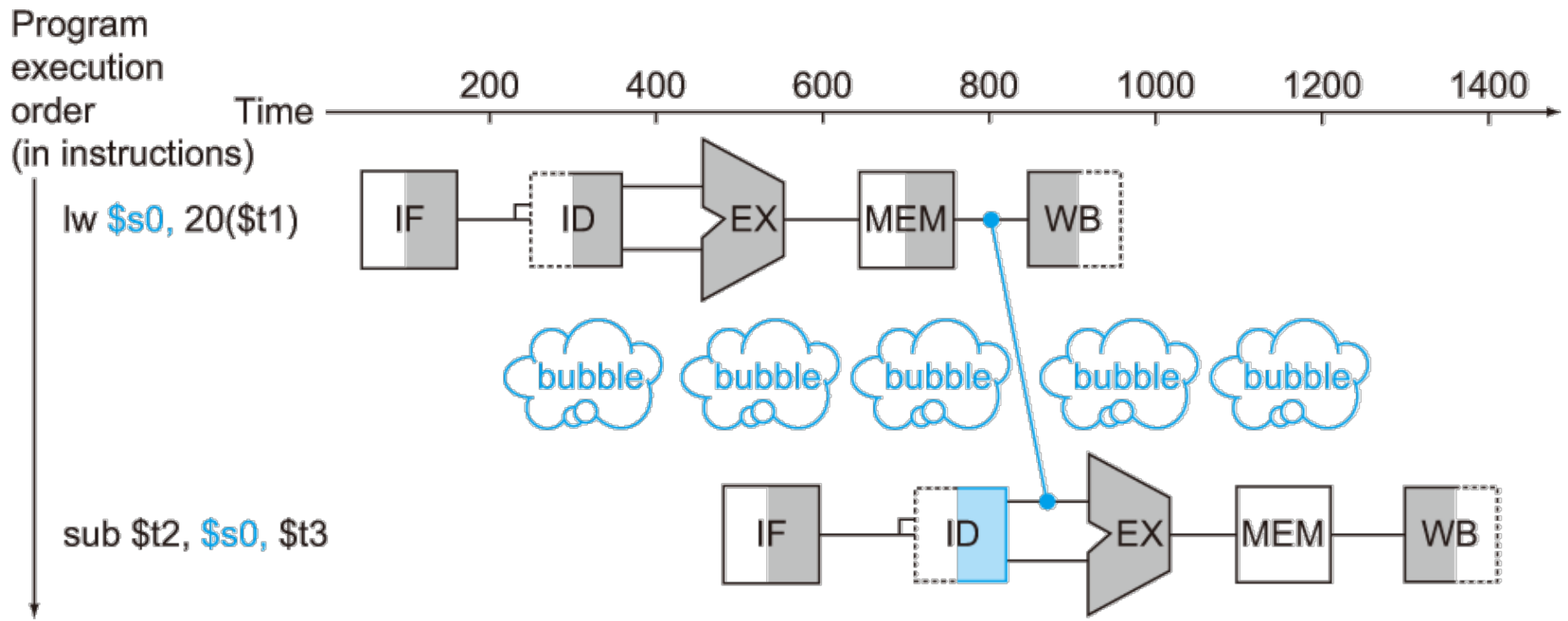


圖4.30 我們在R 格式的指令緊跟在載入指令之後並且要使用載入的數據時即使有前饋，還是需要停滯

如果沒有停滯，從記憶體存取階段輸出到執行階段輸入的路徑將會逆著時間走，這是不可能的。這個圖實際上作了簡化，因為我們在減法指令被擷取與解碼前，並不能知道是否需要暫停。4.7 節說明在這種危障下實際上會發生的細節

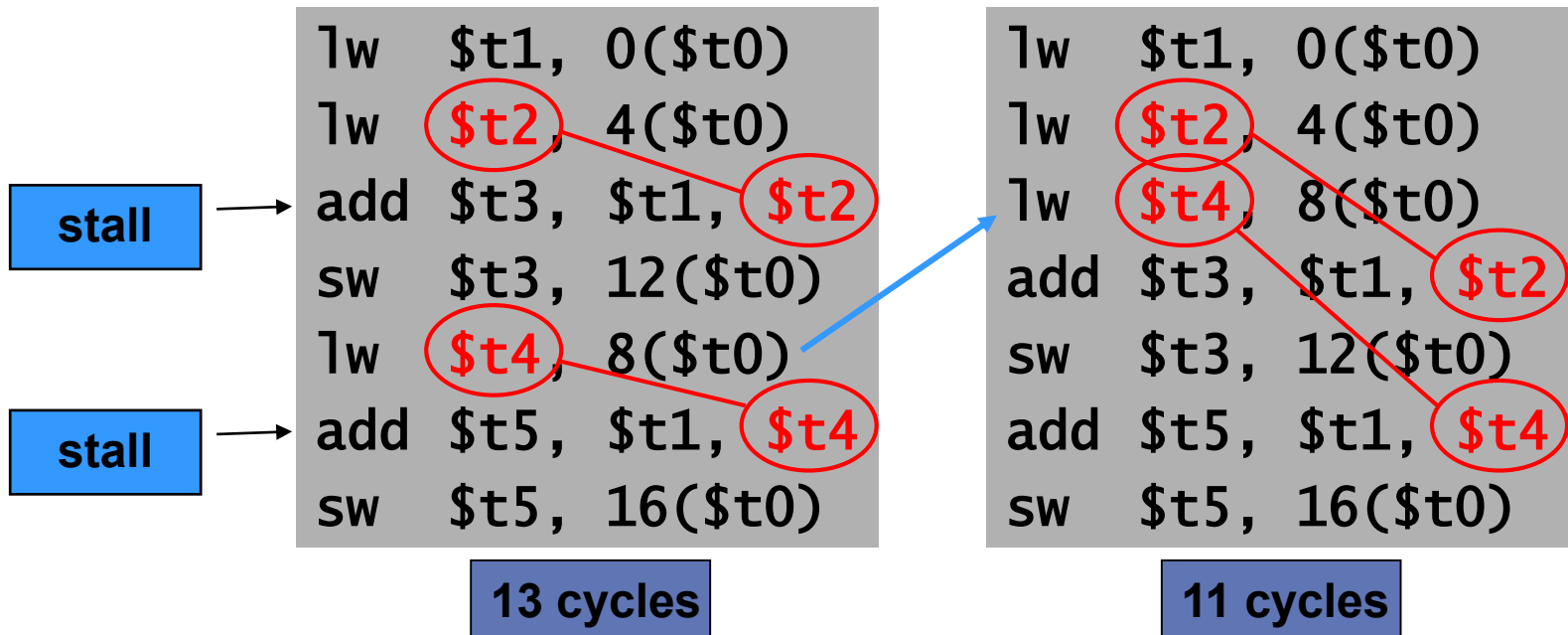


要處理(MIPS)處理器管線中的RA

① Start presenting to display the poll results on this slide.

Code Scheduling to Avoid Stalls

- ▣ Reorder code to avoid use of load result in the next instruction
- ▣ C code for $A = B + E; C = B + F;$





為了要避免因為**RAW**數據危障D

① Start presenting to display the poll results on this slide.

管線的危障

③ 控制危障(control hazard)

- 應對控制危障兩種解決方法中的第一種
 - 停滯(stall)：~ Stall for RAW hazard
保持循序地運作直到第一批制服烘乾，然後重複直到你得到正確的設定為止
- 停滯會造成嚴重的減速，導致了基於prediction理念的第二種解決方法
 - 預測(predict)：~ Forwarding for RAW hazard
假如你相當確定已掌握洗制服的正確設定，那就先預測它是對的並且在第一批制服烘乾的同時清洗第二批制服

□ 控制危障有第三種解法，稱為延遲分支(delayed branch) ~ Code Scheduling/Reorder Code for RAW Hazard

□ 將與分支無關的指令交換至分支指令之後，如下圖，add與beq指令無關，可移至beq之後

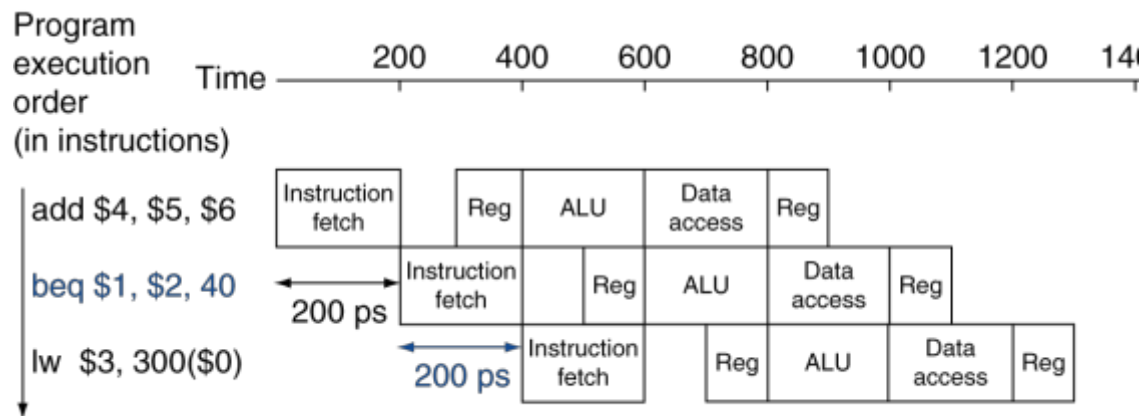
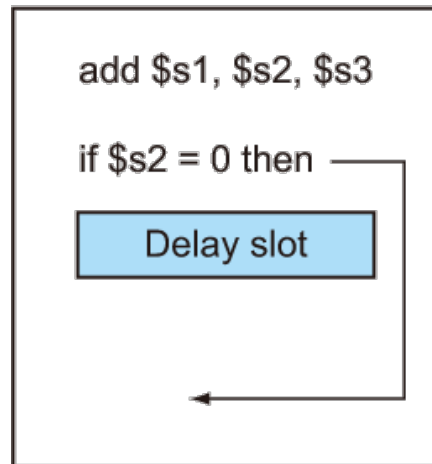
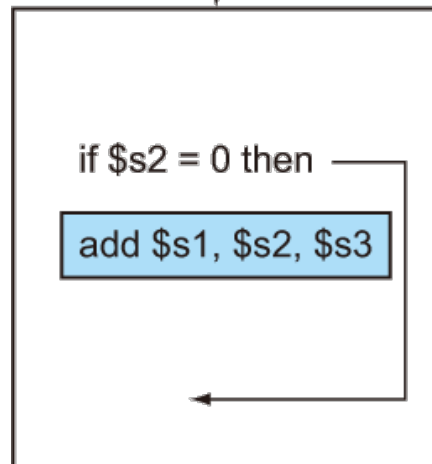


圖4.64 安排分支延遲槽

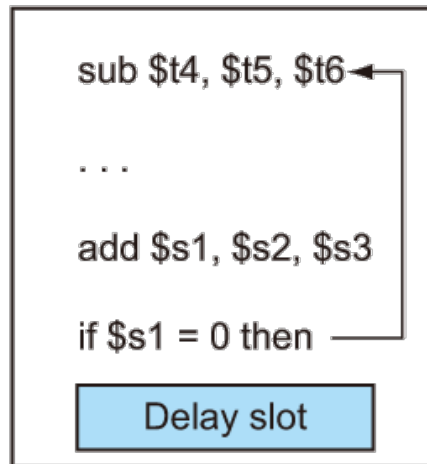
a. From before



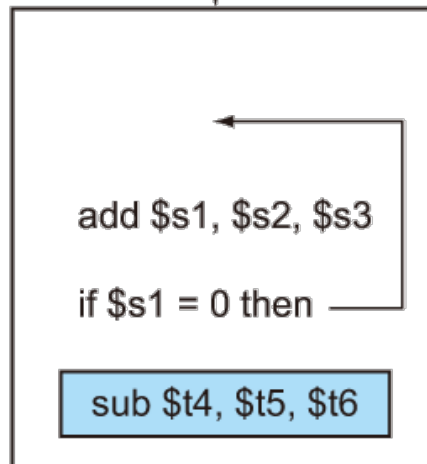
Becomes



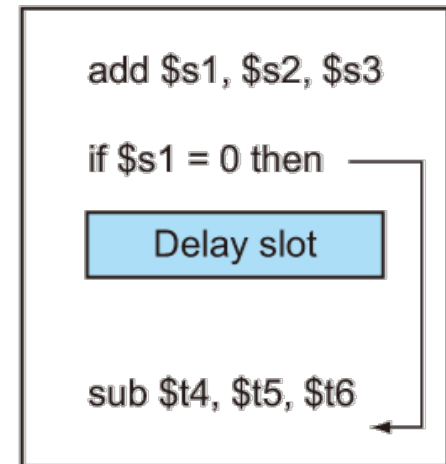
b. From target



Becomes



c. From fall-through



Becomes

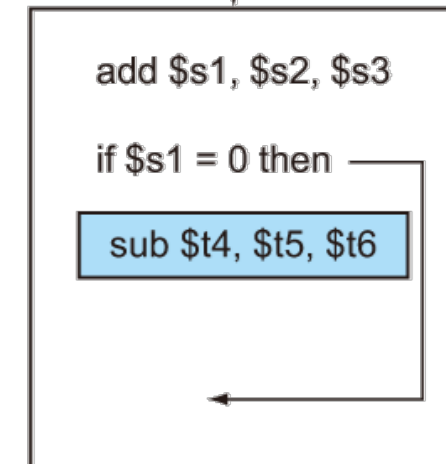


圖4.64 安排分支延遲槽

每一對方塊的上面那塊表示安排前的程式碼：下面那塊表示經過安排的程式碼。在(a)中，延遲槽內放進一道與分支無關的指令，這是最佳的選擇。策略(b)與(c)則是在(a)行不通時使用。在(b)與(c)的程式碼序列中，分支條件中使用了\$**s1**，造成add指令(其目的地暫存器為\$**s1**)無法被移到分支延遲槽中。在(b)中，分支延槽內放進分支的目標指令；通常該目標指令需要被複製過來，因為別的執行路徑可能還是會經過它。像是迴圈分支這類發生機率很高的分支適合採用策略(b)。最後，分支延遲槽也可以放進分支不發生的下一道指令如(c)所示。要使這個最佳化對(b)或(c)有效，在分支結果非如預期時，執行sub指令必須仍是OK的。OK 的意思是做了白工，但是程式仍然會正確地執行。例如，假如分支結果非如預期時\$**t4** 會是一個用不到的暫時暫存器

- 可以重新定義分支為一種特性而非控制危障：延遲的分支
 - ▣ 分支延遲槽(branch delay slot)
- 圖4.64 說明三種安插分支延遲槽的方法
- 延遲分支基本上要靠最佳化編譯器透過指令次序調動(instruction rescheduling) 來實現
- 但對於現代化深度管線化(**deeply pipelined**)的處理器，能使用的分支延遲槽非常有限，所以分支延遲delayed branch的方法實際上，應用性不大，需要訴諸分支預測的技術。



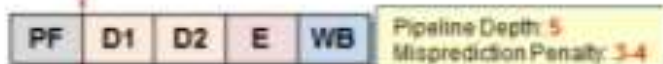
處理管線化處理器的控制危障Cc

① Start presenting to display the poll results on this slide.

Intel x86 Pipeline History

<http://molesterwaterball.blogspot.com>
電腦水球部落格

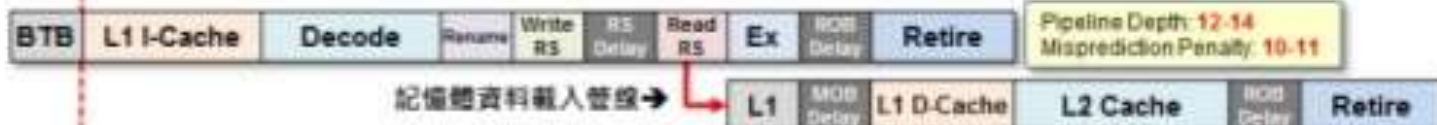
Pentium
(P5 / P54C)



Pentium MMX
(P55C)



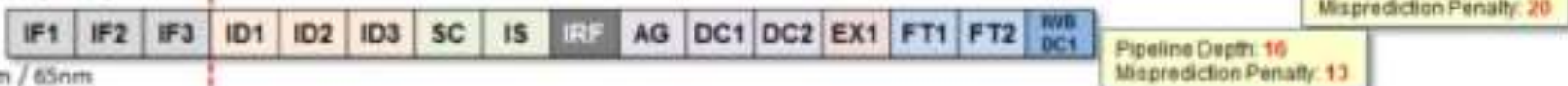
Intel P6 Family
Pentium M



180nm / 130nm
NetBurst



Bonnell (Atom)



90nm / 65nm
NetBurst



Core
(Yonah)



Core 2
(Merom / Penryn)



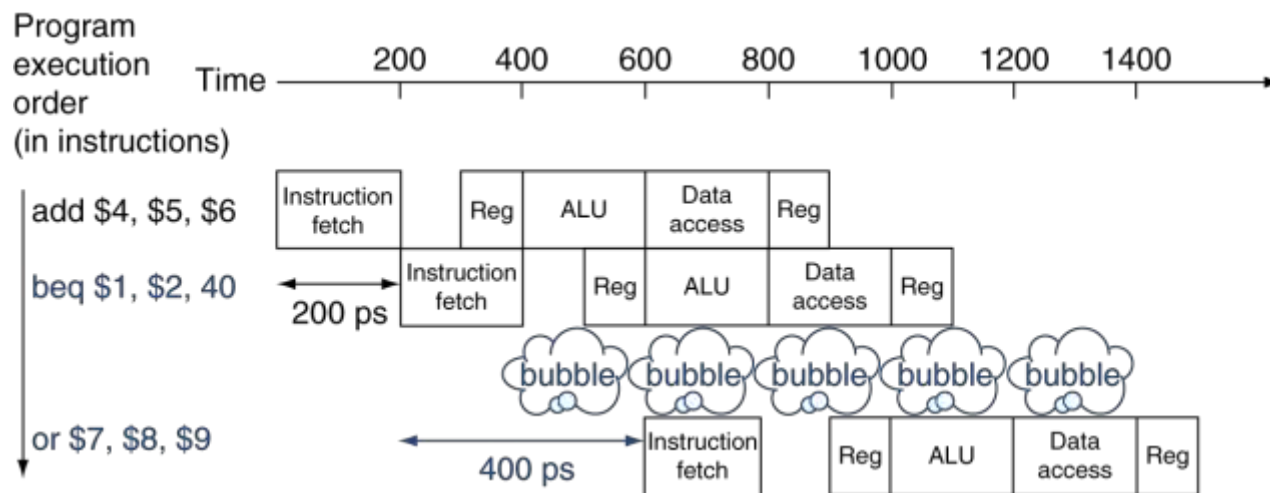
Nehalem
Westmere



Stall on Branch

(Worst cases: out of delay slots or mispredictions)

- Wait until branch outcome determined before fetching next instruction





何種狀況下，控制危障**Control H**

① Start presenting to display the poll results on this slide.

管線的危障

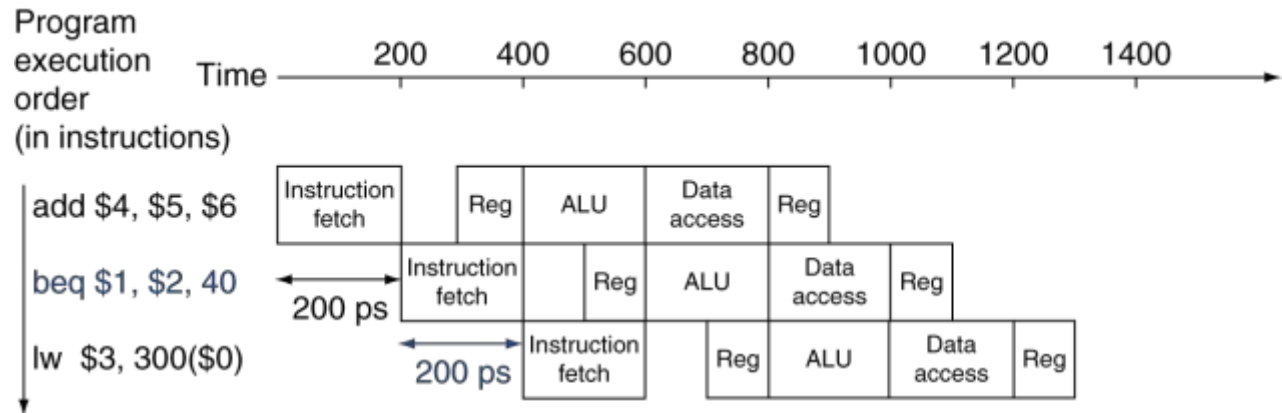
③ 控制危障(control hazard)

- 預測正確時並不會使管線減速。然而當你猜錯時，就必須清除預測錯誤造成的影響並且重新開始執行該做的工作
- 簡單的預測方法是預測分支永遠不發生或永遠發生(NotTaken/Taken)
- 更複雜的分支預測(branch prediction)將可以預測某些分支會發生而某些不發生
- 動態(dynamic)硬體預測器根據每道分支的行為來作預測，並且在程式運作過程中可以改變對一道分支的預測

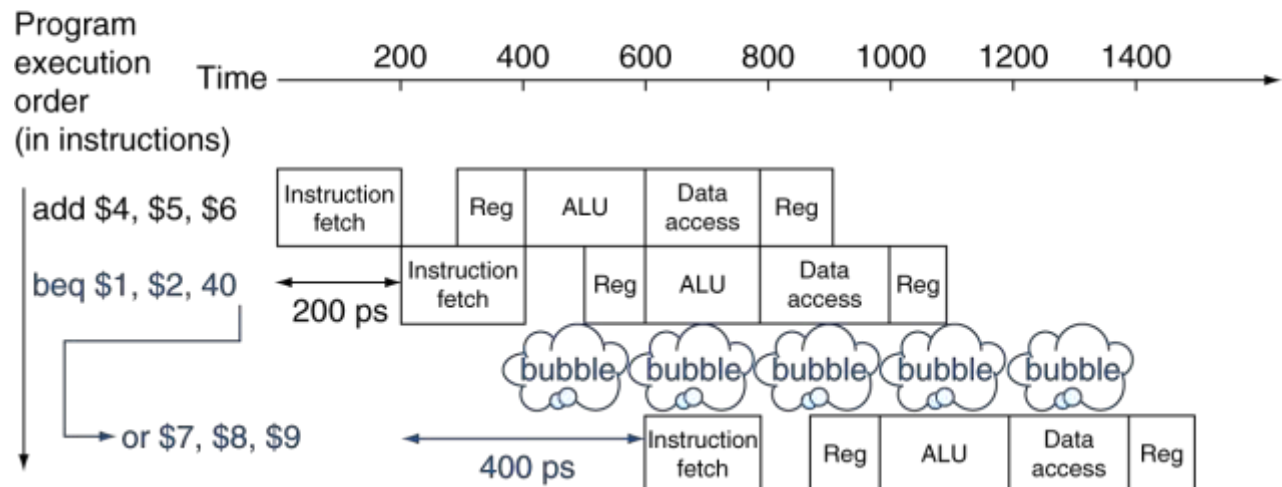
MIPS with Predict Not Taken

(預測分支永不發生)

**Prediction
correct**



**Prediction
incorrect**



管線化處理概觀總結

- ▣ 在循序指令流中開發指令間平行度(parallelism)
- ▣ 基本上是不需要程式師去過問的
- ▣ 管線化改善了指令的處理量然而反而會增加單一指令的執行時間或延遲

4.6 管線化數據通道及控制

- ▣ 圖4.33 表示了標記出管線階級後的第4.4 節中的單一週期數據通道
 - 5 級的管線
 - 代表在任一時脈週期內最多有5 道指令正在執行
 1. IF：指令擷取
 2. ID：指令解碼與暫存器檔案讀取
 3. EX：執行或位址計算
 4. MEM：數據記憶體存取
 5. WB：寫回

IF: Instruction fetch

ID: Instruction decode/
register file read

EX: Execute/
address calculation

MEM: Memory access

WB: Write back

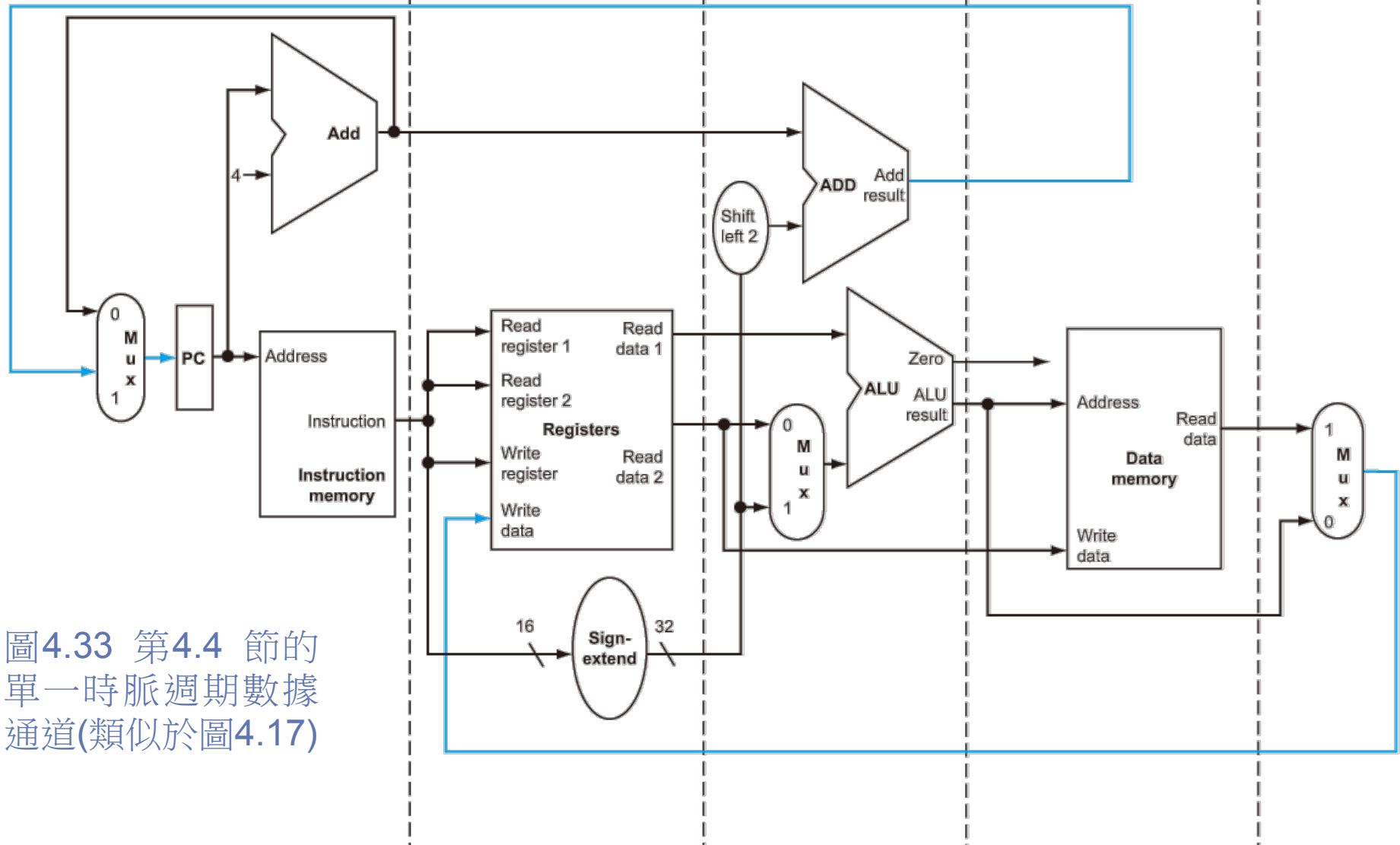


圖4.33 第4.4 節的
單一時脈週期數據
通道(類似於圖4.17)

圖4.33 第4.4 節的單一時脈週期數據通道(類似於圖4.17)

指令的每個步驟都能由左至右對應到數據通道中。例外的是以灰色表示的程式計數器(PC)的更新以及將ALU的結果或左方的記憶體的数据寫回暫存器檔案中的寫回(wite-back)。(一般我們會用灰色的線來代表控制訊號，但在本圖中灰色線則是代表數據線。)

4.6 管線化數據通道及控制

- 指令與數據大致由左向右經過五個階段的移動來完成執行。兩個例外：
 - 寫回階段，其將結果儲存回位於數據通道中間的暫存器檔案中
 - 下一個PC 值的選擇，在遞增的PC 以及由MEM 階段來的分支位址間選擇
 - 注意上述第一種右到左的數據移動可能導致數據危障而第二種可能導致控制危障
- 圖4.34 以在時間軸上畫出所有指令各自使用數據通道的方式來表示圖4.27 中指令的執行過程

圖4.34 指令用圖4.33 的單一時脈週期數據路徑假設以管線化的方式來執行

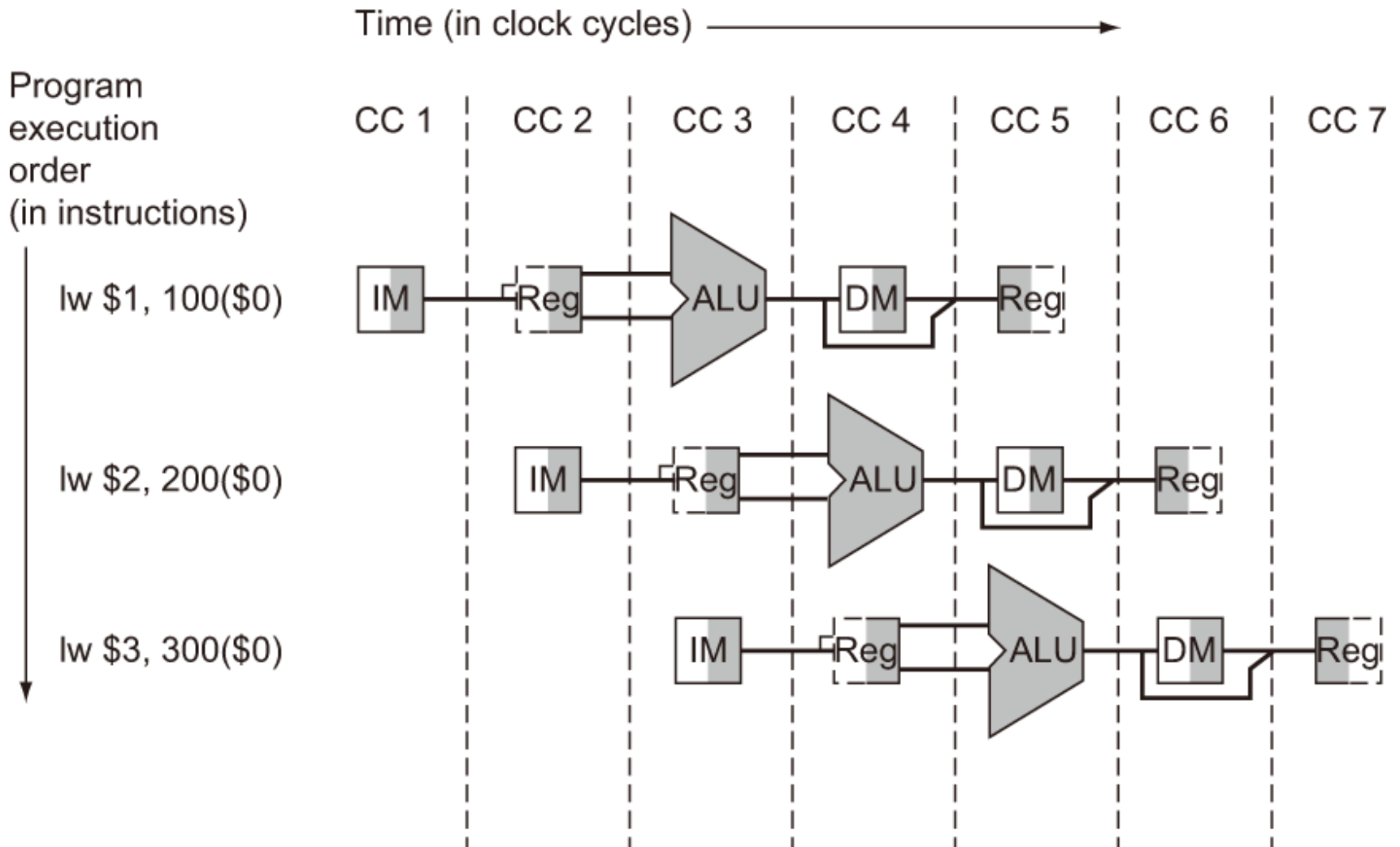


圖4.34 指令用圖4.33 的單一時脈週期數據路徑假設以管線化的方式來執行

這張圖類似於圖4.28 到圖4.30，假設每一個指令擁有自己的數據通道，並且將使用到的部分塗上陰影。不同於那些圖的是每個階段都標示出該級中使用到的對應到圖4.33 中數據通道該部分的實體資源。*IM* 代表指令擷取階段中的指令記憶體和程式計數器，*Reg* 代表指令解碼/暫存器檔案讀取階段(*ID*)中的暫存器檔案和符號延伸器，以及其他。為了維持適當的時間順序，暫存器檔案分成邏輯上的兩個部分：在暫存器擷取(*ID*)階段被讀取的暫存器和寫回(*WB*)階段被寫回的暫存器。在*ID* 階段內暫存器陣列並沒有被寫入，所以我們表示的方法是將左半邊用虛線表示且不加陰影；而在*WB* 階段內暫存器陣列並沒有被讀取，所以是將右半邊用虛線表示且不加陰影。我們如前假設暫存器檔案在前半時脈週期被寫入，而在後半時脈週期被讀取

4.6 管線化數據通道及控制

- ▣ 加入暫存器來保存數據，以便一個數據通道的不同部分在執行指令時可分別被使用
- ▣ 圖4.35 顯示強調管線暫存器(pipeline registers)的管線化數據通道
 - 這些暫存器以前後兩個階段的名稱來命名

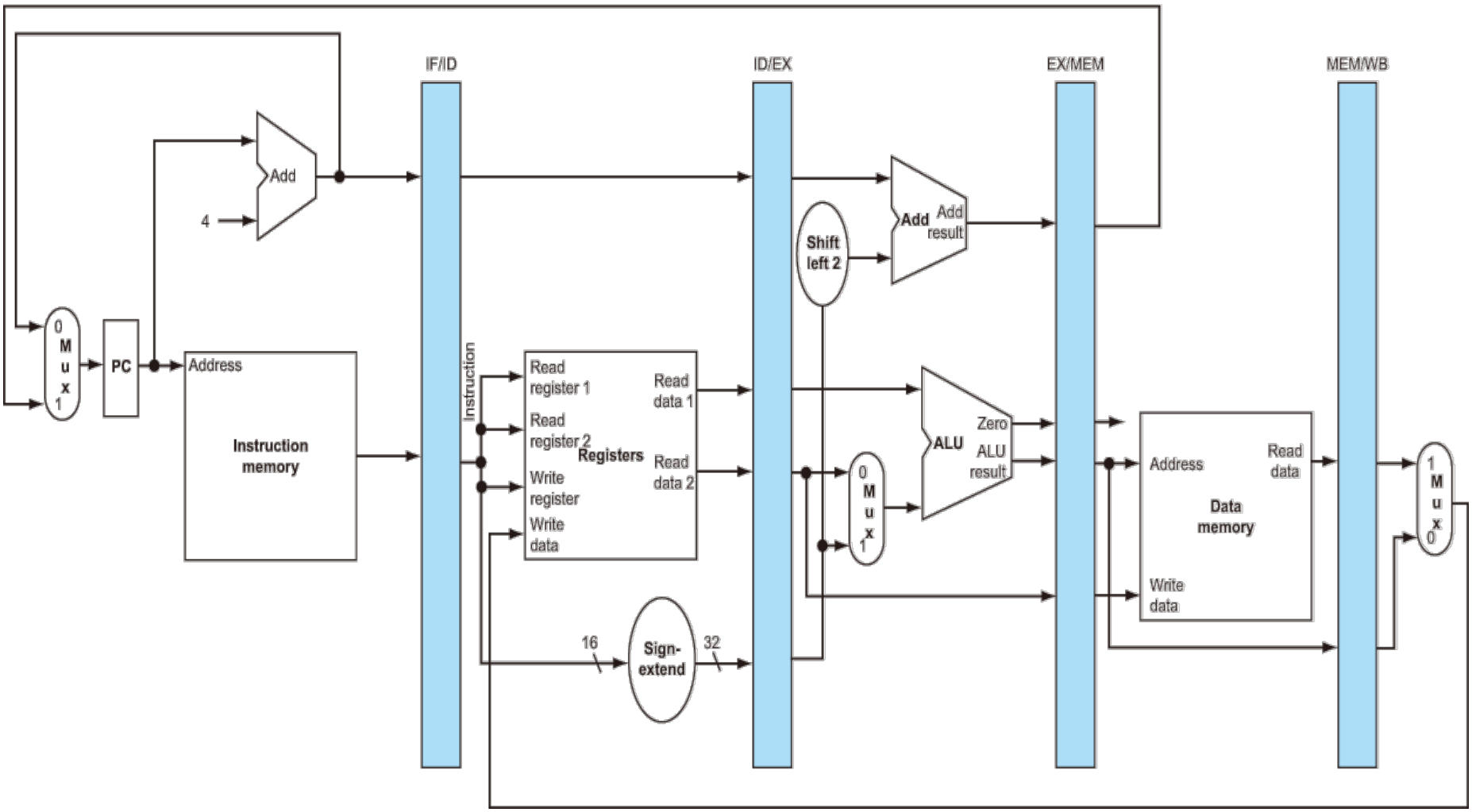


圖4.35 圖4.33 中數據通道的管線化版本

圖4.35 圖4.33 中數據通道的管線化版本

灰色的管線暫存器(**pipeline registers**)將每個管線階段隔開。它們以所隔開的階段來標示；例如第一個被標示為**IF/ID**，因為它將指令擷取和指令解碼階段隔開。這些暫存器必須夠寬以儲存所有會經過它的資料。例如**IF/ID** 暫存器必須是**64** 位元寬，因為它必須儲存從記憶體擷取的**32** 位元指令以及遞增後的**32** 位元**PC** 位址。目前另外的三個管線暫存器各包含**128**、**97** 和**64** 位元，不過我們會在本章稍後逐漸加寬這些暫存器

4.6 管線化數據通道及控制

- ▣ 圖4.36到圖4.38，顯示載入指令通過管線中五個階段時數據通道中強調出有動作的部分的情形
 - 任何在後方管線階段會使用到的資訊都必須透過管線暫存器來傳遞給各該階段
 - 數據通道中的每個邏輯元件如指令記憶體、暫存器讀取埠、ALU、數據記憶體和暫存器寫入埠只能用於唯一的管線階段期間，否則會發生結構危障(見275頁)
- ▣ 現在我們來發現在載入指令設計中的一個錯誤。你看出來了嗎？
 - 寫入暫存器的編號！

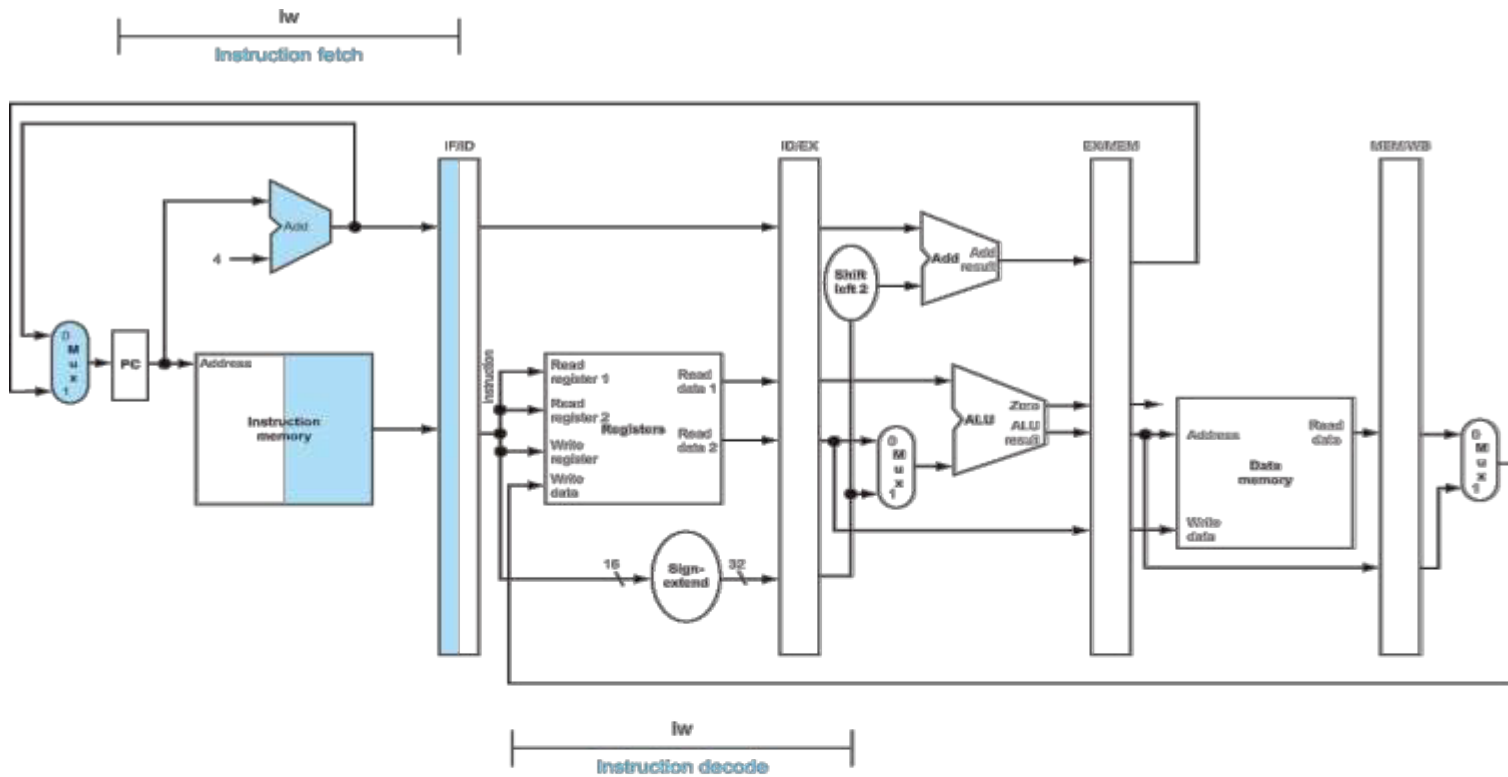
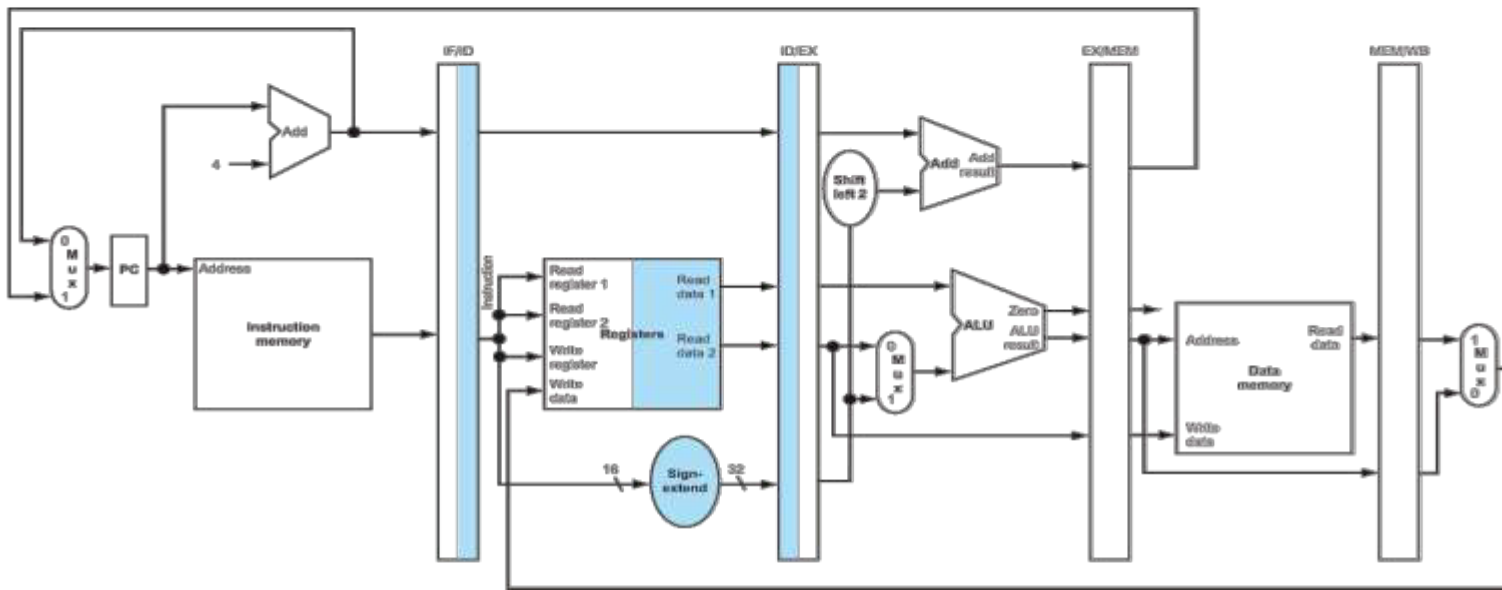


圖 4.36 IF 和 ID：指令的第一和第二个阶段，並將圖4.35的數據通道中運作的部分著灰色





要實現處理器的管線化設計，必須

① Start presenting to display the poll results on this slide.



MIPS處理器有五級的管線，有那

① Start presenting to display the poll results on this slide.

圖4.36 IF 和ID：指令的第一和第二個階段，並將圖4.35 的數據通道中運作的部分著灰色

著色的規則和圖4.28 中所使用的相同。如第4.2 節所述，因為暫存器的內容只會在時脈的邊緣才會改變，所以其讀取與寫入並不會發生混淆。雖然載入指令只需要在第2 級上方的暫存器內容，但是處理器並不知道目前被解碼的是什麼指令，所以符號延伸**16**位元的結果以及讀取的兩個暫存器內容都會被儲存到**ID/EX**管線暫存器中。雖然我們不全需要這三個運算元，但是三個都保存可以簡化控制

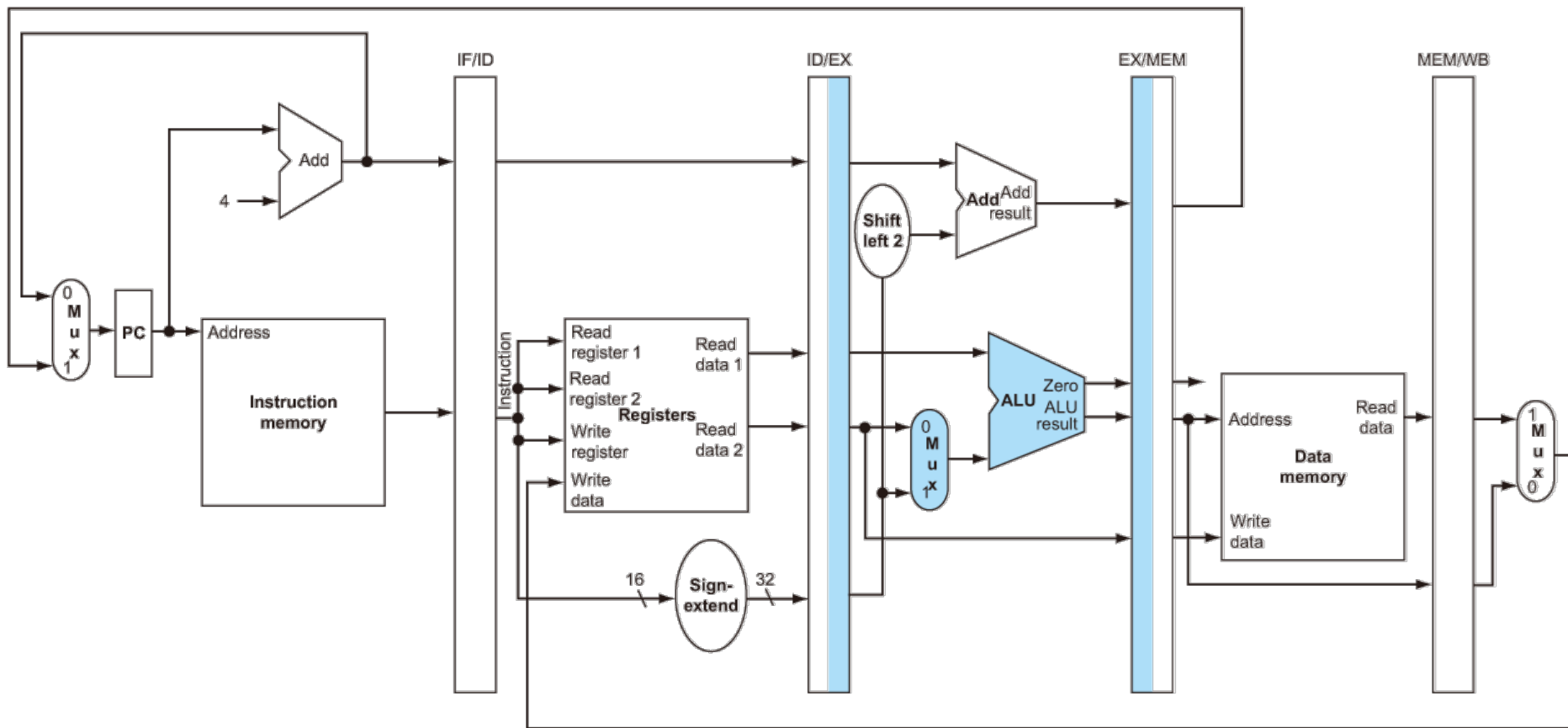


圖4.37 EX：載入指令的第三個管線階段，並將圖4.35 的數據通道中在這個管線階段用到的部分著灰色

暫存器的值和符號延伸後的立即值相加，並將和放置在EX/MEM 管線暫存器中

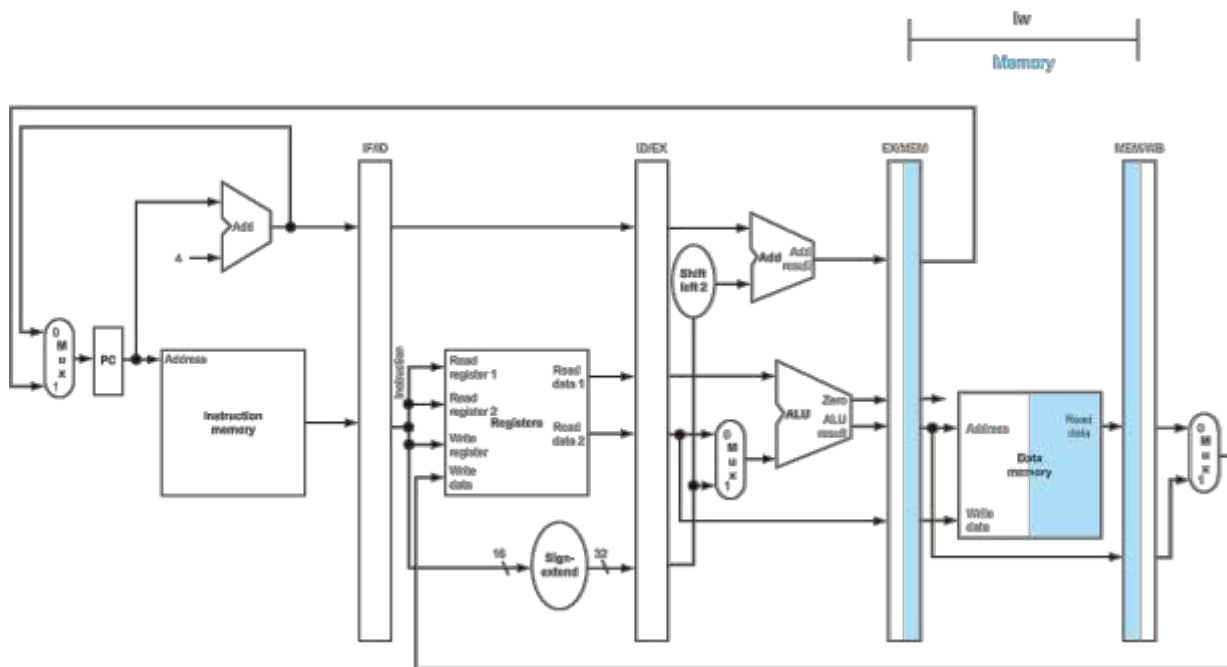
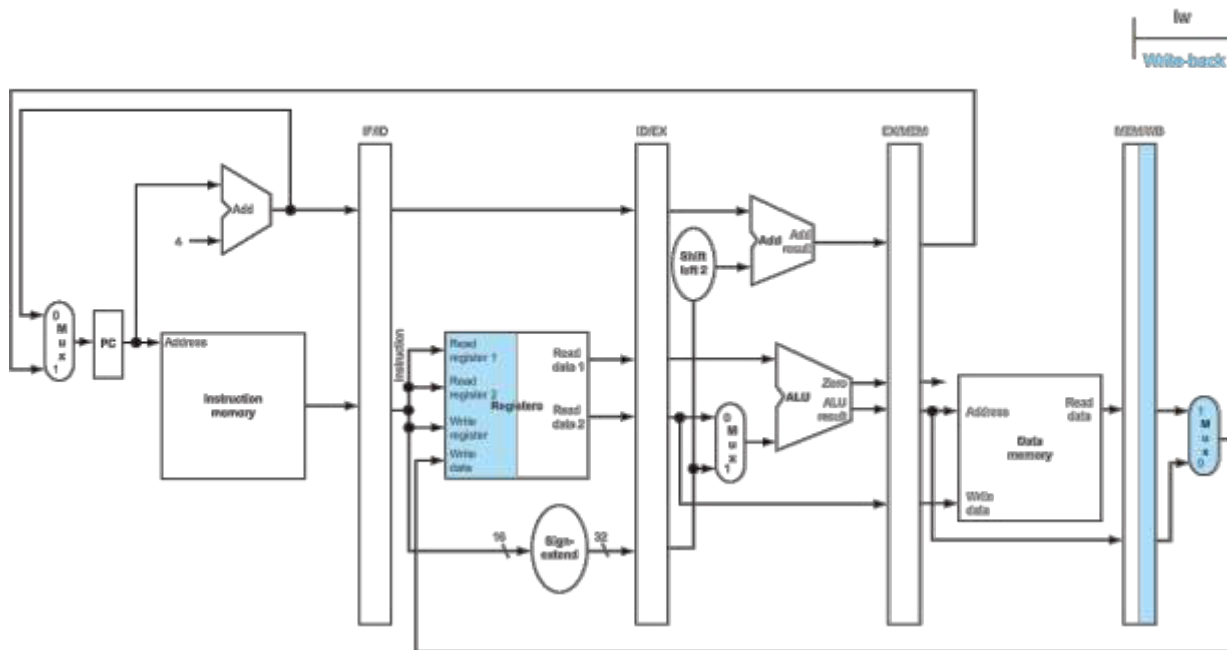


圖 4.38 MEM 和 WB：載入指令的第四個和第五個管線階段，並將圖 4.35 的數據通道中在該管線階段用到的部分著灰色。

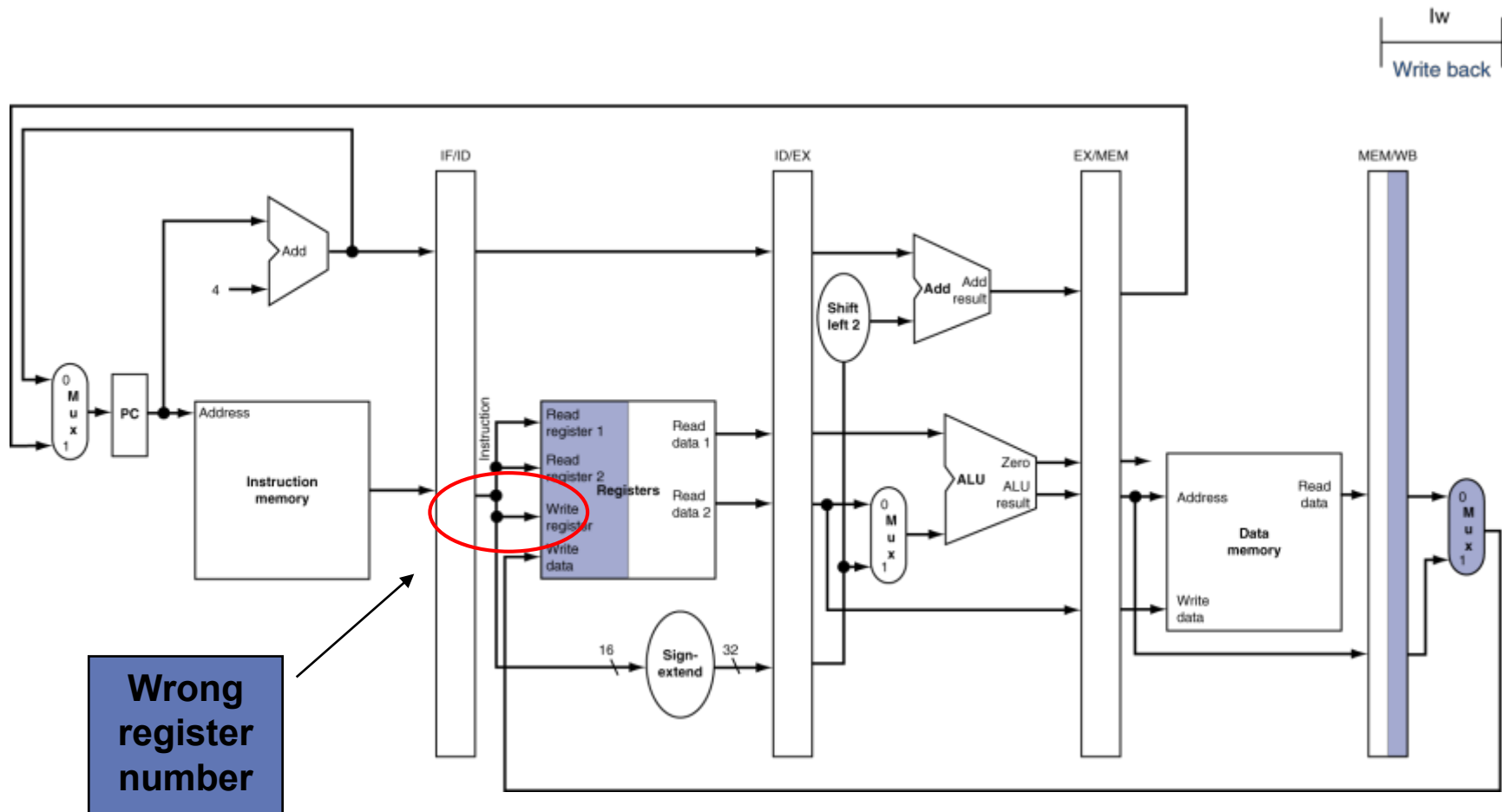
使用 EX/MEM 管線暫存器中的位址來讀取數據記憶體，並將數據放置在 MEM/WB 管線暫存器中。接下來，數據從 MEM/WB 管線暫存器讀取並寫入圖中間的暫存器檔案中。注意：這個設計有一個錯誤並將於圖 4.41 中修正。



4.6 管線化數據通道及控制

- 我們需要將IF 階段所擷取到的指令一路上都保存下來，因此每個管線暫存器會保留一道指令在目前這一級及之後各級所需的內容
- 圖4.41 表示了正確的數據通

WB for Load



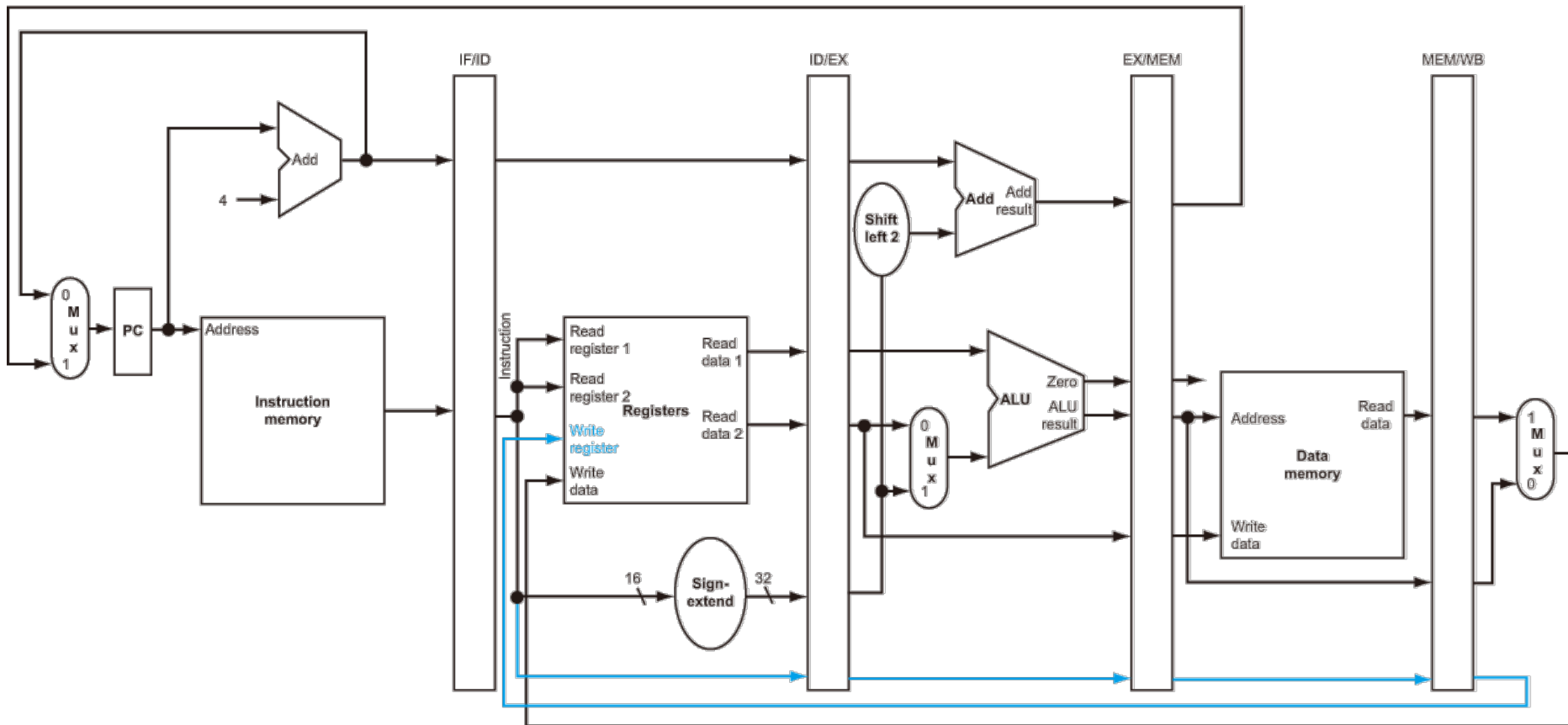


圖4.41 為了正確處理載入指令而修正後的管線化數據通道

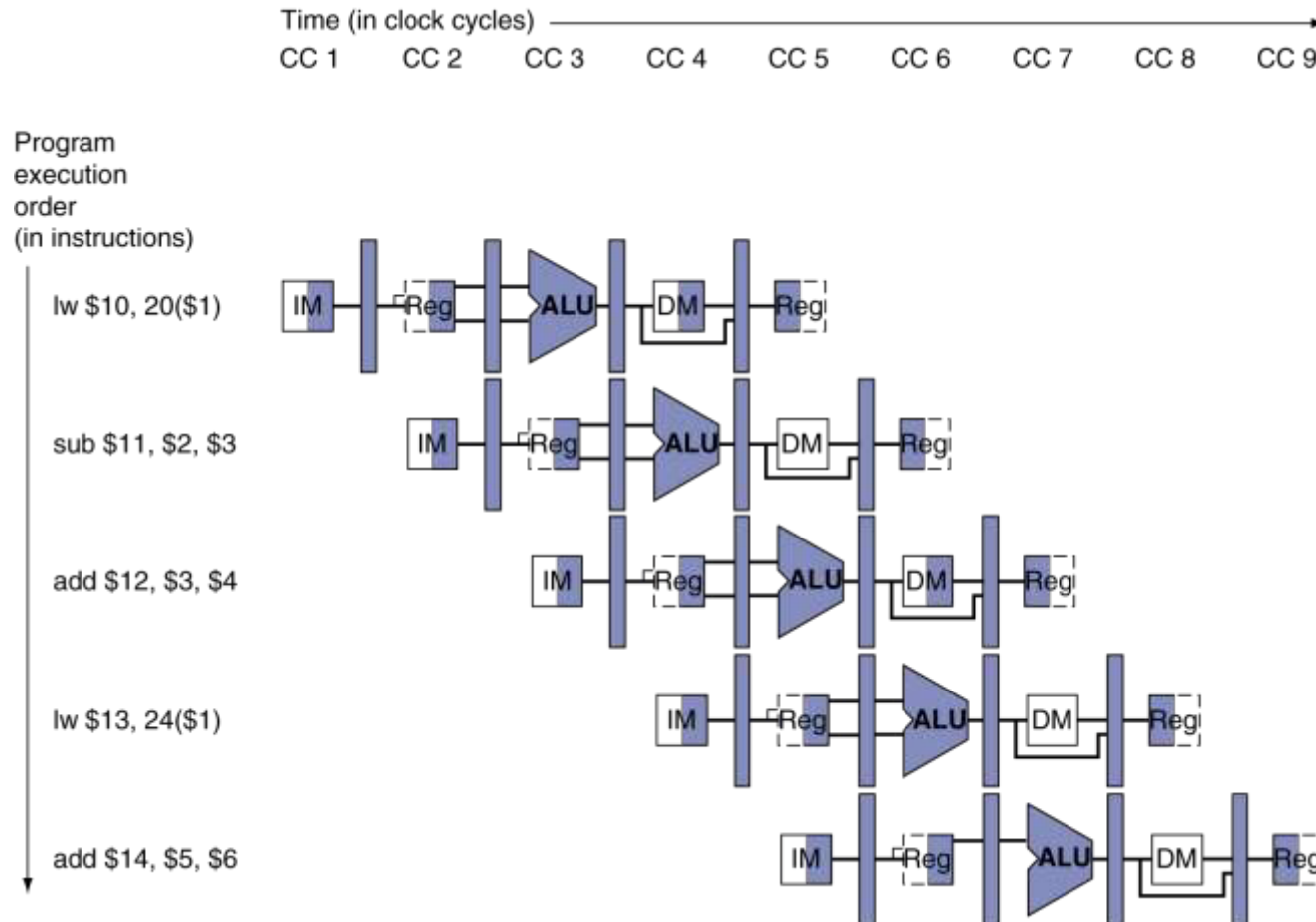
現在寫入暫存器編號與數據都來自MEM/WB 管線暫存器。這個暫存器編號從ID 管線階段傳送過來直到它到達MEM/WB 管線暫存器為止，使得最後三個管線暫存器中各增加5 個位元。圖中新增的路徑以灰色表示

以圖形來表示管線

- 有兩種基本型態的圖形化表示法
 - 如同圖4.34 的多個時脈週期管線圖(multiple-clock-cycle pipeline diagram)
 - 如同圖4.36 到圖4.40 的單一時脈週期管線圖

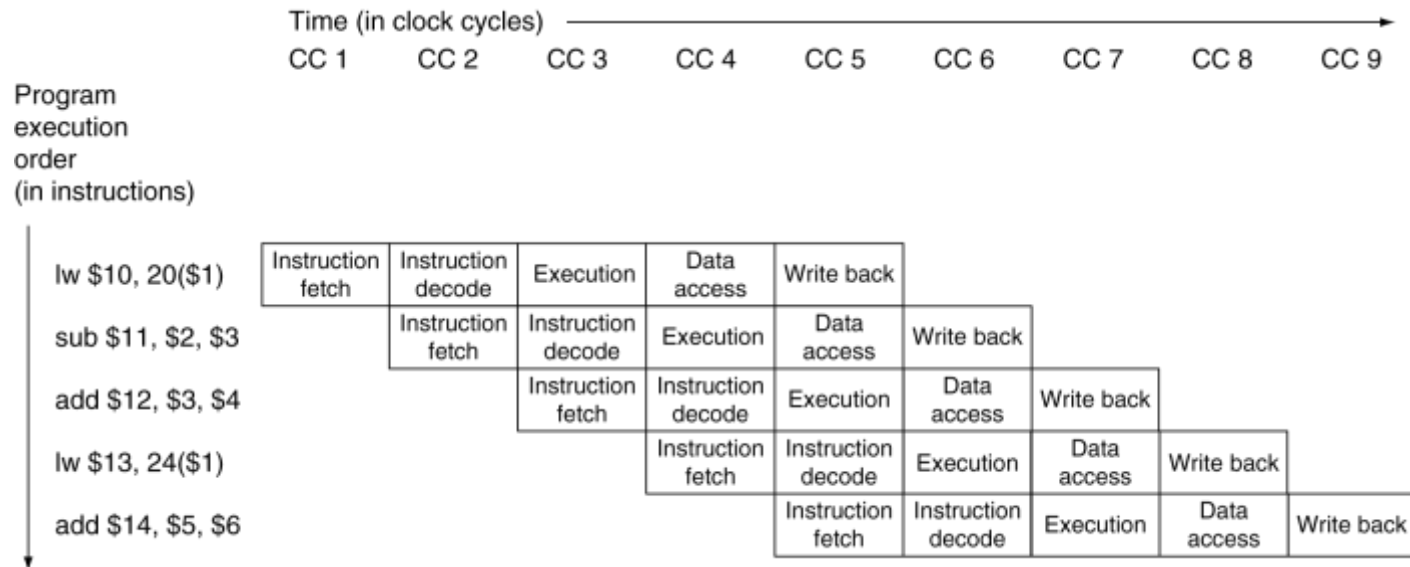
Multi-Cycle Pipeline Diagram

- Form showing resource usage



Multi-Cycle Pipeline Diagram

▣ Traditional form



管線化中的控制

- 第一步是在既有的數據通道中的控制訊號線作標記。
 - 圖4.46 表示這些控制訊號線。

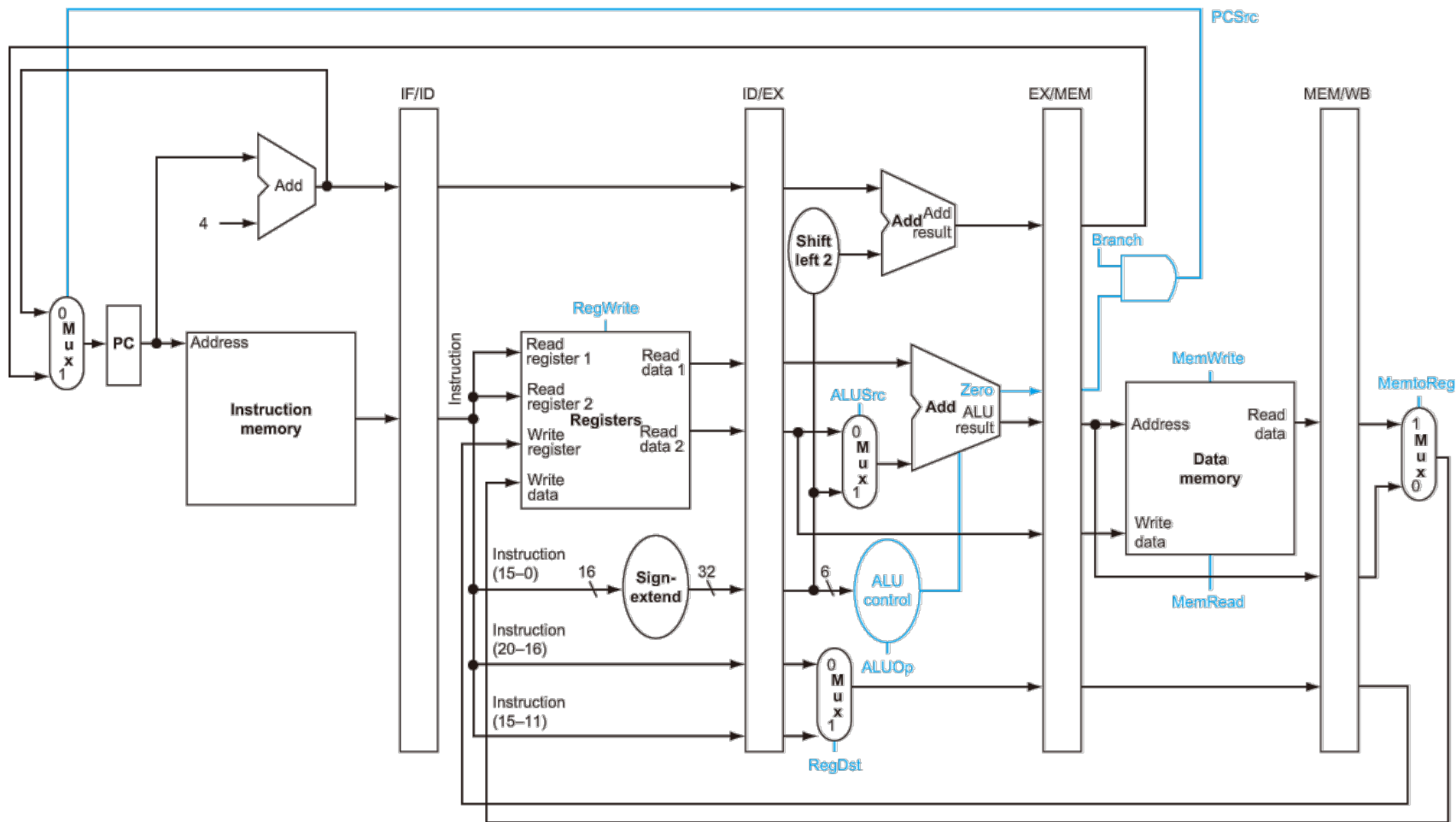


圖4.46 標出控制訊號後的圖4.41 中的管線化數據通道

此數據通道加上了4.4 節中的PC 來源控制邏輯、目的暫存器編號以及ALU 控制。注意現在我們在EX 階段中需要指令的6 位元的功能碼欄位來作為ALU 控制器的輸入，所以這些位元也必須包括在ID/EX 管線暫存器中。回想一下這6 位元其實就是立即值欄位中最低的6 個位元，所以ID/EX 管線暫存器可以由立即值欄位來提供它們，因為符號延伸後這些位元仍是不變的

管線化中的控制

▣ 圖4.47 到圖4.49 中重新將這些重要的資訊呈現

指令運作碼	ALUOp	指令運作	功能欄位	需要的 ALU 動作	ALU 控制輸入
LW	00	載入字組	XXXXXX	加	0010
SW	00	儲存字組	XXXXXX	加	0010
Branch equal	01	若等於則分支	XXXXXX	減	0110
R 型	10	加	100000	加	0010
R 型	10	減	100010	減	0110
R 型	10	及	100100	及	0000
R 型	10	或	100101	或	0001
R 型	10	若小於則設定	101010	若小於則設定	0111

圖4.47 此即圖4.12

這個圖表示ALU 控制位元如何根據ALUOp 控制位元與R 型指令的不同功能碼來設定

訊號名稱	未被設定時的效果	被設定時的效果
RegDst	Write register 的目的暫存器編號來自 rt 欄位(指令的 20:16 位元)。	Write register 目的暫存器編號來自 rd 欄位(指令的 15:11 位元)。
RegWrite	無。	將 Write data 輸入的值寫入 Write register 輸入所指定的暫存器中。
ALUSrc	ALU 的第二個運算元來自暫存器檔案第二輸出埠(Read data 2)。	ALU 的第二個運算元來自指令中最低的 16 個位元經過符號延伸後的值。
PCSrc	由計算 $PC + 4$ 的加法器的輸出來替換原本 PC 的值。	由計算分支目標位址的加法器的輸出來替換原本 PC 的值。
MemRead	無。	將數據記憶體中 address 輸入所指定位址的內容送至 Read data 輸出。
MemWrite	無。	將數據記憶體中 address 輸入所指定位址的內容置換為 Write data 輸入的值。
MemtoReg	暫存器 Write data 輸入的值來自 ALU。	暫存器 Write data 輸入的值來自數據記憶體。

圖4.48 此即圖4.16

七個控制訊號均在此定義。ALU 的控制線(ALUOp)定義在圖4.47 的第二行。當一個二選一多工器的一位元控制訊號被設定(asserted)時，多工器選擇對應到1 的輸入。若該控制訊號沒被設定時，多工器選擇對應到0 的輸入。注意PCSrc 是由圖4.46 中的AND 閘所控制。在Branch 訊號與ALU Zero 訊號都為1 時，PCSrc才是1；否則其為0。控制器只有在執行beq 時才會設定Branch 訊號；否則PCSrc 設為0

指令	執行 / 記憶體位址計算階段 控制訊號線				記憶體存取階段 控制訊號線			寫回階段 控制訊號線	
	Reg-Dst	ALU-Op1	ALU-Op0	ALU-Src	Branch	Mem-Read	Mem-Write	Reg-Write	Memto-Reg
R 格式	1	1	0	0	0	0	0	1	0
lw	0	0	0	1	0	1	0	1	1
sw	X	0	0	1	0	0	1	0	X
beq	X	0	1	0	1	0	0	0	X

圖4.49 控制線的訊號值與圖4.18 中相同，不過已經分別對應最後的三個管線階段分成三群

管線化中的控制

- 不需要特別的PC 寫入訊號
- IF/ID、ID/EX、EX/MEM 和MEM/WB 這些管線暫存器也不需要特別的寫入訊號
- 根據管線的五級，把控制訊號線分成五組：
 1. 指令擷取：並沒有什麼特別要控制的
 2. 指令解碼/暫存器檔案讀取：並沒有需要設定的控制線
 3. 執行/記憶體位址計算：要被設定的控制訊號有RegDst、ALUOp以及ALUSrc
 4. 記憶體存取：要設定的控制訊號有Branch、MemRead與MemWrite

管線化中的控制

- 5. 寫回：兩條控制訊號線。MemtoReg 決定要將 ALU 運算結果或是來自記憶體的值送到暫存器檔案；而RegWrite 用來將所選的值寫入
 - 將數據通道管線化之後也不會改變控制線的意義
- 控制單元的實作就是針對每個指令在每一級中設定這九條控制訊號線的值
 - 最簡單的方法就是將管線暫存器擴充以容納這些控制訊號
 - 圖4.50 顯示這些控制訊號隨著指令在管線中前進而用於適當的管線階段中

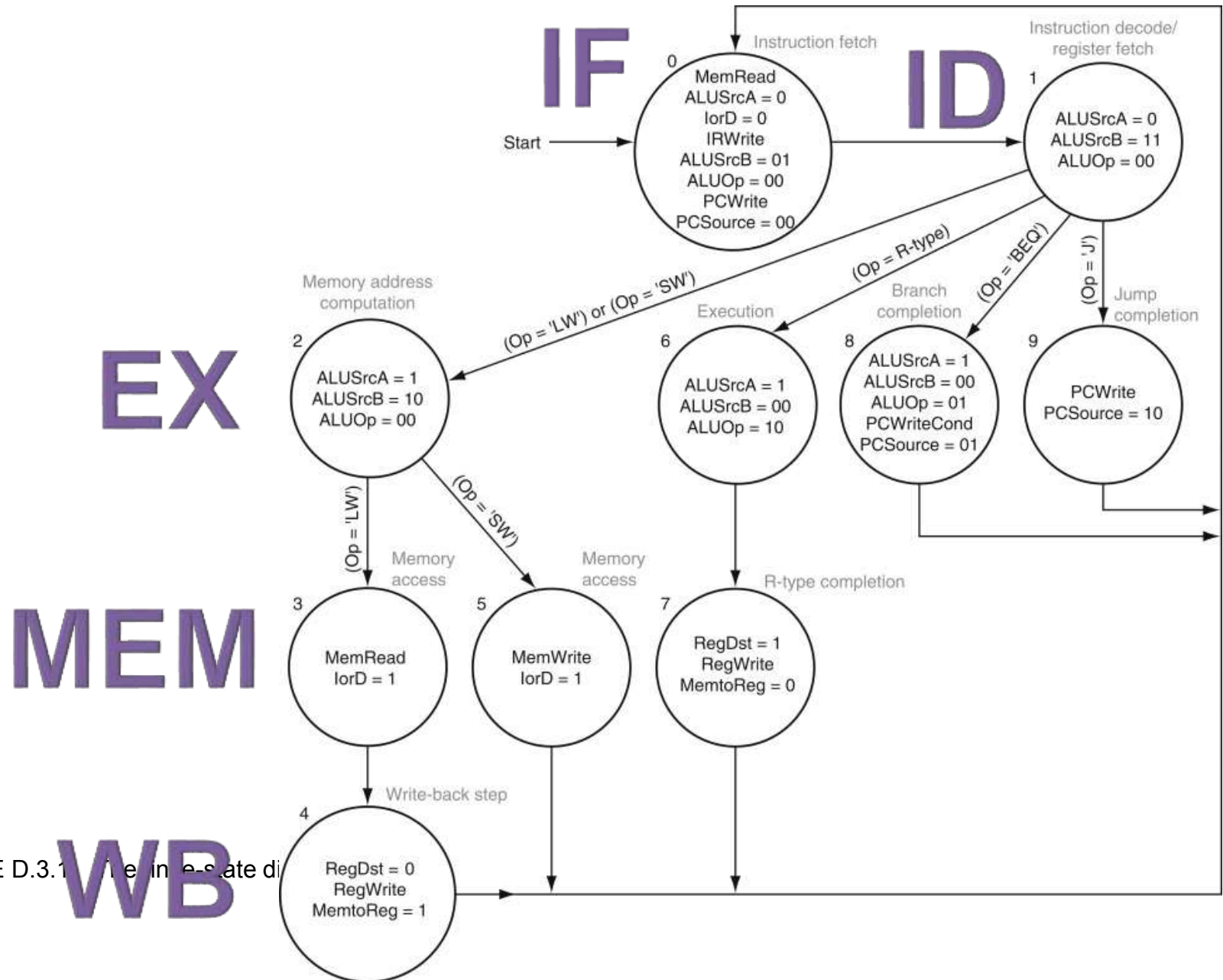


FIGURE D.3.1 The in-state di

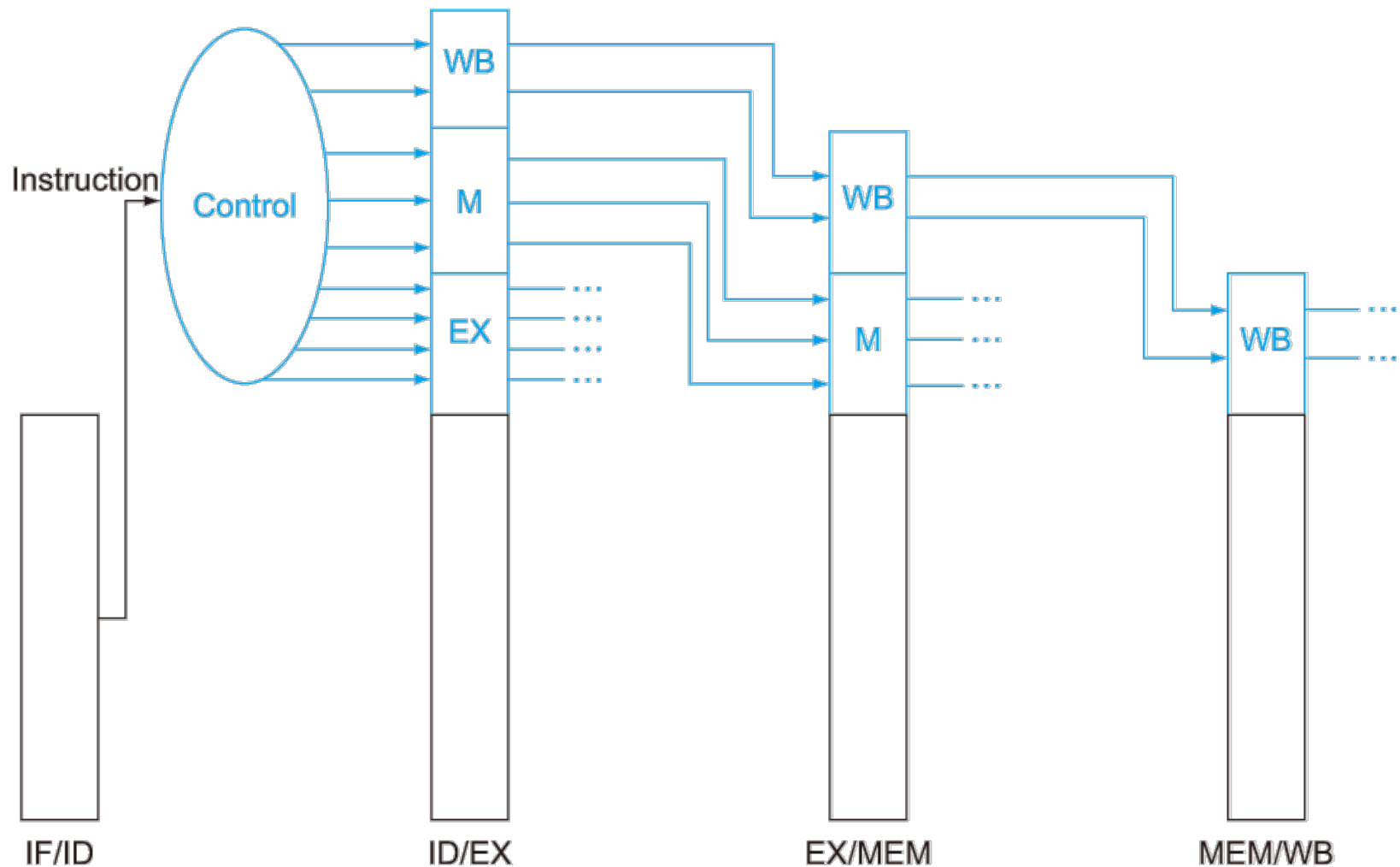


圖4.50 最後三級的控制訊號線

注意九條控制訊號線中有四條用於EX 級，其餘五條繼續送往EX/MEM 管線暫存器中擴充以保存它們的地方；三條用於MEM 級，最後的兩條則再送往MEM/WB 管線暫存器以使用於WB 級

管線化中的控制

■ 圖4.51表示完整數據通道

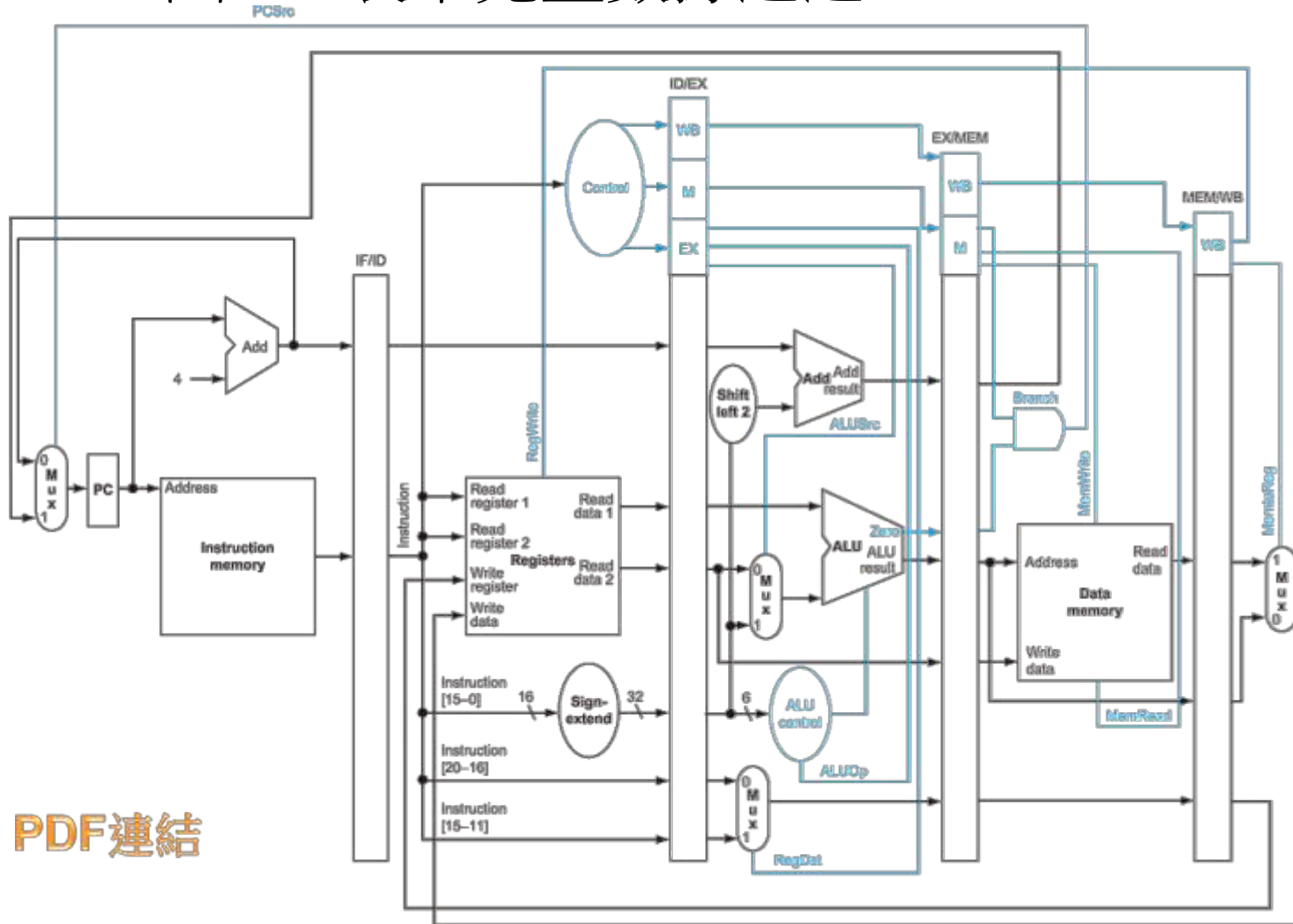


圖4.51 將控制訊號接到管線暫存器中儲存控制訊的部分後的圖4.46 中的管線化數據通道

最後三級的控制訊號在指令解碼的時候產生，然後放到ID/EX 管線暫存器中。每個管線階段的控制訊號被使用之後，剩下來的控制訊號就被送往下一個管線階段



以下數據通道控制信號，那些屬於

① Start presenting to display the poll results on this slide.



以下數據通道控制信號，那些屬於

① Start presenting to display the poll results on this slide.



以下數據通道控制信號，那些屬於

① Start presenting to display the poll results on this slide.

4.7 數據危障：前饋或停滯

▣ 指令序列：

sub \$2, \$1, \$3 # 暫存器\$2被sub指令寫入

and \$12, \$2, \$5 # 第一個運算元(\$2)相依於sub指令

or \$13, \$6, \$2 # 第二個運算元(\$2)相依於sub指令

add \$14, \$2, \$2 # 暫存器\$14被add指令寫入

sw \$15, 100(\$2) # 記憶體基底位置(\$2)相依於sub指令

- 假設暫存器\$2 的值在減法指令運算前為10 以及運算後為-20
- 圖4.52 以多個時脈週期管線表示法來說明這些指令的執行上方顯示了\$2 暫存器的值

圖4.52 以簡化的數據通道來表示五道指令的序列在管線中的相依關係

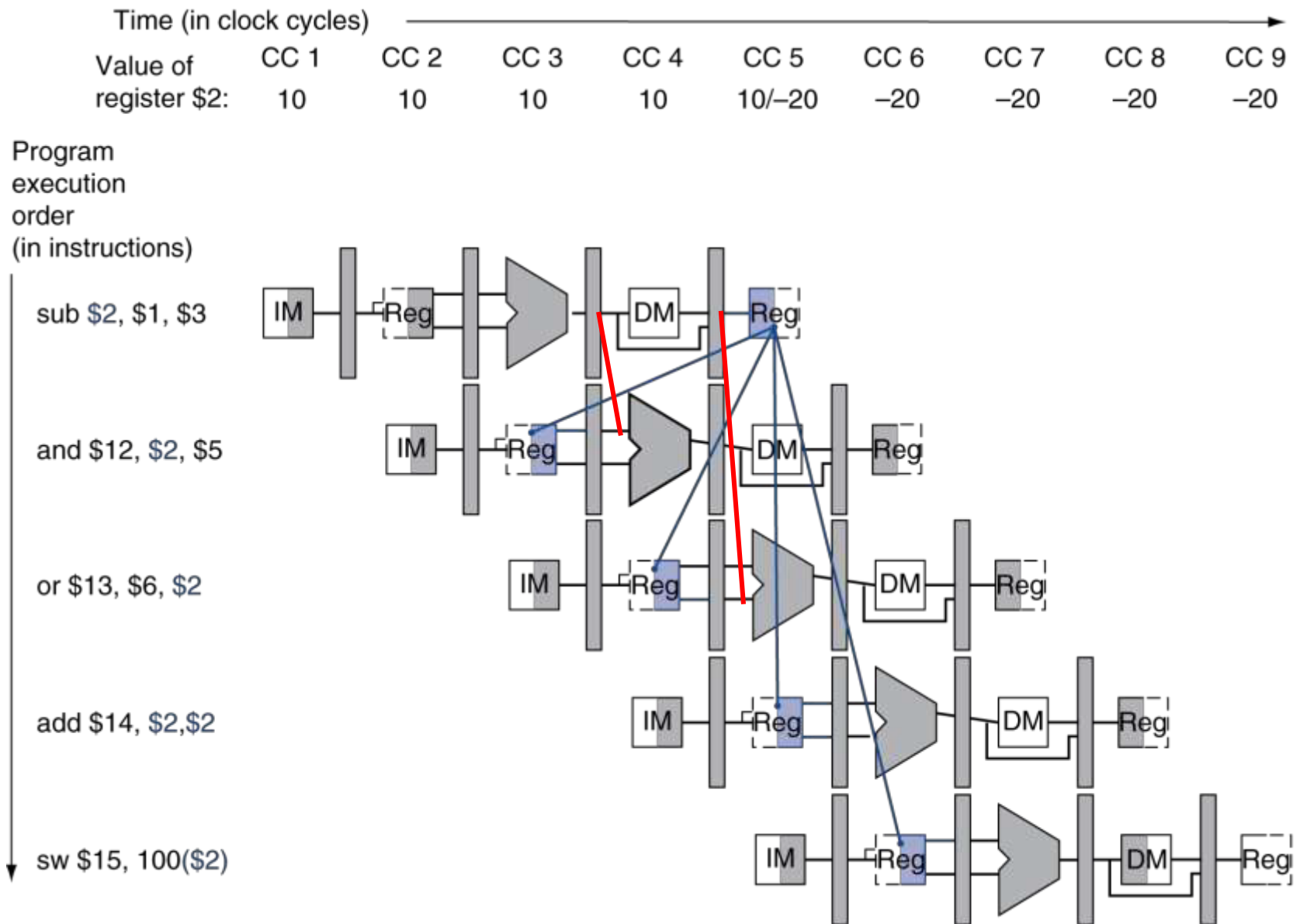


圖4.52 以簡化的數據通道來表示五道指令的序列在管線中的相依關係

所有相依的動作都以灰色來表示，而圖中最上面的**CC 1** 代表第一個時脈週期。第一個指令寫入**\$2**，而之後所有的指令都需要讀取**\$2**。這個暫存器在第五個時脈週期才被寫入，所以適當的值在第五個時脈週期前還不存在(在時脈週期中讀取暫存器時若同時有對該暫存器的寫入發生，則會得到該週期前半所寫入的值)。最上方的數據通道到下方各數據通道的拉線表示出相依關係。那些必須要逆著時間走的拉線就是管線數據危障

4.7 數據危障：前饋或停滯

- 最後的可能危障可以藉由暫存器檔案的硬體設計解決：當一個暫存器在一個週期內同時要讀取和寫入時會如何？
 - 假設寫入在時脈週期的前半完成，讀取在時脈週期的後半完成
- 如第4.5 節中所提及，所需的結果在EX 階段或第三個時脈週期結束時就出來了
 - and 與or 指令實際上是何時才需要這資料呢？
 - 在EX 階段開始的時候，或是分別在第四、五個時脈週期

4.7 數據危障：前饋或停滯

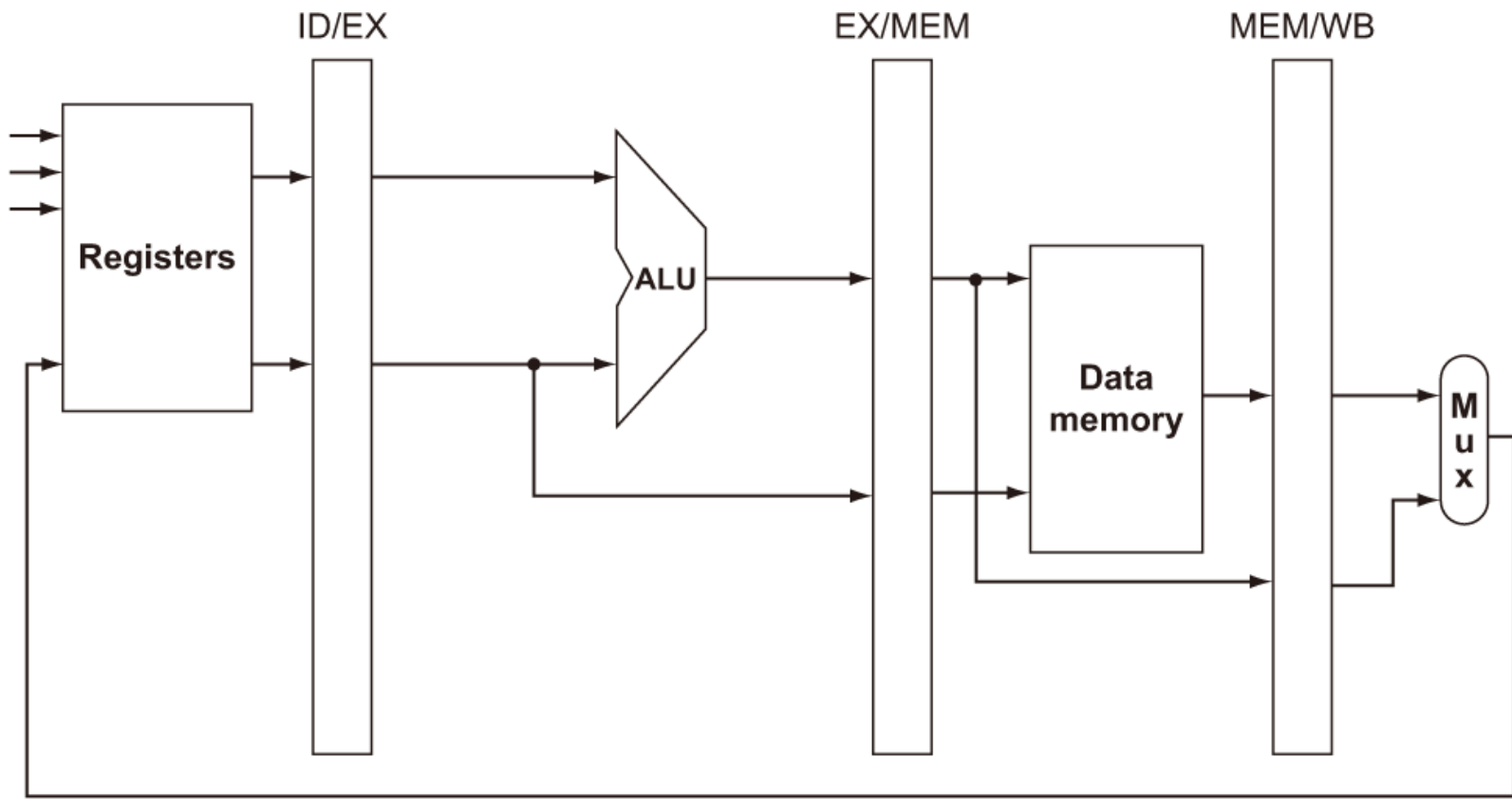
- 立即將資料前饋到需要的地方，我們就可以不需停滯而執行這些指令
- ▣ 前饋是如何辦到的？
 - 將管線暫存器的欄位命名。例如，
“ID/EX.RegisterRs, ID/EX.RegisterRt”
 - 偵側兩對的危障情形可表為：
 - 1a. EX/MEM.RegisterRd = ID/EX.RegisterRs
 - 1b. EX/MEM.RegisterRd = ID/EX.RegisterRt
 - 2a. MEM/WB.RegisterRd = ID/EX.RegisterRs
 - 2b. MEM/WB.RegisterRd = ID/EX.RegisterRt

Fwd from
EX/MEM
pipeline reg

Fwd from
MEM/WB
pipeline reg

4.7 數據危障：前饋或停滯

- 另因為有些指令並不會寫入暫存器，所以上述的檢查法並不完全正確；
 - ① 檢查RegWrite 的值是否為真
 - ② 如果在管線中有指令以\$0 作為目的暫存器，我們要避免前饋可能非0 的值
- 前饋正確的資料：藉著加入ALU 輸入的多工器以及適當的控制
- 圖4.54 顯示了清晰的ALU 與管線暫存器在加入前饋之前與之後的情形
- 圖4.55 則表示位於EX級中用以選擇暫存器檔案或是前饋過來的值的ALU多工器其控制線的值

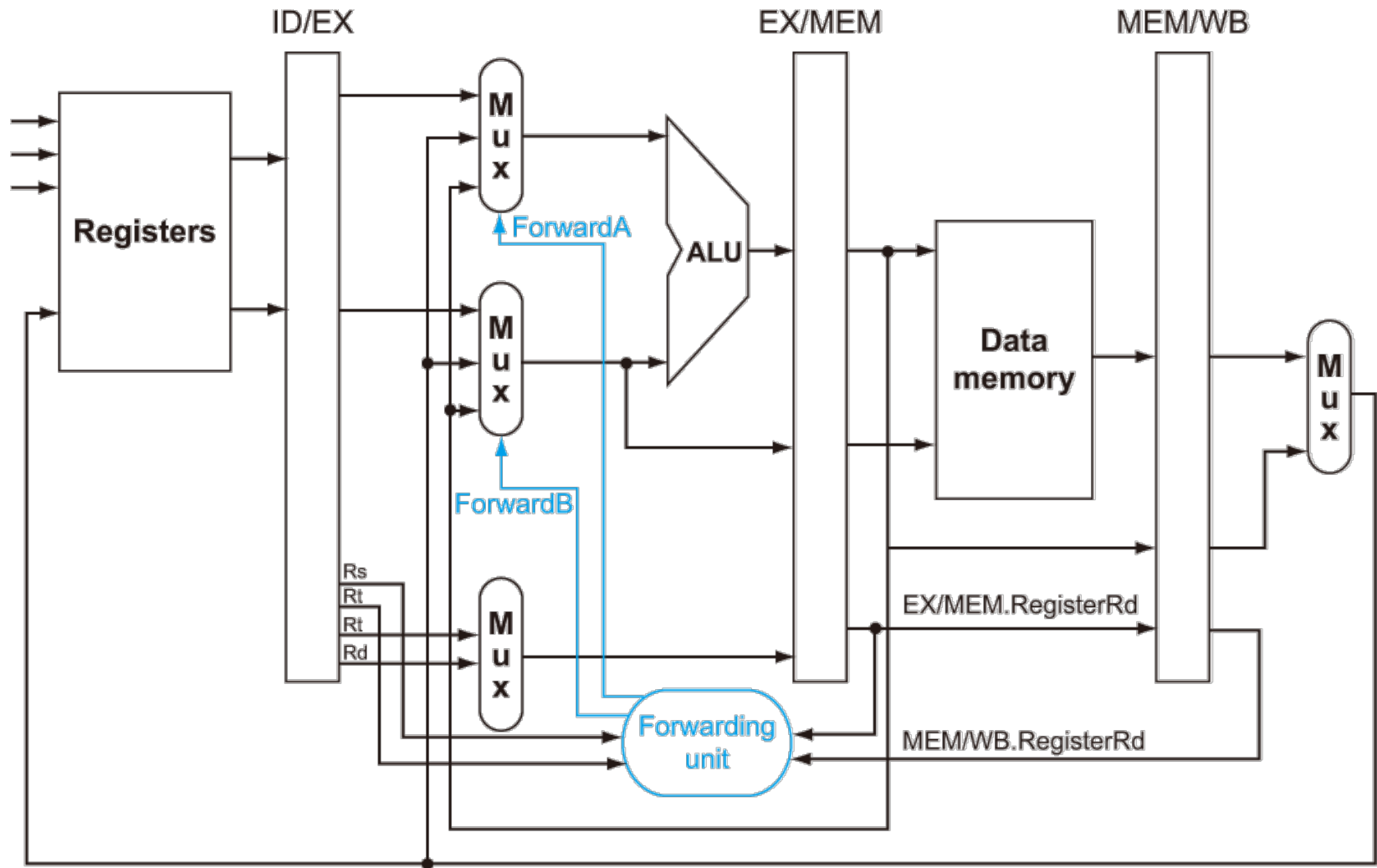


a. No forwarding

圖4.54 上圖是加入前饋功能前的ALU 和管線暫存器

下圖的多工器已經被擴充以加入前饋的路徑，而且也顯示了前饋單元。新加入的硬體以灰色呈現。本圖只是示意圖，省略了完整數據通道的細節如符號延伸的硬體。注意 **D/EX.RegisterRt** 的欄位出現了兩次。一次連接到多工器，另一次則是前饋單元，但這是相同的訊號。如前所述，這裡忽略了前饋儲存指令要存入的值。也注意這個機制對 **slt** 指令同樣可用

圖4.54 上圖是加入前饋功能前的ALU 和管線暫存器



b. With forwarding

多工器控制	來 源	解 釋
ForwardA = 00	ID/EX	ALU 的第一個運算元來自暫存器檔案
ForwardA = 10	EX/MEM	ALU 的第一個運算元前饋自前一個 ALU 運算結果
ForwardA = 01	MEM/WB	ALU 的第一個運算元前饋自數據記憶體或再早的 ALU 運算結果
ForwardB = 00	ID/EX	ALU 的第二個運算元來自暫存器檔案
ForwardB = 10	EX/MEM	ALU 的第二個運算元前饋自前一個 ALU 運算結果
ForwardB = 01	MEM/WB	ALU 的第二個運算元前饋自數據記憶體或再早的 ALU 運算結果

圖4.55 圖4.54 中多工器的控制值

有關ALU 的另外一個輸入有號立即值(**signed immediate**)在本節最後的仔細深思中說明

Forwarding Conditions

▣ EX hazard

- if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))

ForwardA = 10

- if (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))

ForwardB = 10

▣ MEM hazard

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRs))

ForwardA = 01

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and (MEM/WB.RegisterRd = ID/EX.RegisterRt))

ForwardB = 01



MIPS管線處理器發生EX階段RA

① Start presenting to display the poll results on this slide.



MIPS管線處理器發生MEM階段F

① Start presenting to display the poll results on this slide.

4.7 數據危障：前饋或停滯

- ▣ 如果WB 級中指令的結果以及MEM 級中指令的結果與ALU 級中指令的來源運算元同時存在數據危障
 - 此MEM 危障的控制應如下(並強調其中新增的部分)

Double Data Hazard

- ▣ Consider the sequence:

add \$1, \$1, \$2

add \$1, \$1, \$3

add \$1, \$1, \$4

- ▣ Both hazards occur

- Want to use the most recent

- ▣ Revise MEM hazard condition

- Only fwd if EX hazard condition isn't true

Revised Forwarding Condition

■ MEM hazard

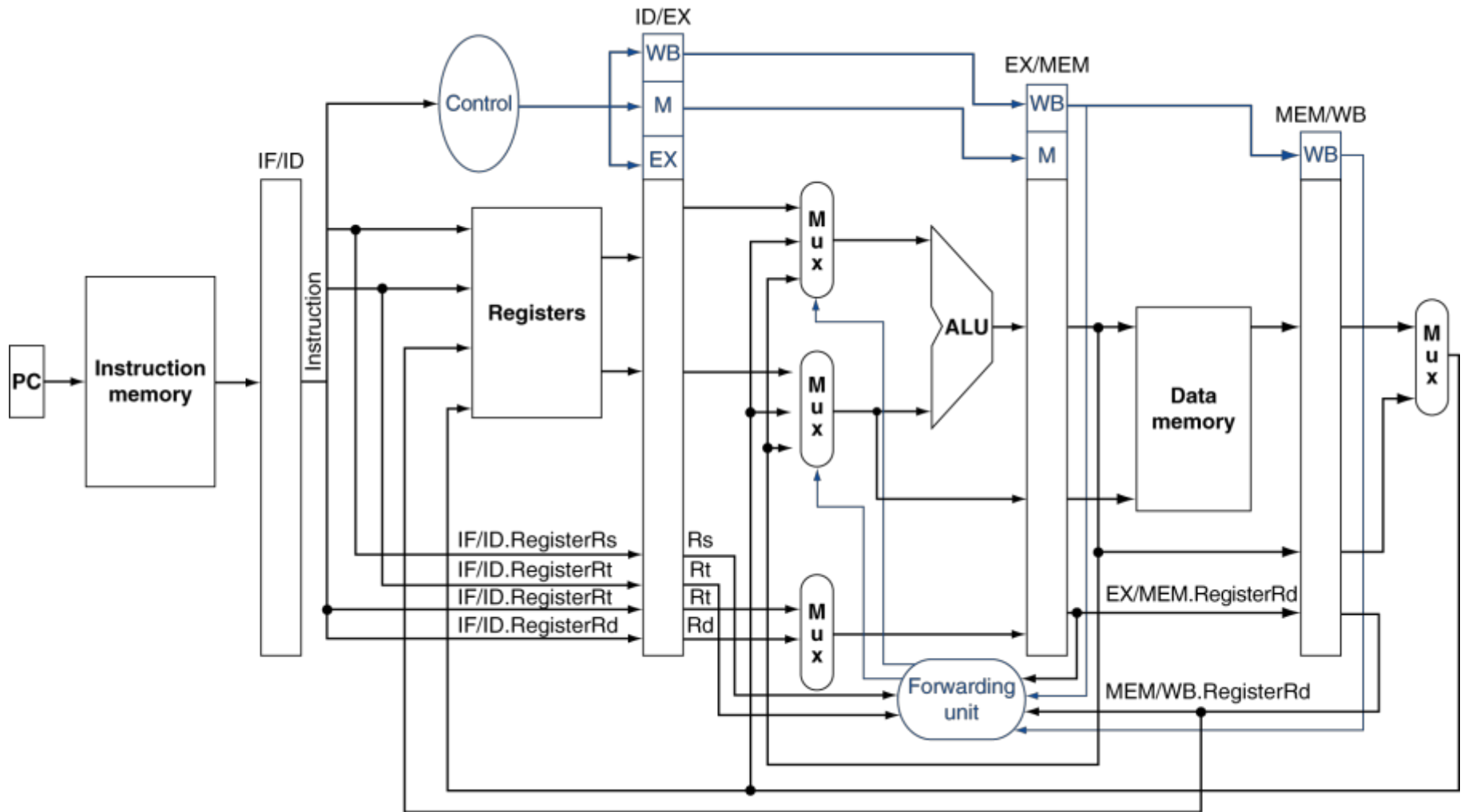
- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRs))
and (MEM/WB.RegisterRd = ID/EX.RegisterRs))

ForwardA = 01

- if (MEM/WB.RegWrite and (MEM/WB.RegisterRd $\neq$ 0)
and not (EX/MEM.RegWrite and (EX/MEM.RegisterRd $\neq$ 0)
and (EX/MEM.RegisterRd = ID/EX.RegisterRt))
and (MEM/WB.RegisterRd = ID/EX.RegisterRt))

ForwardB = 01

Datapath with Forwarding



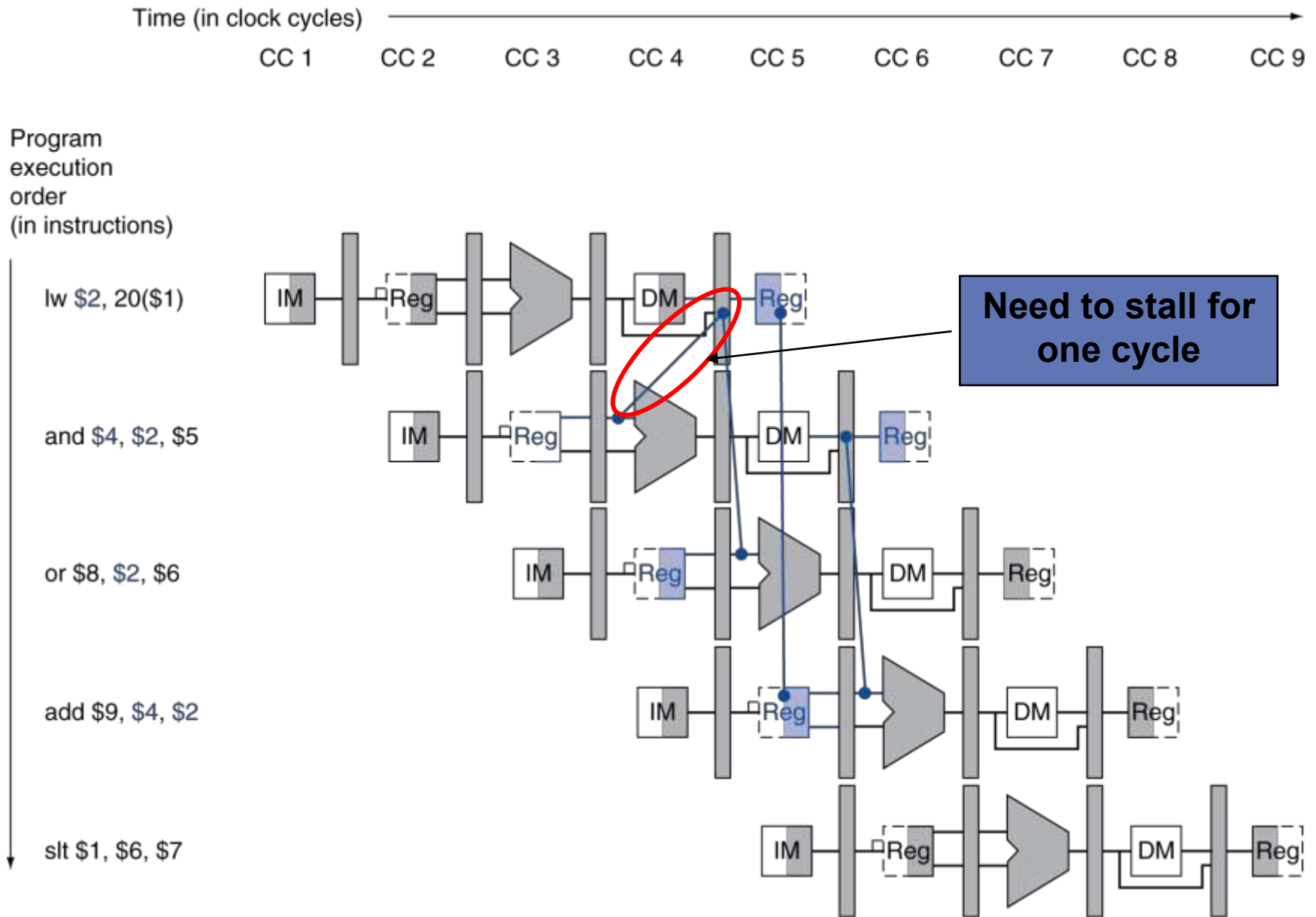
數據危障與停滯

- 當一個指令要讀取前一個載入指令寫入的暫存器時，前饋也無法完全解決
 - 圖4.58

圖4.58 管線化的指令序列

因為在載入指令與緊接在後的指令(**and**)之間的相依關係在時間上是逆著走的，這個危障不能靠前饋解決。因此，這種指令組合得由危障偵測單元來產生一個停滯

圖4.58 管線化的指令序列



數據危障與停滯

- 除了前饋單元之外，我們還需要一個危障偵測單元(hazard detection unit)。

```
if (ID/EX.MemRead and  
    ((ID/EX.RegisterRt = IF/ID.RegisterRs) or  
     (ID/EX.RegisterRt = IF/ID.RegisterRt)))  
    stall the pipeline(停滯管道)
```

- 第一行檢查指令是否為載入
- 後面兩行檢查EX 級中的載入指令其目的暫存器欄位是否與ID 級中指令的某一來源暫存器相符

How to Stall the Pipeline

- ▣ Force control values in ID/EX register to 0
 - EX, MEM and WB do **nop** (no-operation)
- ▣ Prevent update of PC and IF/ID register
 - Using instruction is decoded again
 - Following instruction is fetched again
 - 1-cycle stall allows MEM to read data for 1w
 - Can subsequently forward to EX stage

圖4.60 表示出前饋使用的兩個多工器、危障偵測單元以及前饋單元的管線化控制概覽

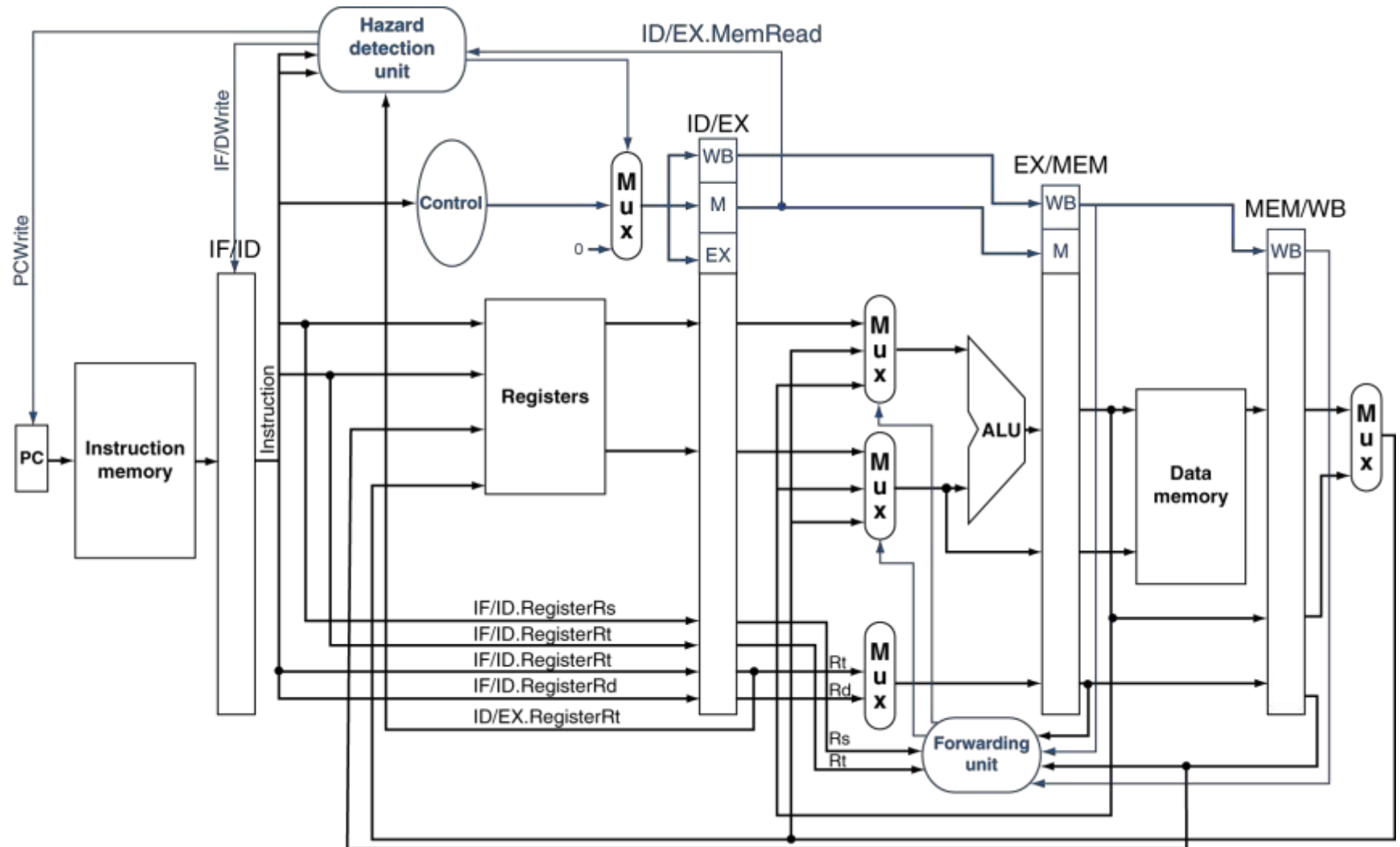


圖4.59 停滯實際上如何插入管線

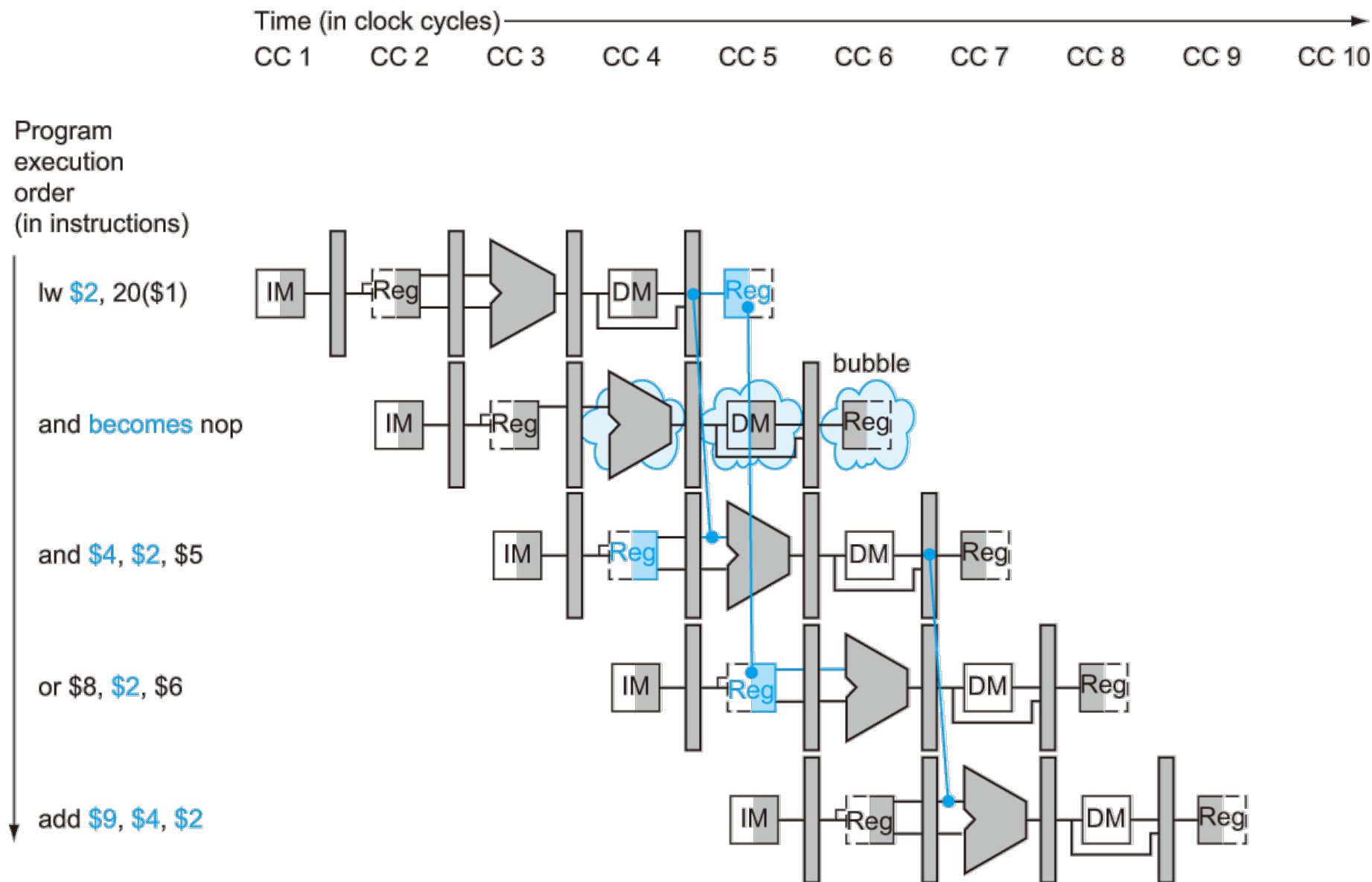


圖4.59 停滯實際上如何插入管線

藉由將**and** 指令改成**nop**，一個氣泡在時脈週期4 開始加了進來。注意**and** 指令在時脈週期2 與3 確實被擷取與解碼，但是其**EX** 級則是延遲到時脈週期5 (相對於未停滯時的時脈週期4)。同樣地**OR** 指令在時脈週期3 被擷取，但是其**ID** 級則延遲到時脈週期5 (相對於未停滯時的時脈週期4)。在插入氣泡之後，所有的相依關係都順著時間走，於是不會有危障發生

圖4.60 表示出前饋使用的兩個多工器、危障偵測單元以及前饋單元的管線化控制概覽

雖然ID 與EX 級被簡化了(省略了符號延伸的立即值部分與分支邏輯)，這個圖表示出前饋硬體的基本需求



危障偵測單元(Hazard Detection)

① Start presenting to display the poll results on this slide.



停滯處理器管線，需要作那些事

① Start presenting to display the poll results on this slide.

Chapter 4 Exercise Assignment 1

- 4.12
- 4.13

4.8 控制危障

- ▣ 圖4.61表示了一序列的指令並指出分支何時在管線中發生
 - 稱為控制危障(control hazard)或分支危障(branch hazard)
 - 介紹兩種解決控制危障的方法以及一種用來改善這兩者的最佳化方法

Branch Hazards

圖4.61 管線對分支指令的影響

- If branch outcome determined in MEM

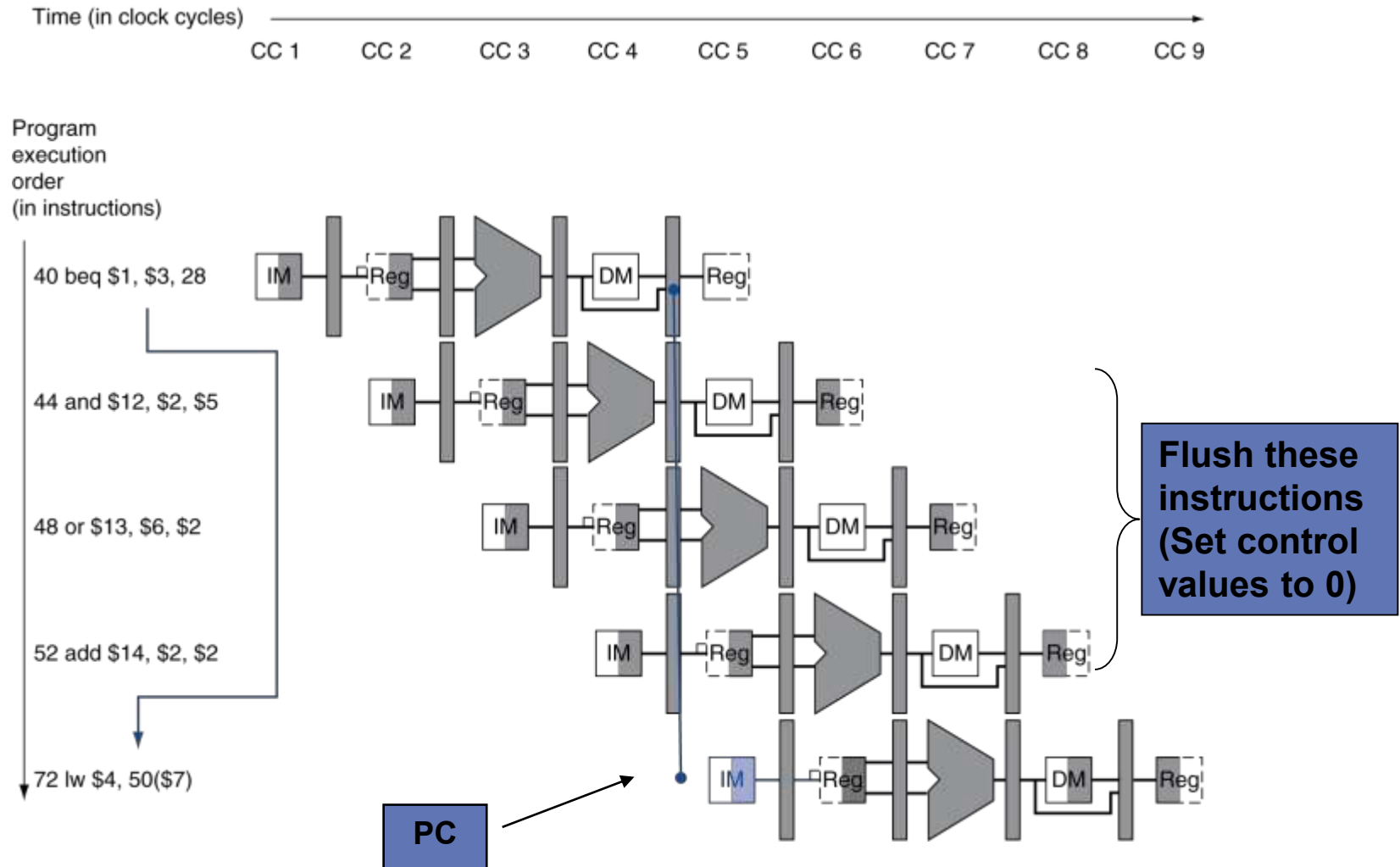


圖4.61 管線對分支指令的影響

指令左邊的數字(40, 44,)是指令的位址。因為分支指令在**MEM** 級(上述**beq** 指令於時脈週期4 中)決定是否要分支，該分支指令後連續三道指令會被擷取並開始執行。在沒有干預的情況下，這三道指令會在**beq**分支到位置72 的**lw** 指令之前開始執行。(圖4.31 假設使用了特別的硬體來將控制危害降低到一個時脈週期：此圖採用未最佳化的數據通道。)

假設分支不發生

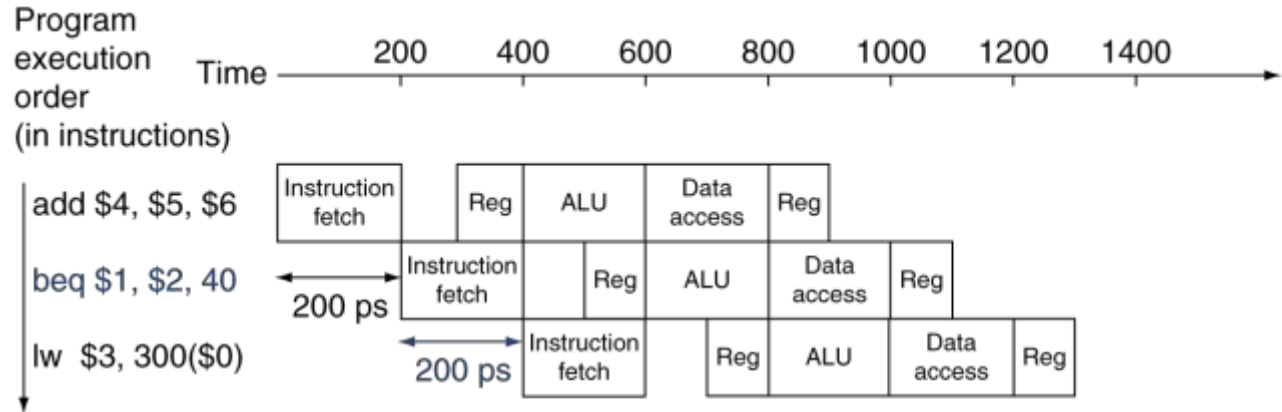
- ▣ 繼續執行接下去的指令

- 若是分支發生了，之後已被擷取與解碼的指令就都要丟棄

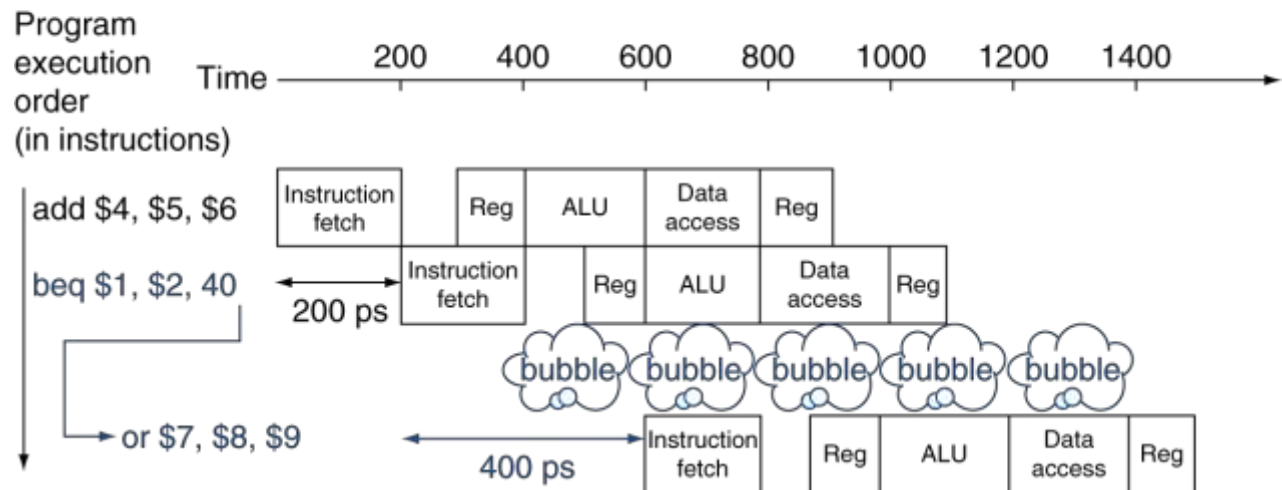
MIPS with Predict Not Taken

圖4.31

**Prediction
correct**



**Prediction
incorrect**



減少分支的延遲

- 要提早作分支決定則需提早做兩件事：計算分支的目標位址以及做分支判斷
 - 在IF/ID 管線暫存器中已經有PC 值與立即值欄位，所以只需將分支位址加法器從EX 級移到ID 級
 - 如果是相等則分支(**beq**)，我們可以比較在ID 級所讀取的兩個暫存器是否相等
 - 要用到額外的前饋與危障偵測硬體
 - 有兩個複雜的因素：

減少分支的延遲

- ① 將相等條件測試單元提前到ID 級中則需要新的前饋邏輯。要繞送的分支運算元可能從ALU/MEM 或是MEM/WB 管線暫存器中傳來
- ② 例如一個緊連在分支之前的ALU 指令會產生分支比較所需的其中一個運算元，這時候就需要一個週期的暫停。如果載入指令後方緊跟一個需要用到其值的分支，需要停滯兩個週期

■ 為了清除IF 級中的指令，可將IF/ID 管線暫存器中的指令欄位設為0

Reducing Branch Delay

■ Move hardware to determine outcome to ID stage

- Target address adder
- Register comparator

■ Example: branch taken

```
36:  sub    $10, $4, $8
40:  beq    $1,  $3, 7
44:  and    $12, $2, $5
48:  or     $13, $2, $6
52:  add    $14, $4, $2
56:  slt    $15, $6, $7

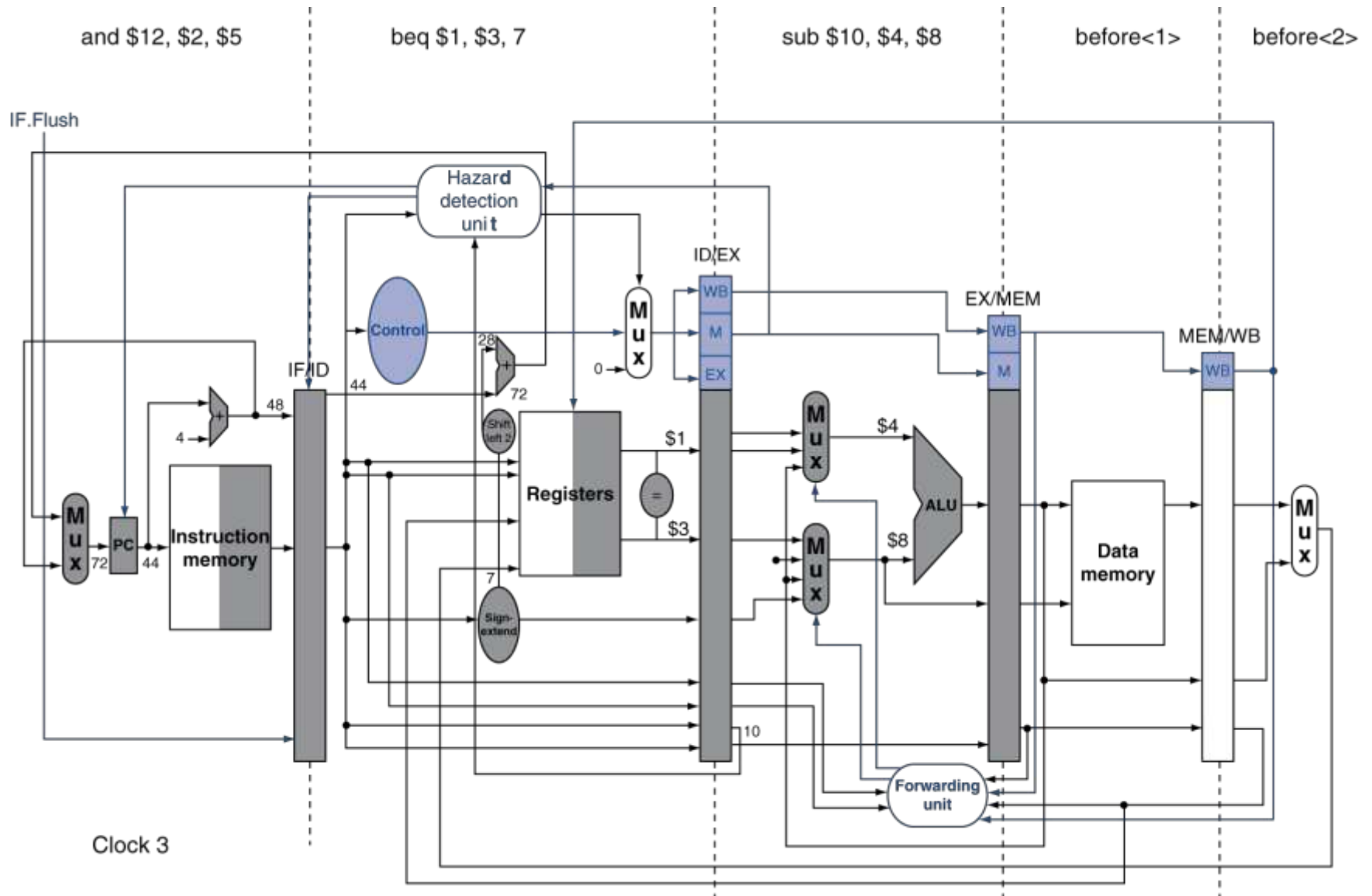
    ...
72:  lw     $4, 50($7)
```



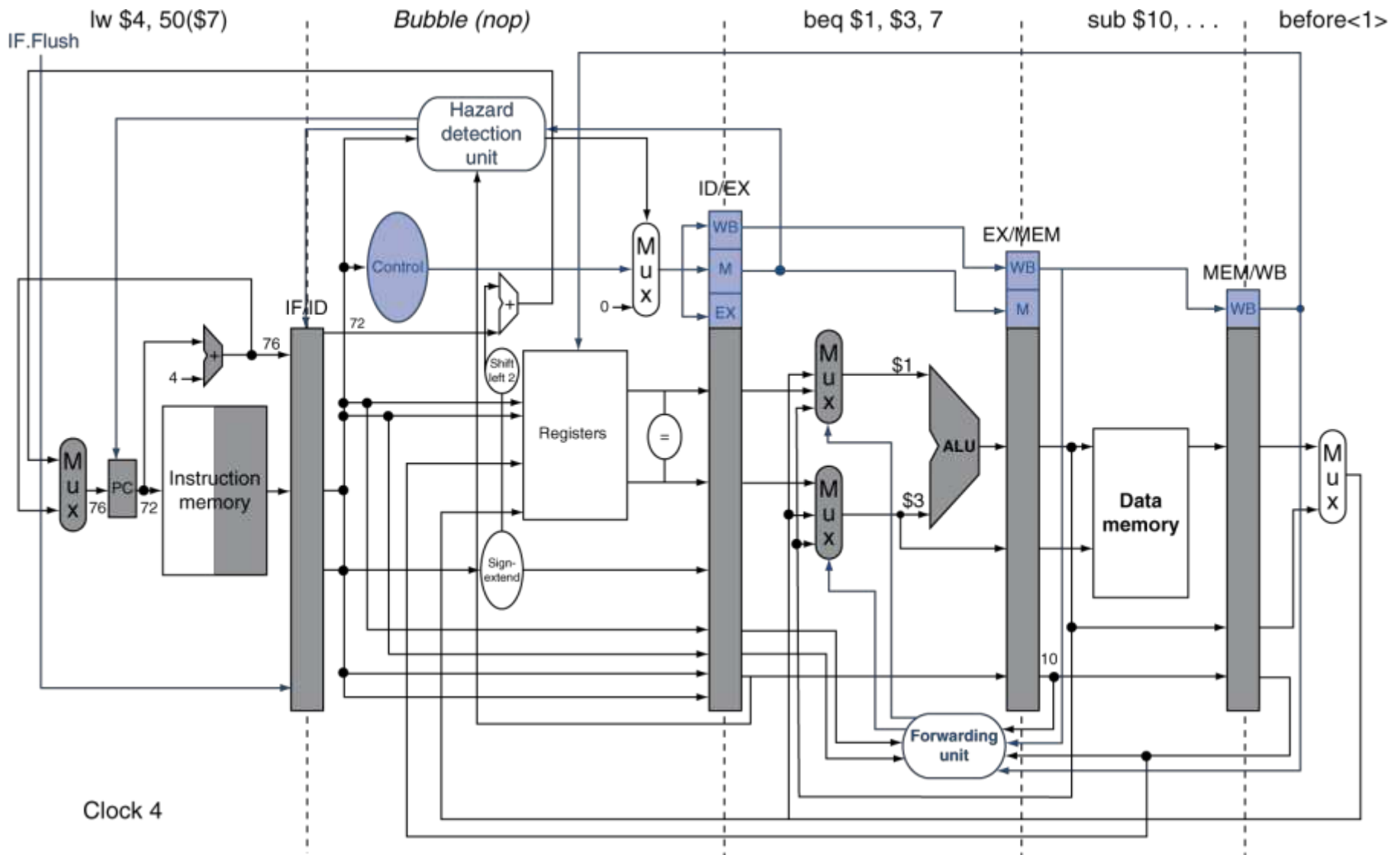
如何減少延遲分支槽(reducing c

① Start presenting to display the poll results on this slide.

Example: Branch Taken

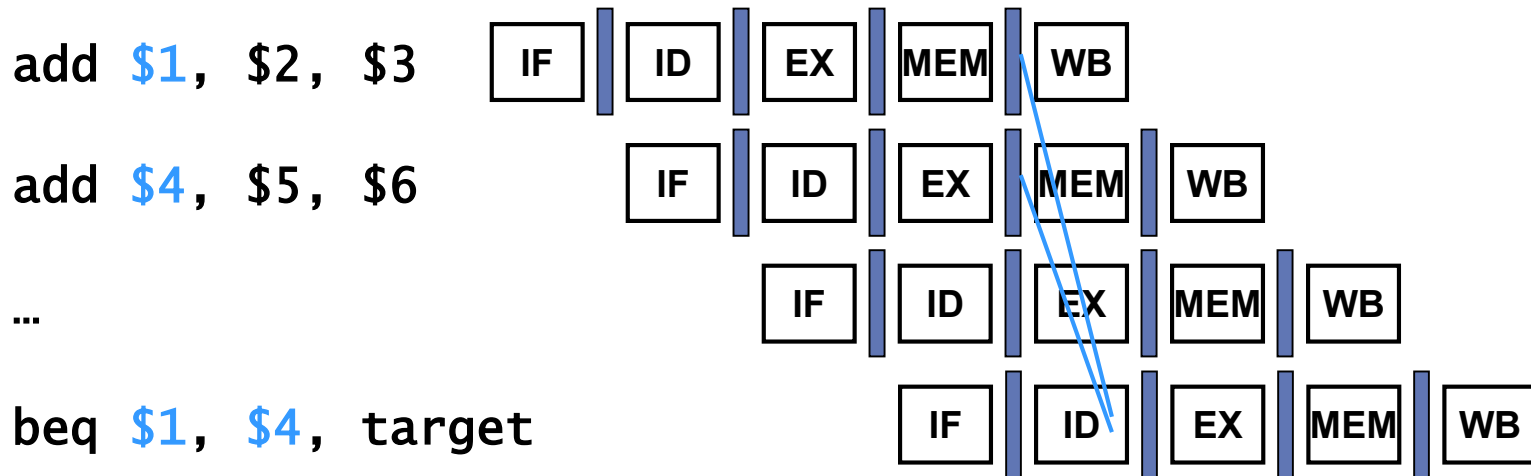


Example: Branch Taken



Data Hazards for Branches

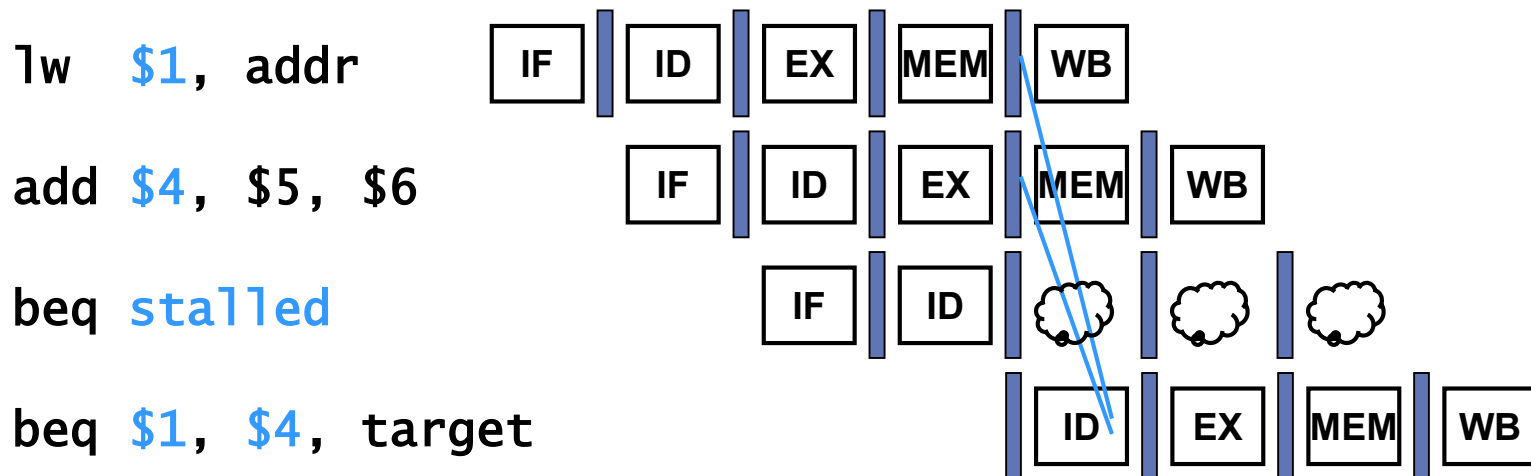
- If a comparison register is a destination of 2nd or 3rd preceding ALU instruction



- Can resolve using forwarding

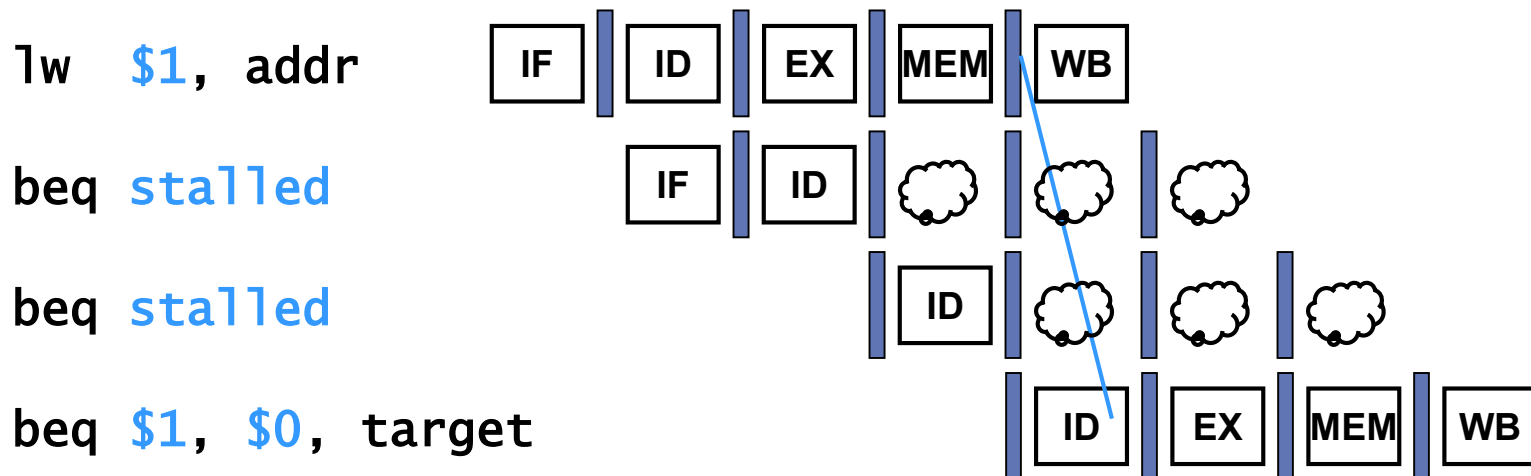
Data Hazards for Branches

- If a comparison register is a destination of preceding ALU instruction or 2nd preceding load instruction
 - Need 1 stall cycle



Data Hazards for Branches

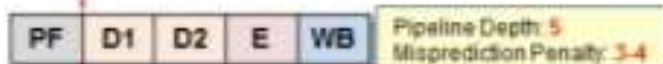
- If a comparison register is a destination of immediately preceding load instruction
 - Need 2 stall cycles



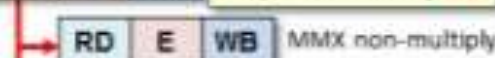
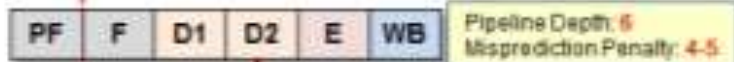
Intel x86 Pipeline History

<http://molesterwaterball.blogspot.com>
電腦水球部落格

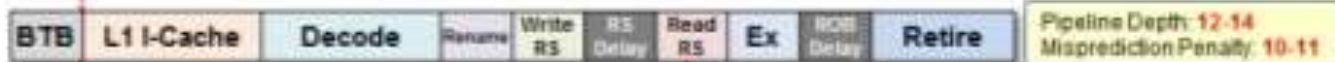
Pentium
(P5 / P54C)



Pentium MMX
(P55C)

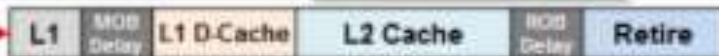


Intel P6 Family
Pentium M



180nm / 130nm
NetBurst

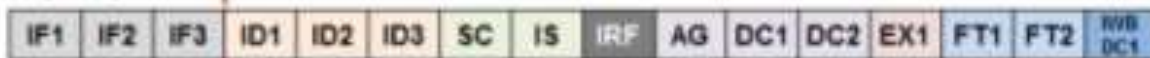
記憶體資料載入管線 →



Bonnelli (Atom)



90nm / 65nm
NetBurst



Core
(Yonah)



Core 2
(Merom / Penryn)



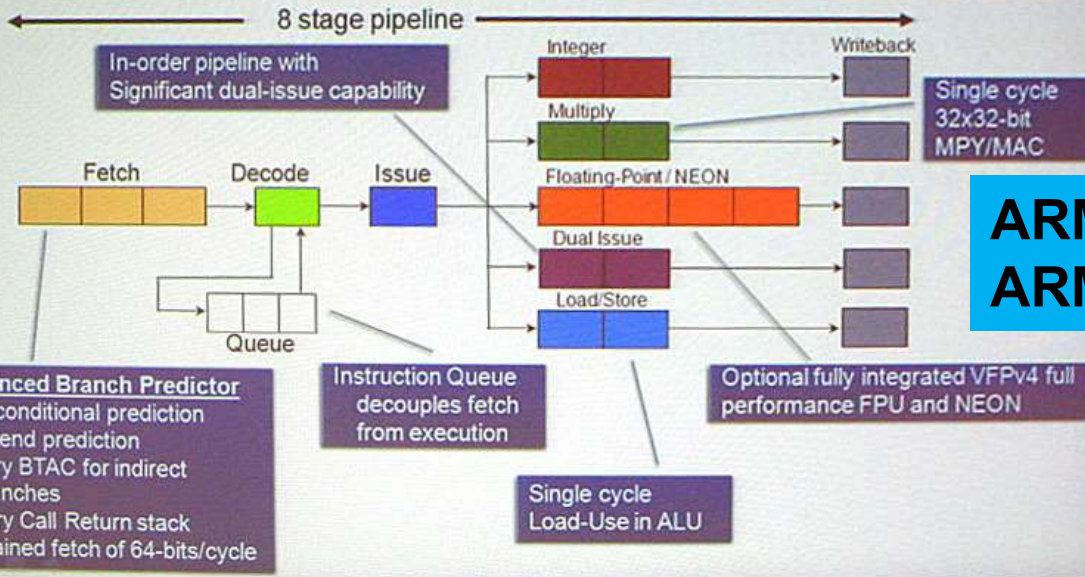
Nehalem
Westmere



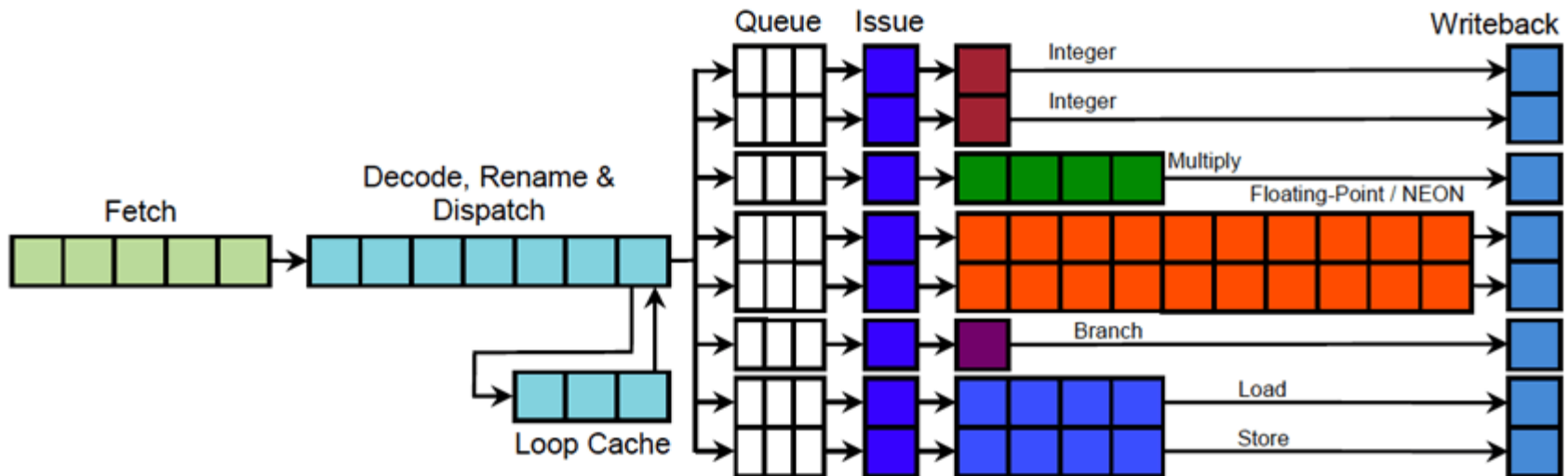
Cortex-A7 Pipeline

ARM®
2011 TechCon™
Join the community defining the future

- Design focused on power efficiency first
- Microarchitecture features improve performance over Cortex-A5 with little to no power impact



ARM Cortex A7: 8-10 stages
ARM Cortex A15: 15-28 stages



動態分支預測(Dynamic Branch Prediction)

- 在現代化處理器深度的管線(deep pipeline)中以及在多重派發(Multiple Issue)的架構下，分支懲罰所造成的指令損失數也會增加
- 動態分支預測(dynamic branch prediction)一種實現的形式是分支預測緩衝器(branch prediction buffer)或是分支歷程表(branch history table)
 - 一位元預測方法(1-bit prediction scheme)有效能上的缺點
 - 二位元預測方法(2-bit prediction scheme)常用以彌補這個弱點。在二位元的預測方法中，預測的結果必須連錯譬如兩次才會改變

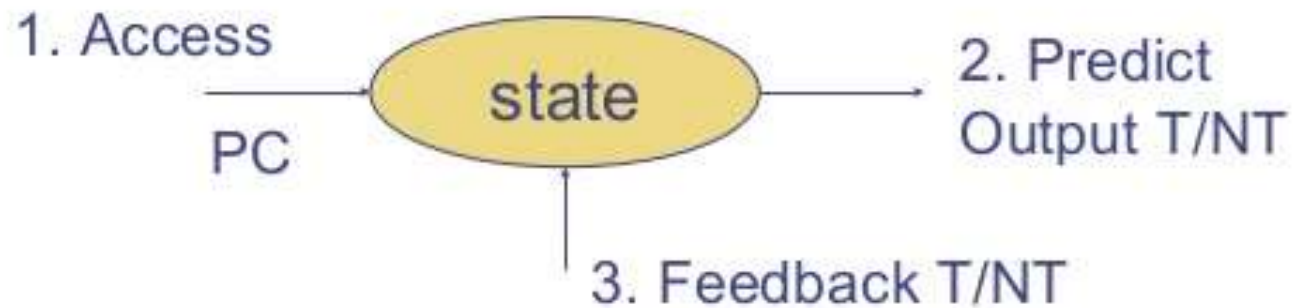


動態分支預測使用的原因，有那些？

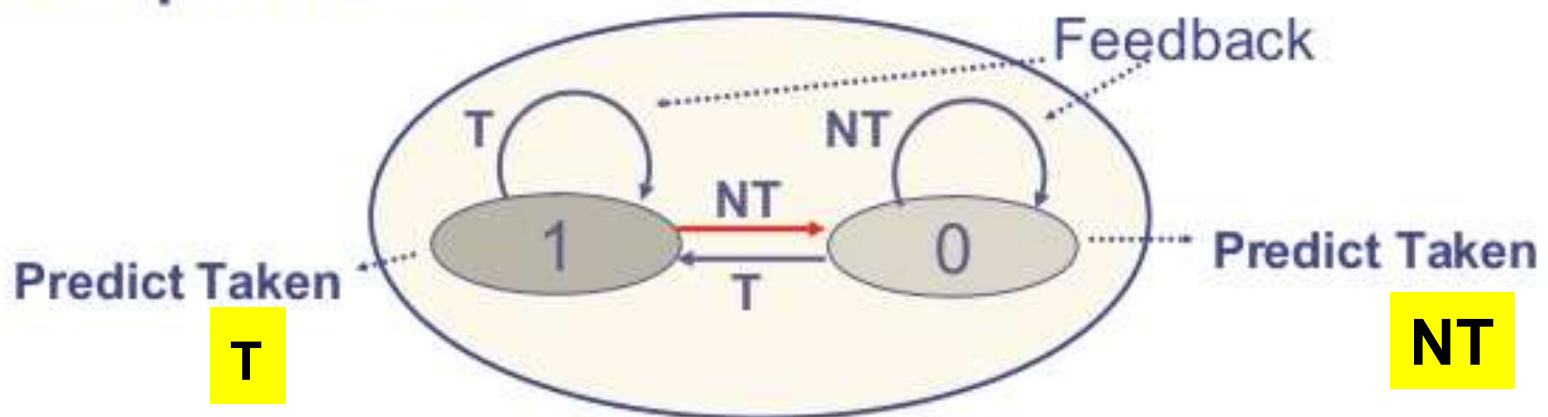
① Start presenting to display the poll results on this slide.

Predictor for a Single Branch

General Form



1-bit prediction



Predict: T T T T T T T T T T N T T ...
Outcome: Y Y Y Y Y Y Y Y Y N N Y ...

1-bit BHT Weakness

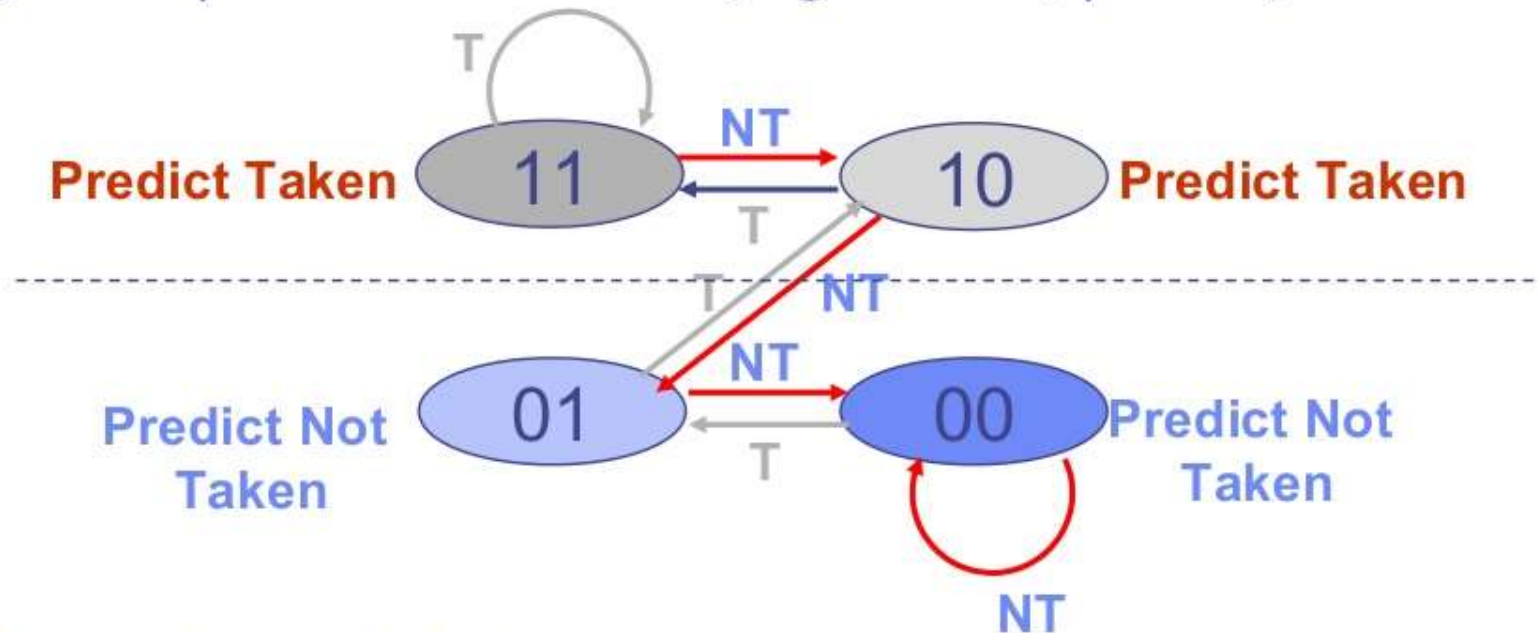
- ◆ Example: in a loop, 1-bit BHT will cause 2 mispredictions
- ◆ Consider a loop of 9 iterations before exit:

```
for (...) {  
    for (i=0; i<9; i++)  
        a[i] = a[i] * 2.0;  
}
```

- End of loop case, when it exits instead of looping as before
- First time through loop on *next* time through code, when it predicts *exit* instead of looping
- Only 80% accuracy even if loop 90% of the time

2-bit Saturating Counter

- ◆ Solution: 2-bit scheme where change prediction only if get misprediction *twice*: (Figure 3.7, p. 249)



- ◆ Blue: stop, not taken
- ◆ Gray: go, taken
- ◆ Adds *hysteresis* to decision making process

動態條件跳躍預測

- 圖4.63 表示二位元預測方法的有限狀態機 (finite-state machine)

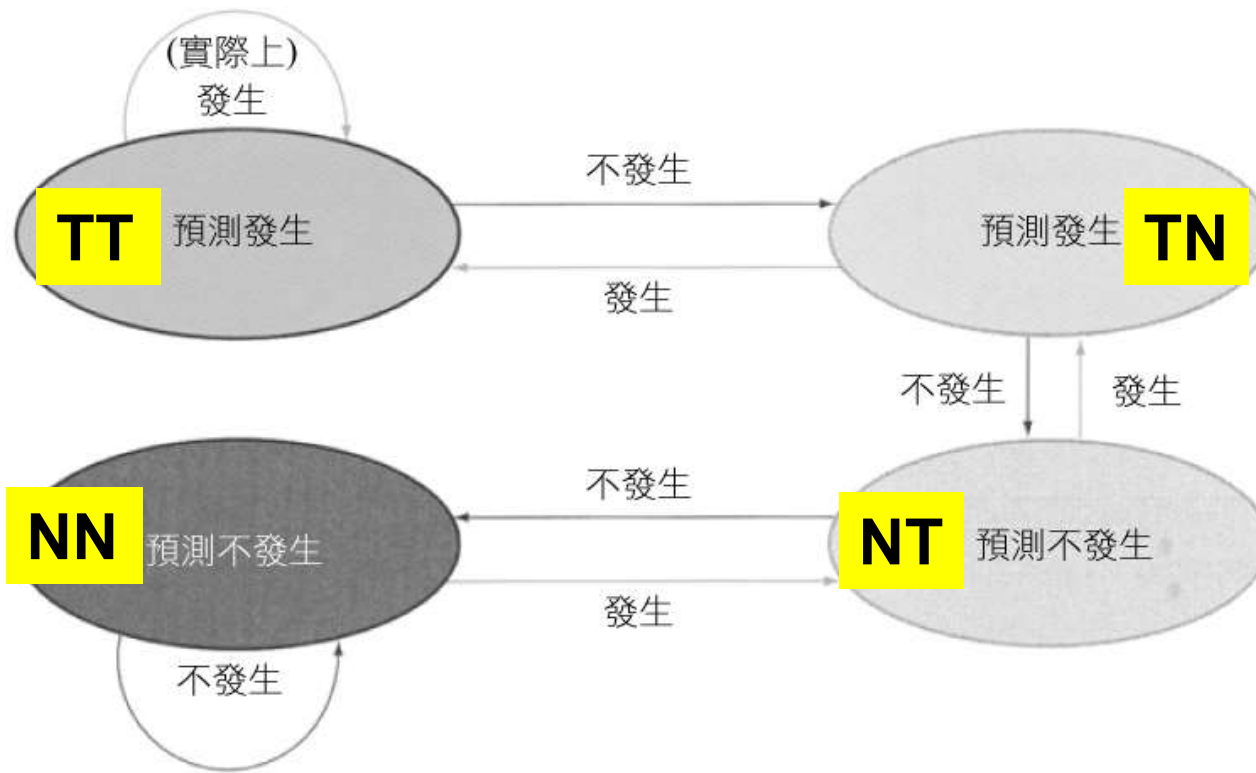


圖4.63 二位元預測方法的狀態

使用二個位元而非一位元，強烈偏向發生或是不發生的分支很多分支都有這種傾向只會猜錯一次。用兩個位元來編碼四個狀態。這種二位元預測方法是計數器型預測器的一個常見例子，其在預測準確時加一否則減一，並以中間點作為預測分支發生或不發生的分界

1-bit BHT Weakness

- ◆ Example: in a loop, 1-bit BHT will cause 2 mispredictions
- ◆ Consider a loop of 9 iterations before exit:

```
for (...) {  
    for (i=0; i<9; i++)  
        a[i] = a[i] * 2.0;  
}
```

Predict: T_TT_TT_TT_TT_TT_TT_TT_TT_T

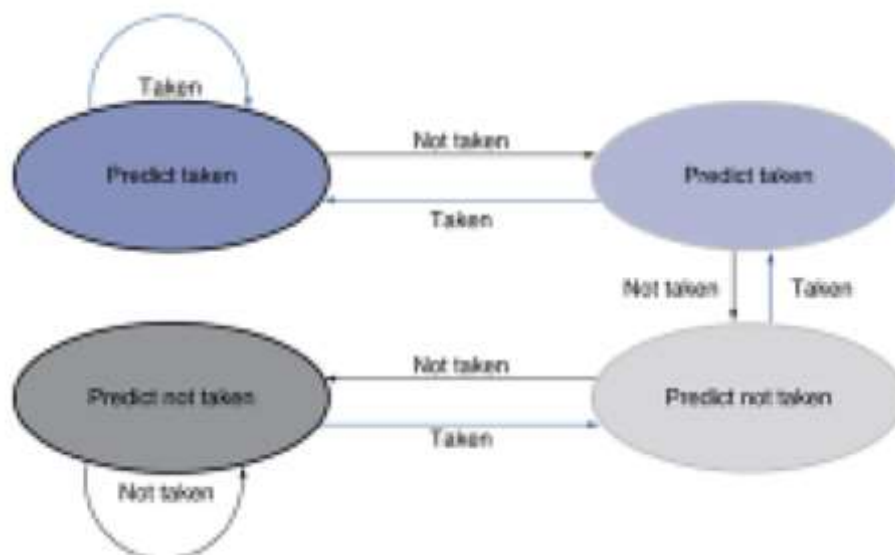
Outcome₁: Y_YY_YY_YY_YY_YY_YN_N

Outcome₂: Y_YY_YY_YY_YY_YY_YN_Y

- End of loop case, when it exits instead of looping as before
- First time through loop on *next* time through code, when it predicts *exit* instead of looping
- Only 80% accuracy even if loop 90% of the time

Branch prediction example

Branch Outcomes:	T	F	T	F	T	T	T	T	F	T	T	T	T
1-bit predictor (Initial state: T)	Y	N	N	N	N	Y	Y	Y	N	N	Y	Y	Y
2-bit predictor (Initial state: TT)	Y	N	Y	N	Y	Y	Y	Y	N	Y	Y	Y	Y





關於一位元與二位元分支預測器

① Start presenting to display the poll results on this slide.

Chapter 4 Exercise Assignment 2

- 4.15
- 4.16

- 如果同時使用該分支以及最近執行過的數道分支這種更廣泛、全局的資訊，在使用相同的預測位元數下，可以有更好的預測準確度
 - 關聯式預測器(correlating predictor)
 - 競賽型預測器(tournament predictors)
 - 使用多個預測器，並對每個分支記錄哪個預測器產生最好的效果
 - 其他分支(方向、目標)預測器技術....
 - 詳情歡迎選修計算機結構課程(第三章)

管線總結

■ 圖4.65 表示最後得到的數據通道與控制

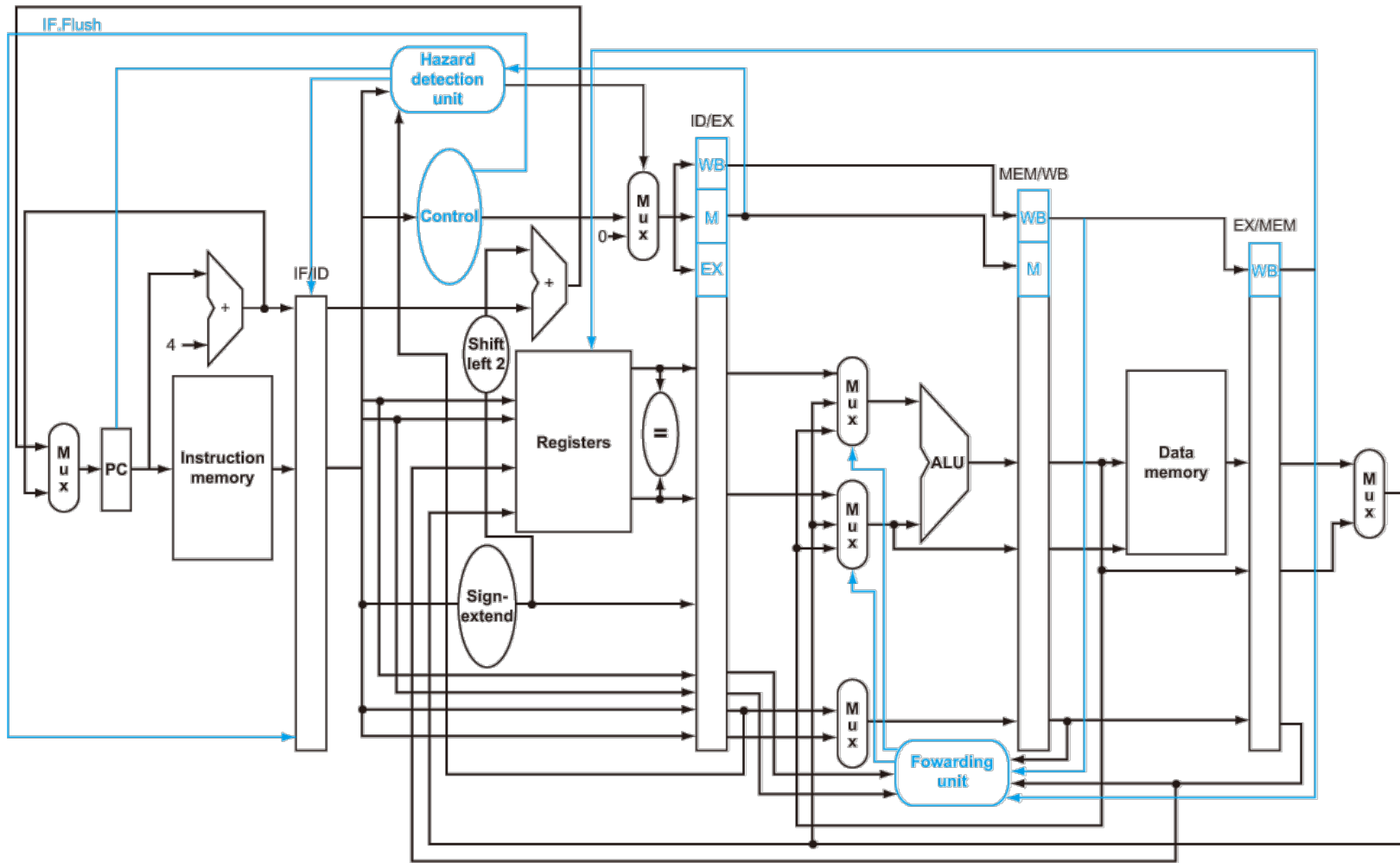


圖4.65 本章的最終數據通道與控制

注意這是一個示意圖而非詳細的數據通道，所以沒有畫出圖4.57中的ALUsrc多工器以及圖4.51中的多工器控制

4.10 平行性與進階的指令階層平行性

- 提高可能的指令階層平行性(instruction-level parallelism, ILP)的主要方法有二
 - 第一種是增加管線深度以重疊更多道指令
 - 另一方法是複製計算機內部的元件以便其能夠在每一個管線階級中執行多道指令多重派發(multiple issue)
 - CPI 小於1。IPC，或每時脈週期指令數(instructions per clock cycle)，大於1
 - 目前的高階微處理器每個時脈週期中可以試圖發出三到六(8!)道指令

4.10 平行性與進階的指令階層平行性

- 基本上對哪些類型的指令可以在同一瞬間執行以及有相依性時應該如何處理仍有許多限制
- 靜態多重派發(static multiple issue)以及動態多重派發(dynamic multiple issue)
- 有兩個必須要面對的主要而且明顯不同的工作：
 1. 將指令包裝進派發槽(issue slots)中
 2. 處理資料以及控制危障

猜測(Speculation投機性執行)的概念

- 尋找以及利用更多ILP 的最重要方法之一就是猜測。例如，
 - 對一個分支預測其結果，以便其後的指令得以提早執行
 - 猜測一個在載入指令之前的儲存指令並不會存取到同一個位址，因此載入得以在儲存前執行
- 針對不正確猜測的回復機制差異相當大
 - 在軟體來做猜測的情況下
 - 在硬體做猜測時
- 猜測引發另一個可能的問題：預測性地執行一些指令可能會造成不應發生的例外

動態多重派發處理器

- 動態多重派發處理器亦稱超純量(superscalar)處理器
- 最簡單的超純量處理器中，指令依序派發，處理器決定在某一時脈週期中可以派發的指令數
 - 仍需編譯器協助排序指令來拉開相依性以便提高指令派發率
- 許多超純量將動態派發判斷所用的基本架構擴充來兼作動態管線排程(dynamic pipeline scheduling)
- 詳情歡迎選修計算機結構課程(第三章)

4.11 實例：ARM Cortex-A8 與Intel Core i7 的管線

處理器	ARM A8	Intel Core i7 920
市場	個人移動裝置	伺服器、雲端
溫控設計功耗	2 Watts	130 Watts
時脈速率	1 GHz	2.66 GHz
核數/晶片	1	4
浮點？	無	有
多重派發？	動態	動態
峰值指令點/時脈週期	2	4
管道階數	14	14
管道排程	靜態依序	動態亂序且有投機
分支預測	2 層	2 層
第一層快取/核	32 KiB I, 32 KiB D	32 KiB I, 32 KiB D
第二層快取/核	128 - 1024 KiB	256 KiB
第三層快取(共用)	-	2 - 8 MiB

圖4.74
ARM
Cortex-A8
與Intel
Core i7 920
的規格

ARM Cortex-A8

▣ 圖4.75 顯示整個管線的構

- 最前的三階一次擷取兩道指令來試圖保持一個12 道指令的預取緩衝儲存器是滿的
- 使用一個包含有512 個項目的分支目標緩衝器、4096 個項目的全域歷程緩衝器，以及8 個項目的返回堆疊
- 13 個時脈週期的預測錯誤懲罰
- 解碼的五階
- 指令執行區段的六階中提供載入儲存指令一條管線，以及算術運算兩條管線、其中只有第一條能執行乘法。三條管線間具有完整的資料前饋能力

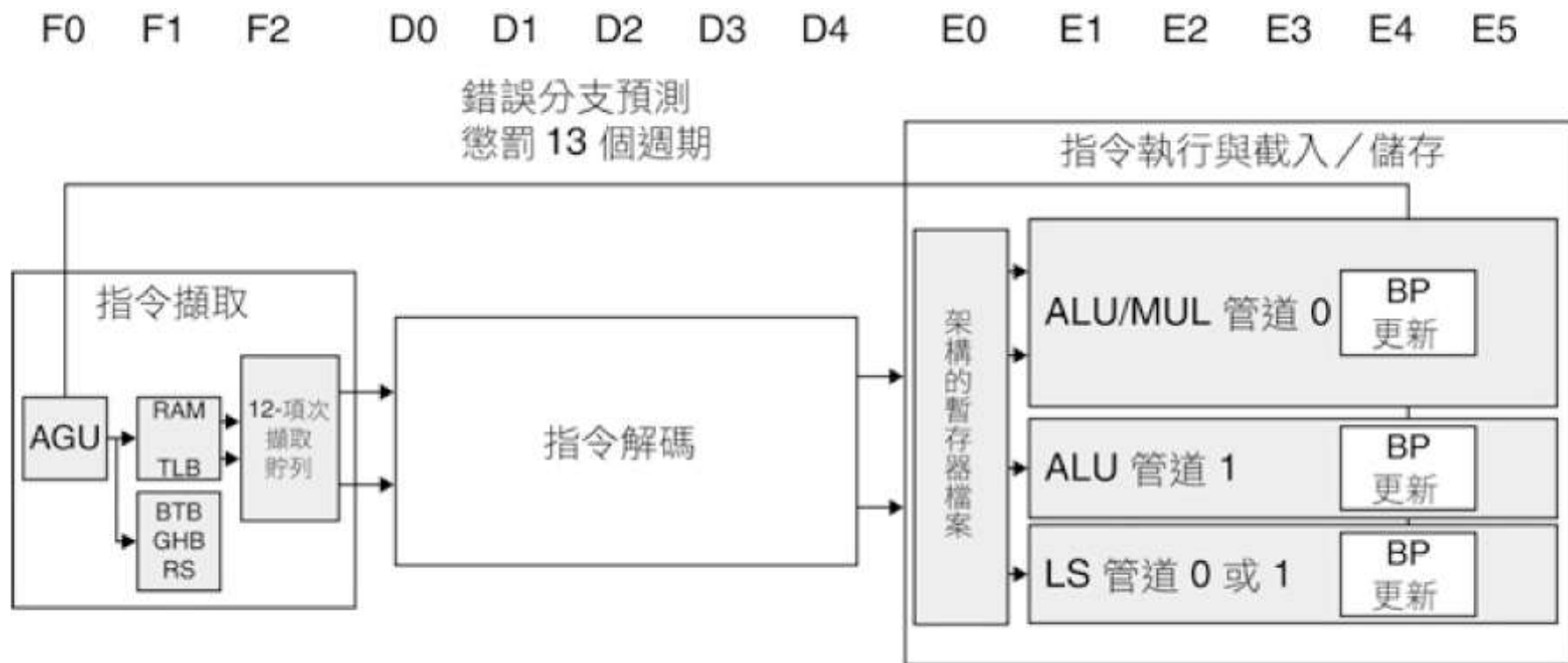


圖4.75 A8 的管線

最前的三階將指令擷取入12 個項次的指令擷取緩衝器中。一個位址產生單元(Address Generation Unit, AGU)使用分支目標緩衝器(Branch Target Buffer, BTB)、全域歷史緩衝器(Global History Buffer, GHB)以及返回堆疊(Return (Address) Stack, RS)來預測分支以求保持擷取貯列中放滿指令。指令解碼使用五階以及指令執行使用六階

Intel Core i7 920

- ▣ 14 階的管線中使用動態多重派發以及動態管線排程加上亂序執行和猜測(分支結果)
 - Intel 擷取x86 指令並將之轉換成其稱之為微運作 (micro-operations)的(一至多道)像MIPS 指令的內部指令
 - 然後這些微指令會在每時脈週期可執行六道微指令的複雜的、可動態排程、具猜測能力的管線上執行

Intel Core i7 920

- 採用微架構(microarchitecture)這名詞來表示處理器詳細的內部架構
 - 具有暫存器重命名(register renaming)功能的重序緩衝器(reorder buffer)
 - 架構上的暫存器(architectural registers)
 - 詳情歡迎選修計算機結構課程(第三章)

- 圖4.77 表示Core i7 管線的整體組織

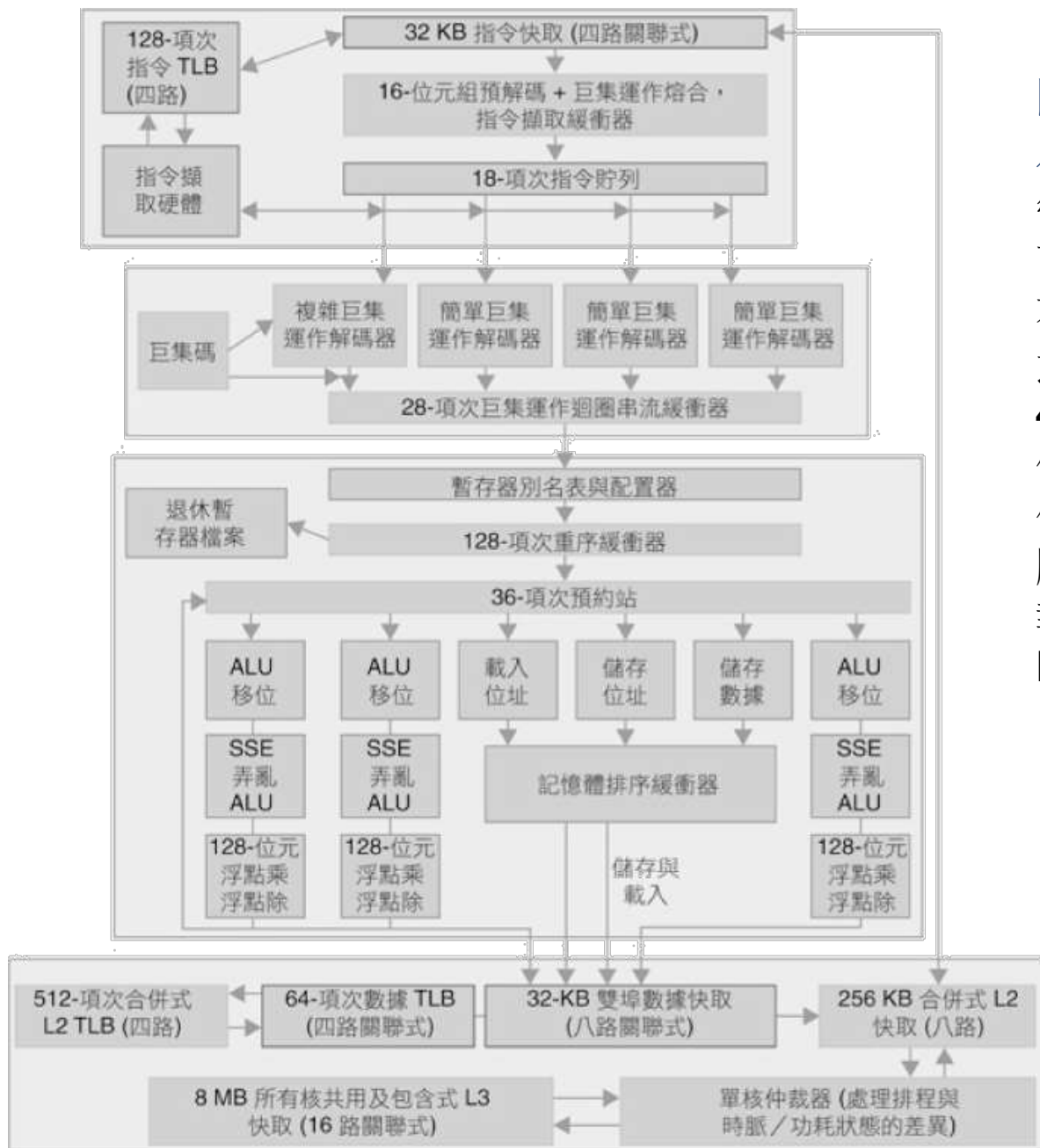
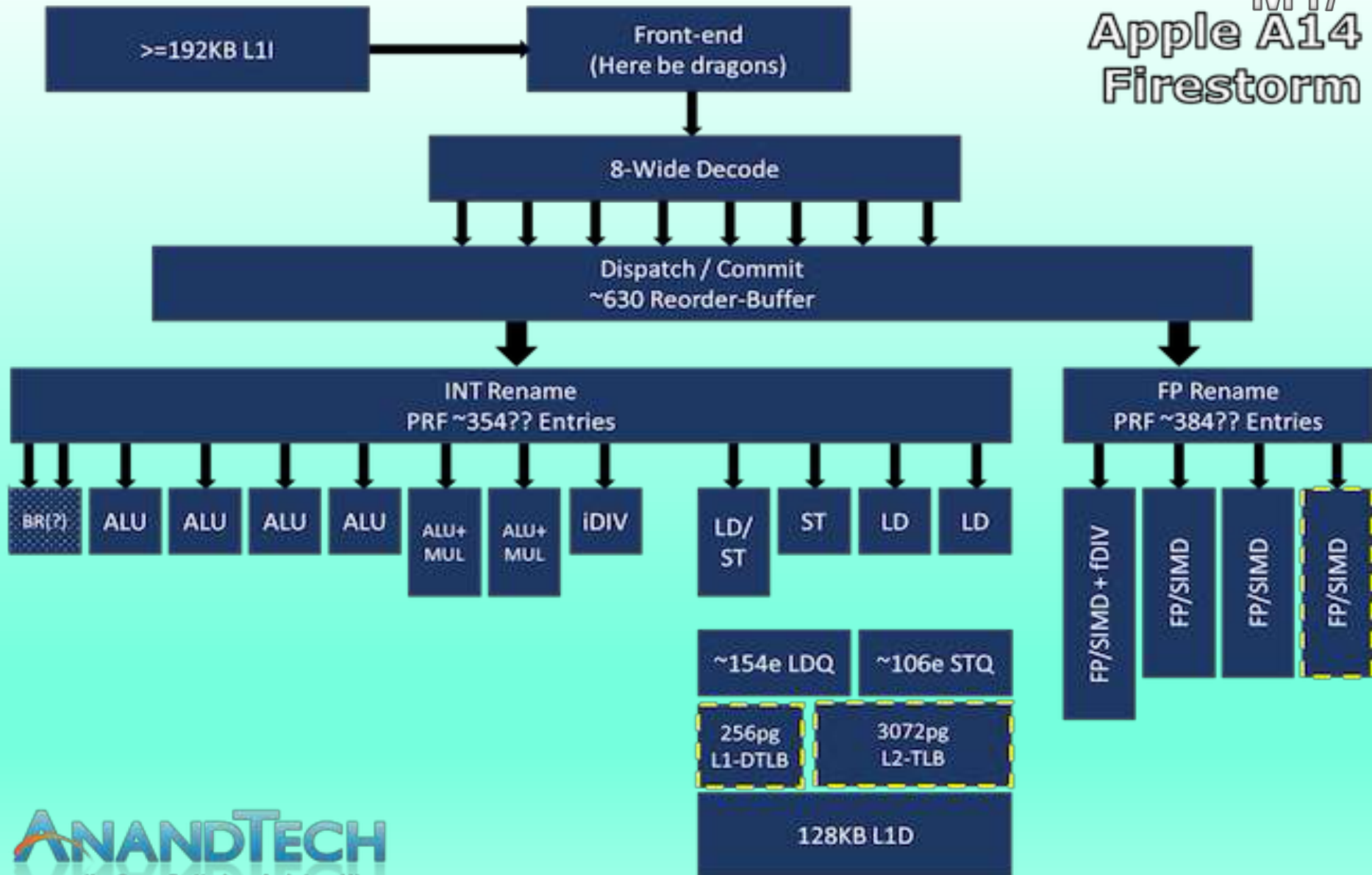


圖4.77 Core i7 管線以及各個記憶體組件。

管線的總深度有**14** 階，而錯誤的分支預測需花費**17** 個時脈週期。這個設計可以緩衝住**48** 道載入指令與**32** 道儲存指令。裡頭的六個獨立的單元每個時脈週期中都可以開始執行一道準備好了的 RISC 運作。

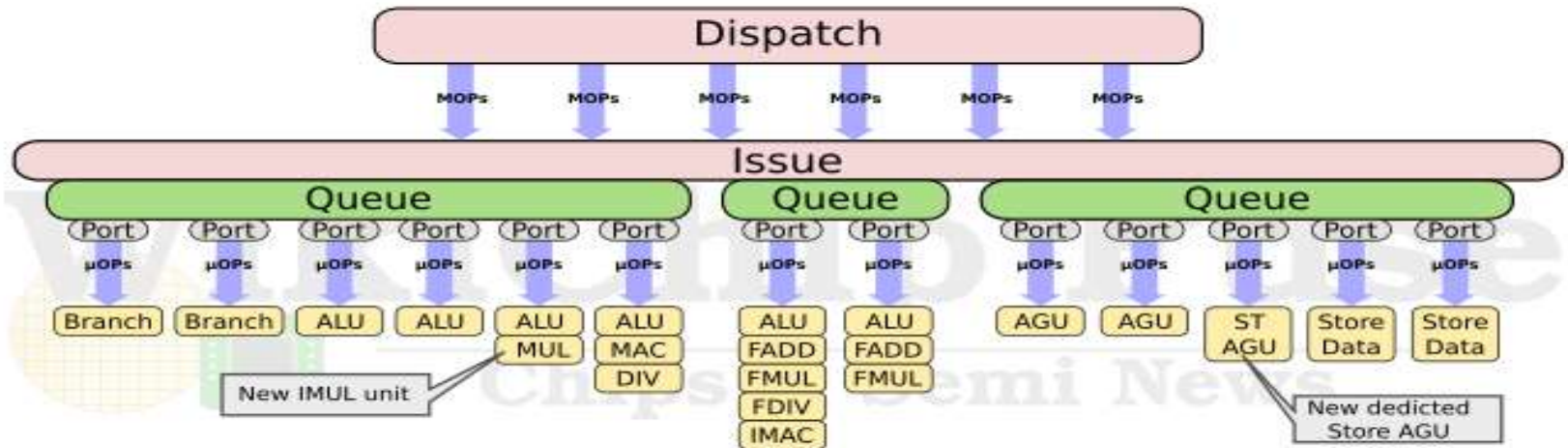
計算機組織 Computer Organization

M1/
Apple A14
Firestorm

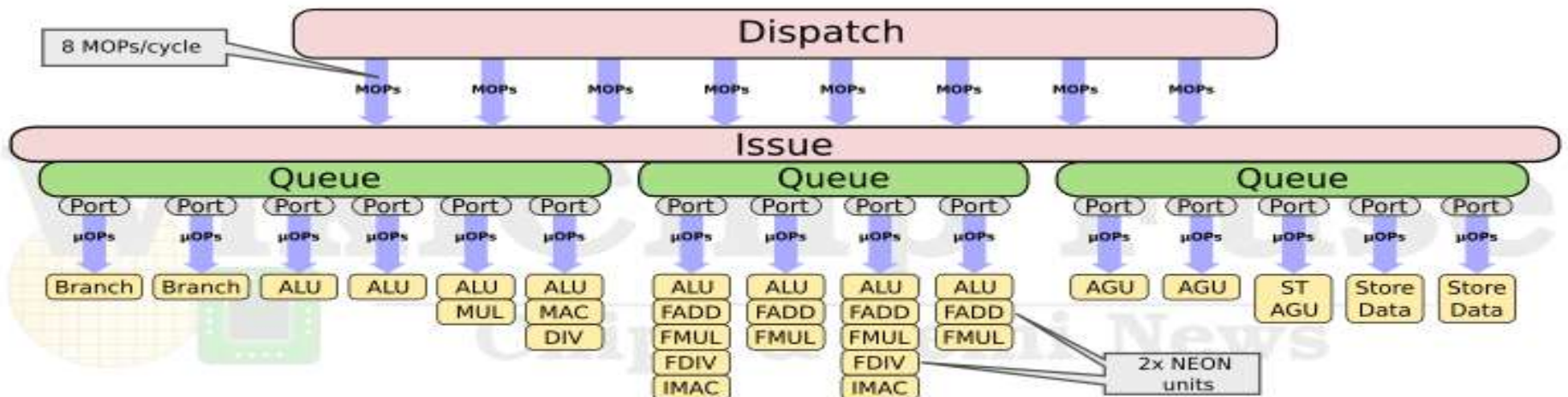


計算機組織 Computer Organization

Cortex-A78 Changes (vs. A77)



Cortex-X1 Changes (vs. A78)





關於現代高效能處理器，以下那

4.14 謬誤與陷阱

- ▣ 謬誤：管線化處理很容易
- ▣ 謬誤：管線處理的構想可以在不考慮半導體技術的情況下來設計
- ▣ 陷阱：不仔細思考指令集的設計將會嚴重影響管線化的處理
 - 變化很大的指令長度以及執行時間可以造成管線階段間的失衡並使得危障偵測極端困難
 - 繁複的各種定址模式可能造成各種問題

4.15 總結評論

- ▣ 處理器的數據通道與控制二者的設計都可以始於指令集架構以及對現行科技各項基本特性的瞭解
- ▣ 管線化處理改善了處理量而非指令本身的執行時間，或稱指令延遲(instruction latency)
- ▣ 相較於管線化，多重指令派發明顯地是著重於降低每道指令所需的時脈週期數(CPI)

Slido 互動問題

- ▣ 那些指令在本章被探討？多選題
- ▣ 那一種數位電路，能整合不同的功能於共用的功能方塊？單選題
- ▣ ALU control提到那些 ALU 功能？
- ▣ ALUop 有那三種不同工作方式？
- ▣ 那些指令有用到加法？那些
- ▣ Control 單元有那些控制輸出信號？
- ▣ 那一種控制信號只有某個指令使用？